

应用开发指南

发布日期 2022-12-30

前言

概述

本文用于指导开发人员基于现有模型、使用 SVP ACL (Smart Vision Processing Advanced Computing Language) 提供的 C 语言 API 库开发图像分析工具应用，用于实现目标识别、图像分类等功能。

产品版本

与本文档相对应的产品版本如下。

产品名称	产品版本
SS928	V100
SS626	V100

读者对象

本文档适用于基于 SVP ACL 接口进行应用开发的人员，通过本文档您可以达成：

- 了解 SVP ACL 的功能架构、基本概念以及接口的典型调用流程。
- 使用 SVP ACL 接口进行应用开发的基本流程和实现方法。
- 能够基于本文档中的样例，扩展进行其它应用的开发。

掌握以下经验和技能可以更好地理解本文档：

- 具备 C++/C 语言程序开发能力。
- 对机器学习、图像分析方法有一定的了解。

修改记录

发布日期	文档版本	修改说明
2022-12-30	05	第五次正式版本发布 "9.12.11.18 svp_acl_md1_get_input_default_stride"、 "9.12.11.19 svp_acl_md1_get_output_default_stride"及 "9.12.14 svp_acl_md1_config_attr"涉及修改
2022-10-28	04	第四次正式版本发布 4.4.2、9.8.1 和 9.8.6 小节, "LSTM"改为"Recurrent" 9.1 小节, 接口列表涉及修改 9.8.12 与 9.9.13 小节, 参数说明涉及修改 9.9.1 小节功能涉及修改
2022-07-30	03	第三次正式版本发布 9.8.5 小节, 修改“约束说明” 9.8.6 小节涉及修改 10.2.2 和 10.3.2 小节中步骤 2 添加“注意” 新增“11.2 SVP_NNN 离线模型保护”章节
2022-05-20	02	第二次正式版本发布 3.7.2、8.5、10.2.2 及 10.3.2 小节涉及修改 新增 “9.8.13 svp_acl_md1_get_dynamic_batch”和 “9.11.3 svp_acl_ext_get_md1_net_type”小节 10.2.1.4、10.3.1.4 小节, 修改目录结构 新增“11 FAQ”章节
2022-04-30	01	第一次正式版本发布 新增 svp_acl_md1_get_first_aapp_info 新增 svp_acl_aapp_input_format 和 svp_acl_aapp_info 10.2.2 及 10.3.2 小节涉及修改
2022-03-10	00B02	第二次临时版本发布。 3.7.2、8.5 小节涉及修改

发布日期	文档版本	修改说明
2022-01-06	00B01	第一次临时版本发布。

目 录

前 言.....i

1 使用约束.....1

2 新手指引.....2

2.1 文档结构总览.....2

2.2 表达约定.....3

2.2.1 接口命名规则.....3

2.2.2 接口类别.....3

3 简介.....4

3.1 什么是 SVP ACL.....4

3.2 基本概念.....5

3.3 进程、线程、Device、Context、Stream 之间的关系.....7

3.3.1 Device、Context、Stream 之间的关系.....7

3.3.2 线程、Context、Stream 之间的关系.....7

3.3.3 一个进程内多个线程间的 Context 迁移.....8

3.3.4 默认 Context 和默认 Stream 的使用场景.....9

3.3.5 多线程、多 stream 的性能说明.....10

3.4 SVP ACL 内存申请使用说明.....10

3.5 如何获取 Sample.....11

3.6 如何查看日志.....12

3.7 如何查看 Proc 信息.....12

3.7.1 概述.....12

3.7.2 Proc 信息说明.....13

4 接口调用流程介绍.....18

4.1 主要接口调用流程	18
4.2 运行管理资源申请	19
4.3 模型加载	21
4.4 模型执行	22
4.4.1 基本的模型执行流程	22
4.4.2 设置动态 Batch 和 Total_t	23
4.4.3 准备模型执行的输入/输出数据	25
4.5 同步等待	31
4.5.1 多 Device 场景	31
4.5.2 多 Stream 场景	32
4.5.3 Callback 场景	33
4.6 运行管理资源释放	34
5 开发流程	36
6 准备环境	38
6.1 准备开发环境	38
6.2 准备运行环境	39
7 开发首个应用	40
7.1 开发场景分析	40
7.1.1 开发场景	40
7.1.2 场景分析	40
7.2 创建代码目录	41
7.3 开发应用	41
7.3.1 资源初始化	41
7.3.1.1 SVP ACL 初始化	41
7.3.1.2 运行管理资源申请（单进程+单线程+单 Stream）	42
7.3.1.3 模型推理资源申请	43
7.3.2 读入图片数据	45
7.3.3 模型推理	46
7.3.3.1 示例代码	47
7.3.4 运行管理资源释放与 SVP ACL 去初始化	49

7.4 编译及运行应用.....	49
8 开发典型功能点的介绍	52
8.1 Stream 管理.....	52
8.1.1 原理介绍	52
8.1.2 单线程单 Stream	52
8.1.3 单线程多 Stream	53
8.1.4 多线程多 Stream	54
8.2 同步等待	55
8.2.1 原理介绍	55
8.2.2 关于 Stream 内任务的同步等待.....	55
8.2.3 关于 Device 的同步等待.....	56
8.3 模型推理	56
8.3.1 单 Batch+固定 shape+单模型同步推理.....	56
8.3.2 异步推理+callback 回调处理	57
8.3.3 多模型推理.....	62
8.4 带 cache 属性的内存管理.....	62
8.5 推理使用模式识别 CPU 算子	64
8.6 Profiling 性能数据采集	65
9 SVP ACL API 参考	70
9.1 接口列表	70
9.2 系统配置	75
9.2.1 svp_acl_init.....	75
9.2.2 svp_acl_finalize	76
9.3 Device 管理	77
9.3.1 svp_acl_rt_set_device	77
9.3.2 svp_acl_rt_reset_device.....	78
9.3.3 svp_acl_rt_get_device.....	78
9.3.4 svp_acl_rt_get_run_mode.....	79
9.3.5 svp_acl_rt_get_device_count.....	79
9.4 Context 管理	80
9.4.1 svp_acl_rt_create_context.....	80
9.4.2 svp_acl_rt_destroy_context.....	80
9.4.3 svp_acl_rt_set_current_context.....	81

9.4.4 svp_acl_rt_get_current_context.....	82
9.5 Stream 管理.....	82
9.5.1 svp_acl_rt_create_stream	82
9.5.2 svp_acl_rt_destroy_stream	83
9.6 同步等待.....	83
9.6.1 svp_acl_rt_synchronize_device.....	83
9.6.2 svp_acl_rt_synchronize_stream	84
9.6.3 svp_acl_rt_subscribe_report	84
9.6.4 svp_acl_rt_launch_callback.....	85
9.6.5 svp_acl_rt_process_report	86
9.6.6 svp_acl_rt_unsubscribe_report.....	86
9.6.7 svp_acl_rt_set_op_wait_timeout.....	87
9.7 内存管理.....	88
9.7.1 svp_acl_rt_malloc.....	88
9.7.2 svp_acl_rt_malloc_cached	89
9.7.3 svp_acl_rt_mem_flush.....	90
9.7.4 svp_acl_rt_mem_invalidate	91
9.7.5 svp_acl_rt_free	91
9.7.6 svp_acl_rt_malloc_host.....	92
9.7.7 svp_acl_rt_free_host.....	92
9.8 模型加载与执行.....	93
9.8.1 svp_acl_mdl_load_from_mem.....	93
9.8.2 svp_acl_mdl_execute	94
9.8.3 svp_acl_mdl_execute_async.....	95
9.8.4 svp_acl_mdl_unload.....	97
9.8.5 svp_acl_mdl_set_dynamic_batch_size	97
9.8.6 svp_acl_mdl_set_total_t.....	98
9.8.7 svp_acl_mdl_init_dump.....	99
9.8.8 svp_acl_mdl_set_dump.....	100
9.8.9 svp_acl_mdl_finalize_dump	101
9.8.10 svp_acl_mdl_set_config_opt.....	102
9.8.11 svp_acl_mdl_load_with_config.....	103
9.8.12 svp_acl_mdl_get_first_aapp_info	104
9.8.13 svp_acl_mdl_get_dynamic_batch.....	105
9.9 Profiling 配置	105
9.9.1 功能及约束说明.....	105
9.9.2 svp_acl_prof_init.....	108
9.9.3 svp_acl_prof_start	109
9.9.4 svp_acl_prof_stop	109
9.9.5 svp_acl_prof_finalize.....	110

9.9.6	svp_acl_prof_model_subscribe	110
9.9.7	svp_acl_prof_model_unsubscribe	111
9.9.8	svp_acl_prof_get_op_desc_size	111
9.9.9	svp_acl_prof_get_op_num	112
9.9.10	svp_acl_prof_get_op_type_len.....	113
9.9.11	svp_acl_prof_get_op_type.....	113
9.9.12	svp_acl_prof_get_op_name_len.....	114
9.9.13	svp_acl_prof_get_op_name	115
9.9.14	svp_acl_prof_get_op_start	116
9.9.15	svp_acl_prof_get_op_end	116
9.9.16	svp_acl_prof_get_op_duration	117
9.9.17	svp_acl_prof_get_model_id.....	118
9.10	获取数据大小.....	118
9.10.1	svp_acl_data_type_size	118
9.11	SoC 扩展接口	119
9.11.1	svp_acl_ext_process_aacpu_task	119
9.11.2	svp_acl_ext_get_mdl_aacpu_task_num.....	119
9.11.3	svp_acl_ext_get_mdl_net_type	120
9.12	数据类型及其操作接口.....	121
9.12.1	svp_acl_error.....	121
9.12.2	svp_acl_data_type.....	128
9.12.3	svp_acl_float16.....	128
9.12.4	svp_acl_format.....	128
9.12.5	svp_acl_rt_context.....	129
9.12.6	svp_acl_rt_stream	129
9.12.7	svp_acl_rt_run_mode.....	129
9.12.8	svp_acl_mdl_io_dims	129
9.12.9	svp_acl_data_buffer	130
9.12.9.1	svp_acl_create_data_buffer	130
9.12.9.2	svp_acl_destroy_data_buffer.....	130
9.12.9.3	svp_acl_get_data_buffer_addr	131
9.12.9.4	svp_acl_get_data_buffer_size	131
9.12.9.5	svp_acl_get_data_buffer_stride.....	132
9.12.9.6	svp_acl_update_data_buffer	132
9.12.10	svp_acl_mdl_dataset.....	133
9.12.10.1	svp_acl_mdl_create_dataset.....	133
9.12.10.2	svp_acl_mdl_destroy_dataset.....	133
9.12.10.3	svp_acl_mdl_add_dataset_buffer.....	134
9.12.10.4	svp_acl_mdl_get_dataset_num_buffers	134
9.12.10.5	svp_acl_mdl_get_dataset_buffer.....	135
9.12.11	svp_acl_mdl_desc	135

9.12.11.1	svp_acl_mdl_create_desc	135
9.12.11.2	svp_acl_mdl_destroy_desc	135
9.12.11.3	svp_acl_mdl_get_desc	136
9.12.11.4	svp_acl_mdl_get_num_inputs	136
9.12.11.5	svp_acl_mdl_get_num_outputs	137
9.12.11.6	svp_acl_mdl_get_input_size_by_index	137
9.12.11.7	svp_acl_mdl_get_output_size_by_index	138
9.12.11.8	svp_acl_mdl_get_input_dims	138
9.12.11.9	svp_acl_mdl_get_output_dims	139
9.12.11.10	svp_acl_mdl_get_input_name_by_index	140
9.12.11.11	svp_acl_mdl_get_output_name_by_index	140
9.12.11.12	svp_acl_mdl_get_input_format	141
9.12.11.13	svp_acl_mdl_get_output_format	141
9.12.11.14	svp_acl_mdl_get_input_data_type	142
9.12.11.15	svp_acl_mdl_get_output_data_type	143
9.12.11.16	svp_acl_mdl_get_input_index_by_name	143
9.12.11.17	svp_acl_mdl_get_output_index_by_name	144
9.12.11.18	svp_acl_mdl_get_input_default_stride	144
9.12.11.19	svp_acl_mdl_get_output_default_stride	145
9.12.12	svp_acl_prof_config	146
9.12.12.1	svp_acl_prof_create_config	146
9.12.12.2	svp_acl_prof_destroy_config	147
9.12.13	svp_acl_prof_subscribe_config	148
9.12.13.1	svp_acl_prof_create_subscribe_config	148
9.12.13.2	svp_acl_prof_destroy_subscribe_config	148
9.12.14	svp_acl_mdl_config_attr	149
9.12.15	svp_acl_mdl_config_handle	150
9.12.15.1	svp_acl_mdl_create_config_handle	150
9.12.15.2	svp_acl_mdl_destroy_config_handle	150
9.12.16	svp_acl_aapp_input_format	151
9.12.17	svp_acl_aapp_info	151
10	SVP ACL 样例使用指导	153
10.1	总体说明	153
10.2	基于 Caffe ResNet-50 网络实现图片分类（同步推理）	153
10.2.1	样例介绍	153
10.2.1.1	获取样例	153
10.2.1.2	功能描述	153
10.2.1.3	原理介绍	154
10.2.1.4	目录结构	155

10.2.2 编译及运行应用.....	156
10.3 基于 Caffe ResNet-50 网络实现图片分类（异步推理）	160
10.3.1 样例介绍	160
10.3.1.1 获取样例	160
10.3.1.2 功能描述	160
10.3.1.3 原理介绍	161
10.3.1.4 目录结构	163
10.3.2 编译及运行应用.....	164
11 FAQ	168
11.1 SS928V100 解决方案使用 SVP_NNN 模块注意事项	168
11.2 SVP_NNN 离线模型保护	168

插图目录

图 3-1 逻辑架构图.....	5
图 4-1 接口调用流程图.....	18
图 4-2 运行管理资源申请流程.....	20
图 4-3 模型加载流程	21
图 4-4 基本的模型推理流程	22
图 4-5 模型输入/输出数据	25
图 4-6 模型执行的输入/输出数据准备流程.....	26
图 4-7 svp_acl_md5_dataset 类型与 svp_acl_data_buffer 类型的关系.....	27
图 4-8 模型输入/输出数据排布（通道为 2，batch 为 2 示意图）	28
图 4-9 YUV420SP/YUV420SP 数据排布（通道为 2，frame 为 2 示意图）	29
图 4-10 检测网阈值输入数据排布	29
图 4-11 检测框结果数据排布（通道为 2，chn 为 2 示意图）	30
图 4-12 同步等待流程多 Device 场景.....	31
图 4-13 同步等待流程多 Stream 场景	32
图 4-14 同步等待流程 Callback 场景模型推理	33
图 4-15 运行管理资源释放流程.....	34
图 5-1 开发流程（不包含数据预处理）	36
图 10-1 Sample 示例	154
图 10-2 Sample 示例	160

表格目录

表 3-1 概念介绍5

表 3-2 Sample 列表 11

表 7-1 头文件与库文件的对应关系..... 50

表 9-1 返回码列表..... 121

表 9-2 图像输入维度定义规则..... 139

表 9-3 图像输入数据类型定义规则..... 142

表 10-1 Sample 中的关键功能点 155

表 10-2 Sample 中的关键功能点 162

1 使用约束

- 不支持使用 `fork` 函数创建多个进程，且在进程中调用 SVP ACL 接口的场景，否则进程运行时会报错或者卡死。
- 对于创建类接口（例如：`svp_acl_rt_create_context`、`svp_acl_rt_create_stream`、`svp_acl_create_data_bufferr` 等），用户调用该类接口创建对应的资源后，资源使用完成后，建议及时调用对应的销毁类接口（例如：`svp_acl_rt_destroy_context`、`svp_acl_rt_destroy_stream`、`svp_acl_destroy_data_buffer` 等），否则，程序可能会异常。
- 对于销毁类接口（例如：`svp_acl_rt_destroy_context`、`svp_acl_rt_destroy_stream`、`svp_acl_rt_free`、`svp_acl_destroy_data_buffer` 等），用户调用该类接口后，不能继续使用已释放或销毁的资源，建议用户调用销毁类接口后，将相关资源设置为无效值（例如，置为 `NULL`）。
- 一个 Device 上默认最多只能支持 64 个用户进程。

2 新手指引

2.1 文档结构总览

本文档分为以下几章：

- [前言](#)：介绍文档概述和读者对象。
- [1 使用约束](#)：介绍 SVP ACL 的总体使用约束。
- [2 新手指引](#)：介绍文档结构、接口命名规则。
- [3 简介](#)：介绍 SVP (Smart Vision Platform) ACL (Advanced Computing Language) 的基础知识，包括功能、基本概念、各概念之间的关系、如何查看日志等。
- [4 接口调用流程介绍](#)：介绍各场景下的 SVP ACL 接口调用流程。
- [5 开发流程](#)：介绍使用 SVP ACL 提供的接口开发应用的基本步骤。
- [6 准备环境](#)：介绍准备开发环境和板端环境时需要参考的文档。
- [7 开发首个应用](#)：以开发图片分类应用（不包含抠图、缩放、解码等数据预处理的开发）为例，按照开发流程，结合示例代码，介绍各步骤的基本原理。
- [8 开发典型功能点的介绍](#)：SVP ACL 各功能点的详细介绍。
- [9 SVP ACL API 参考](#)：介绍 API 的功能、原型、参数等。
- [10 SVP ACL 样例使用指导](#)：介绍如何使用 SVP ACL 提供的样例。

2.2 表达约定

2.2.1 接口命名规则

接口命名同时满足如下规则：

1. 规则 1: `svp_acl+接口类别缩写+操作动词+对象`
2. 规则 2: 操作动词和对象均采用小写

2.2.2 接口类别

接口类别	缩写	描述
runtime	rt	表示运行管理类的接口。
model	mdl	表示模型推理类的接口

注：

1. 缩写原则上不超过 4 个字母
2. 在接口命名中，如果类别与操作对象重叠时，操作动词后的对象将省略。

如：`svp_acl_mdload_from_mem`，表示 model 类接口，这个接口表示含义是 load model from mem，因此在接口命名中 Load 后面 mdl 将被省略。

3 简介

本文用于指导开发人员基于现有模型、使用 SVP ACL (Smart Vision Processing Advanced Computing Language) 提供的 C 语言 API 库开发*图像分析工具*应用，用于实现目标识别、图像分类等功能。

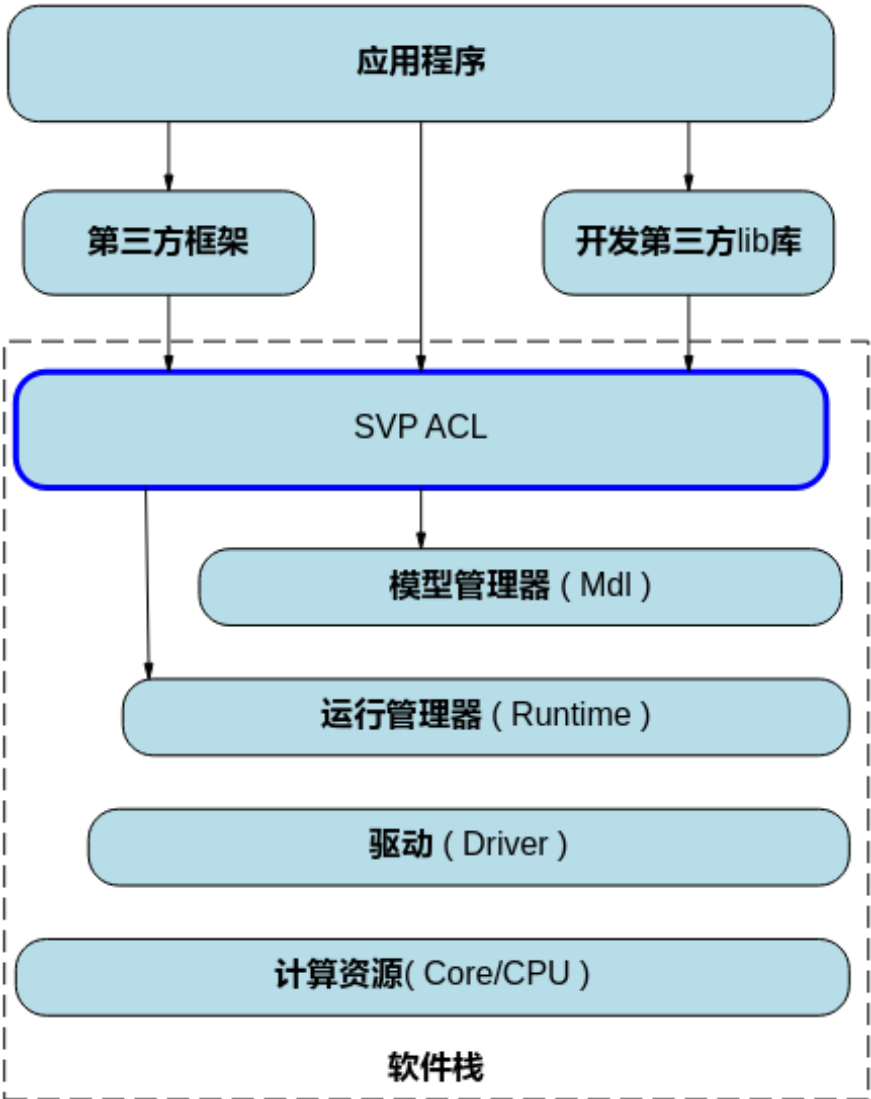
3.1 什么是 SVP ACL

SVP ACL (Smart Vision Processing Advanced Computing Language) 提供 Device 管理、Context 管理、Stream 管理、内存管理、模型加载与执行等 C 语言 API 库供用户开发*图像分析工具*应用，用于实现目标识别、图像分类等功能。用户可以通过第三方框架调用 SVP ACL 接口，以便使用 SoC 的计算能力；用户还可以使用 SVP ACL 封装实现第三方 lib 库，以便提供 SoC 的运行管理、资源管理能力。

在运行应用时，SVP ACL 调用 mdl 管理器提供的接口实现模型的加载与执行、调用运行管理器的接口实现 Device 管理/Context 管理/Stream 管理/内存管理等。

计算资源层是 SoC 的硬件算力基础，主要完成*图像分析工具*的矩阵相关计算、完成控制算子/标量/向量等通用计算和执行控制功能、完成图像和视频数据的预处理，为*图像分析工具*计算提供了执行上的保障。

图3-1 逻辑架构图



3.2 基本概念

表3-1 概念介绍

概念	描述
同步/异步	本文中提及的同步、异步是站在调用者和执行者的角度，在当前场景下，若在板端环境调用接口后不等待 Device 执行完成再返回，则表示板端环境的调度是异步的；若在板端环境调用接口后需等待 Device 执行完成再返回，则表示板端环境

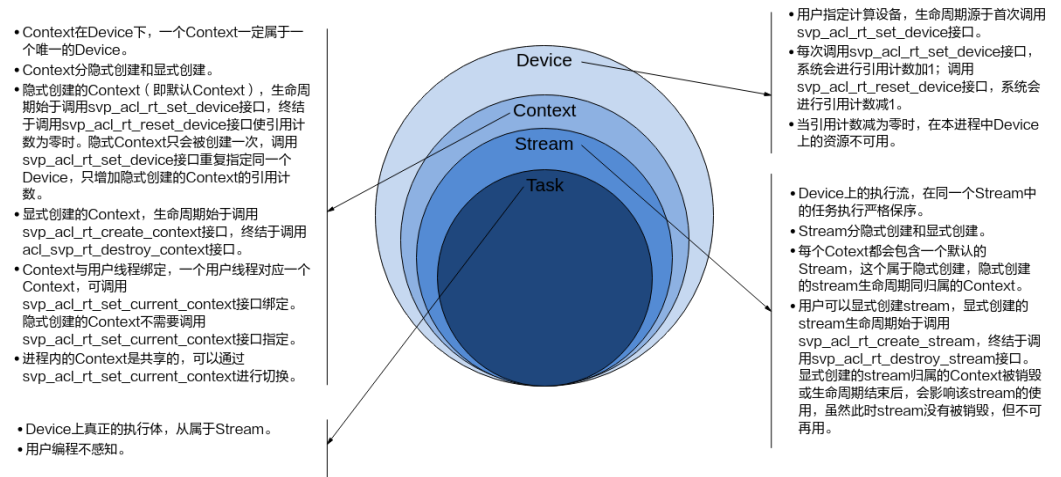
概念	描述
	的调度是同步的。
进程/线程	本文中提及的进程、线程，若无特别注明，则表示板端环境上的进程、线程。
Device	Device 表示板端环境上的 图像分析引擎 。
Context	Context 作为一个容器，管理了所有对象（包括 Stream、设备内存等）的生命周期。不同 Context 的 Stream 之间、不同的 Context 之间是完全隔离的，无法建立同步等待关系。 Context 分为两种： - 默认 Context：调用 svp_acl_rt_set_device 接口指定用于运算的 Device 时，系统会自动隐式创建一个默认 Context，一个 Device 对应一个默认 Context，默认 Context 不能通过 svp_acl_rt_destroy_context 接口来释放。 - 显式创建的 Context：推荐，在进程或线程中调用 svp_acl_rt_create_context 接口显式创建一个 Context。
Stream	Stream 用于维护一些异步操作的执行顺序，确保按照应用程序中的代码调用顺序在 Device 上执行。 Stream 分两种： - 默认 Stream：调用 svp_acl_rt_set_device 接口指定用于运算的 Device 时，系统会自动隐式创建一个默认 Stream，一个 Device 对应一个默认 Stream，默认 Stream 不能通过 svp_acl_rt_destroy_stream 接口来释放。 - 显式创建的 Stream：推荐，在进程或线程中调用 svp_acl_rt_create_stream 接口显式创建一个 Stream。
动态 Batch/动态分辨率	在某些场景下，模型每次输入的 Batch 数或分辨率是不固定的，如检测出目标后再执行目标识别网络，由于目标个数不固定导致目标识别网络输入 BatchSize 不固定。 动态 Batch：用户执行推理时，其 Batch 数是动态可变的。 - 动态分辨率：不支持动态分辨率。

概念	描述
动态维度（ND 格式）	为了支持 Transformer 等网络在输入格式的维度不确定的场景，需要支持 ND 格式下任意维度的动态设置。 ND 表示支持任意格式，当前 N≠4。
通道	在 RGB 色彩模式下，图像通道就是指单独的红色 R、绿色 G、蓝色 B 部分。也就是说，一幅完整的图像，是由红色绿色蓝色三个通道组成的，它们共同作用产生了完整的图像。

3.3 进程、线程、Device、Context、Stream 之间的关系

各基本概念的介绍请参见 3.2 基本概念。

3.3.1 Device、Context、Stream 之间的关系



3.3.2 线程、Context、Stream 之间的关系

- 一个用户线程一定会绑定一个 Context，所有 Device 的资源使用或调度，都必须基于 Context。
- 一个线程中当前会有一个唯一的 Context 在用，Context 中已经关联了本线程要使用的 Device。

- 可以通过 `sva_acl_rt_set_current_context` 进行 Device 的快速切换。示例代码如下，仅供参考，不可以直接拷贝编译运行：

```
...
sva_acl_rt_create_context(&ctx1, 0);
sva_acl_md_execute(md1, input1, output1);
sva_acl_rt_create_context(&ctx2, 1);

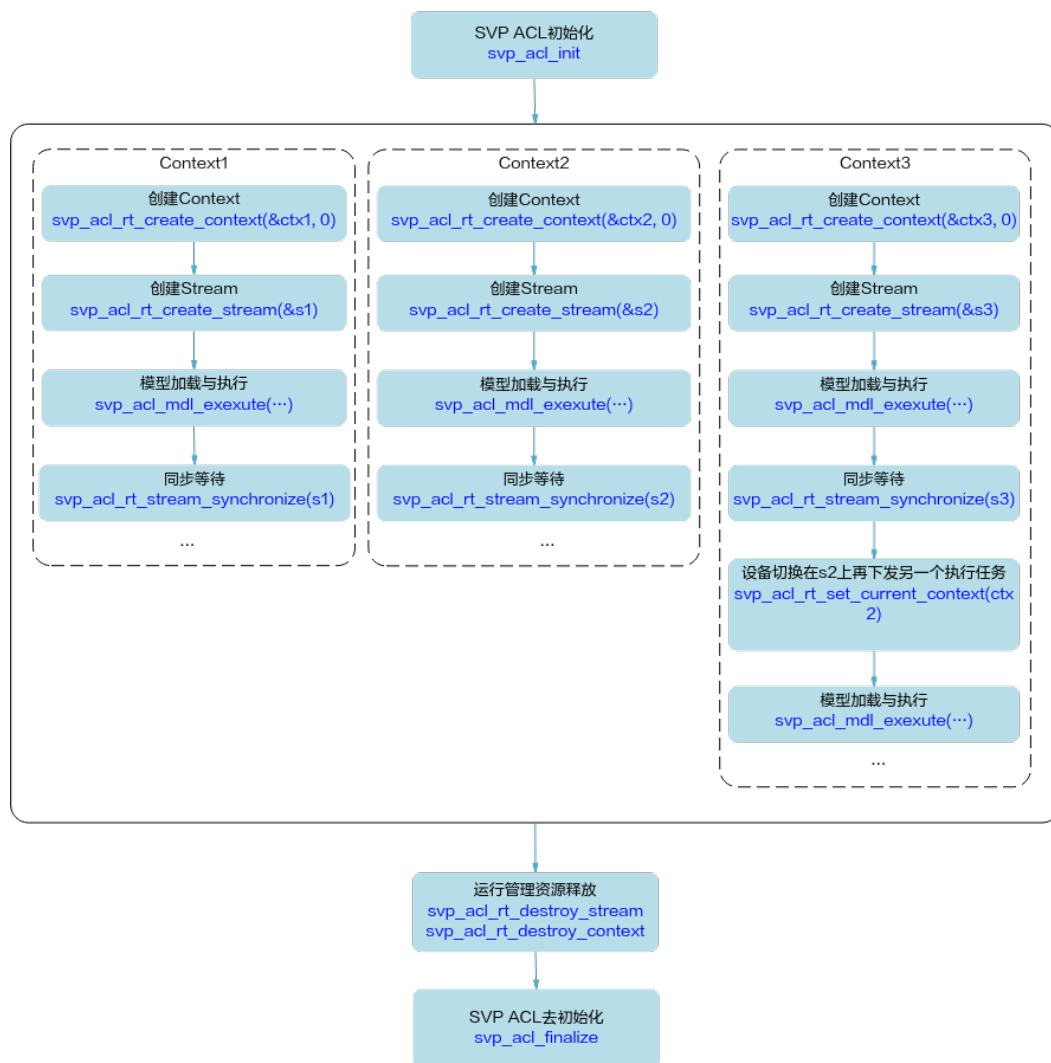
/*在当前线程中，创建ctx2后，当前线程对应的Context切换为ctx2，对应在Device 1进行后续的计算任务，本例中将在Device 1上进行md2的执行调用 */
sva_acl_md_execute(md2, input2, output2);
sva_acl_rt_set_current_context(ctx1);

/*在当前线程中，通过Context切换，使后续模型计算任务在对应的Device 0上进行*/
sva_acl_md_execute(md3, input3, output3);
...
```

- 一个线程中可以创建多个 Stream，不同的 Stream 上计算任务是可以并行执行；多线程场景下，也可以每个线程创建一个 Stream，线程之间的 Stream 在 Device 上相互独立，每个 Stream 内部的任务是按照 Stream 下发的顺序执行。
- 多线程的调度依赖于运行应用的操作系统调度，Device 侧多 Stream 调度，由 Device 上调度组件进行调度。

3.3.3 一个进程内多个线程间的 Context 迁移

- 一个进程中可以创建多个 Context，但一个线程同一时刻只能使用一个 Context。
- 线程中创建的多个 Context，线程缺省使用最后一次创建的 Context。
- 进程内创建的多个 Context，可以通过 `sva_acl_rt_set_current_context` 设置当前需要使用的 Context。



3.3.4 默认 Context 和默认 Stream 的使用场景

- Device 上执行操作下发前，必须有 Context 和 Stream，这个 Context、Stream 可以显式创建，也可以隐式创建。隐式创建的 Context、Stream 就是默认 Context、默认 Stream。
默认 Stream 作为接口入参时，直接传 NULL。
- 默认 Context 不允许用户执行 `svp_acl_rt_get_current_context` 或 `svp_acl_rt_set_current_context` 操作，也不允许执行 `svp_acl_rt_destroy_context` 操作。
- 默认 Context、默认 Stream 一般适用于简单应用，用户仅仅需要一个 Device 的计算场景下。多线程应用程序建议全部使用显式创建的 Context 和 Stream。

示例代码如下，仅供参考，不可以直接拷贝编译运行：

...

```
svp_acl_init(...);
svp_acl_rt_set_device(0);

/*已经创建了一个default ctx，在default ctx中创建了一个default stream，并且在当前线程可用*/
...
svp_acl_md1_execute_async(md1,input1,output1,NULL); //最后一个NULL表示在default stream上
执行模型md1

svp_acl_md2_execute_async(md2,input2,output2,NULL); //最后一个NULL表示在default stream上执
行模型md2

svp_acl_rt_synchronize_stream(NULL);

/*等待计算任务全部完成（md1、md2执行结束），用户根据需要获取计算任务的输出结果*/
...
svp_acl_rt_reset_device(0); //释放计算设备0，对应的default ctx及default stream生命周期也终止
```

3.3.5 多线程、多 stream 的性能说明

- 线程调度依赖运行的操作系统，Stream 上下发了任务后，Stream 的调度由 Device 的调度单元调度，但如果一个进程内的多 Stream 上的任务在 Device 存在资源争抢的时候，性能可能会比单 Stream 低。
- 当前芯片有不同的执行部件，如模式识别 Core、模式识别 CPU 等，对应使用不同执行部件的任务，**建议多 Stream 的创建按照算子执行引擎划分**。
- 单线程多 Stream 与多线程多 Stream（进程属于多线程，每个线程中一个 Stream）性能上哪个更优，具体取决于应用本身的逻辑实现，一般来说前者性能略好，原因是相对后者，应用层少了线程调度开销。

3.4 SVP ACL 内存申请使用说明

用户内存管理有两种管理方式：

1. 独立内存管理，根据需要单独申请所需的内存，内存不做拆分或者二次分配。
2. 内存池管理内存，用户一次性申请一块较大内存，并在使用时从这块较大内存中二次分配所需内存。

在内存二次分配时，使用如下接口从内存池申请对应内存，由于接口对申请的内存地址、大小有约束，在内存池管理时，需要关注，否则容易出现内存越界。

接口	用途	输入内存/输出内存
svp_acl_rt_malloc	申请 Device 上的内存，同步接口。	- 使用 svp_acl_rt_malloc 接口申请的内存，需要通过 svp_acl_rt_free 接口释放内存。 - 频繁调用 svp_acl_rt_malloc 接口申请内存、调用 svp_acl_rt_free 接口释放内存，会损耗性能，建议用户提前做内存预先分配或二次管理，避免频繁申请/释放内存。
svp_acl_rt_malloc_host	申请 Host 或 Device 上的内存，Device 上的内存按普通页申请。同步接口。	- 使用 svp_acl_rt_malloc_host 接口申请的内存，需要通过 svp_acl_rt_free_host 接口释放内存。 - 如果没有 Host 端，调用该接口会获取 Device 侧内存，也可调用 svp_acl_rt_free 释放。 - 频繁调用 svp_acl_rt_malloc_host 接口申请内存、调用 svp_acl_rt_free_host 接口释放内存，会损耗性能，建议用户提前做内存预先分配或二次管理，避免频繁申请/释放内存。

3.5 如何获取 Sample

当前 SVP ACL 提供的样例如[表 3-2](#) 所示。

表3-2 Sample 列表

Sample	基本功能	样例介绍、编译及运行运行
resnet50_imagenet_classification	基于 Caffe ResNet-50 网络实现图片分类（同步推理）	10.2 基于 Caffe ResNet-50 网络实现图片分类（同步推理）
resnet50_async_imagenet_classification	基于 Caffe ResNet-50 网络实现图片分类（异步推理）	10.3 基于 Caffe ResNet-50 网络实现图片分类（异步推理）

Sample	基本功能	样例介绍、编译及运行运行
		理)

切至发布包 Sample 目录，解压 samples.tar.gz

```
tar -zxvf samples.tar.gz
```

3.6 如何查看日志

在控制台上可以使用 cat 命令查看信息，cat /dev/logmpp 查看错误日志。

3.7 如何查看 Proc 信息

3.7.1 概述

调试信息采用了 Linux 下的 proc 文件系统，可实时反映当前系统的运行状态，所记录的信息可供问题定位及分析时使用。

【文件目录】

/proc/umap

【信息查看方法】

- 在控制台上可以使用 cat 命令查看信息，cat /proc/umap/svp_nnn；也可以使用其他常用的文件操作命令，例如 cp /proc/umap/svp_nnn ./，将文件拷贝到当前目录。
- 在应用程序中可以将上述文件当作普通只读文件进行读操作，例如 fopen、fread 等。

 说明

参数在描述时有以下 2 种情况需要注意：

- 取值为{0, 1}的参数，如未列出具体取值和含义的对应关系，则参数为 1 时表示肯定，为 0 时表示否定。
- 取值为{aaa, bbb, ccc}的参数，未列出具体取值和含义的对应关系，但可直接根据取值 aaa、bbb 或 ccc 判断参数含义。

3.7.2 Proc 信息说明

【调试信息】

```
# cat /proc/umap/svp_nnn
[SVP_NNM] Version: Version: [xxxxVx.x.x.x B0xx Release], Build Time[mm dd yyyy, hh:mm:ss]

-----svp_nnn module param-----
nnn_save_power    max_task_node_num
0                  512

-----svp_nnn resource info-----
free_model_num
63
device_id  free_stream_num  free_report_num  free_task_node_num
0          0               127               128                511

-----svp_nnn busy stream info-----
device_id  stream_id  report_id  block_type  send_task_num
0          0      0          -1          1              0
timeout_err_cnt hw_err_cnt  aacpu_err_cnt
0              0          0              0
model_task_handle  model_task_handle_wrap  model_task_finish
1                  0              0              0
model_task_finish_wrap
0
callback_task_handle  callback_task_handle_wrap  callback_task_finish
0                      0              0
callback_task_finish_wrap
0

-----svp_nnn sync model task info-----
device_id  task_send_num  task_send_num_wrap  task_finish_num  task_finish_num_wrap
0          1              0              1              0

-----svp_nnn irq info-----
device_id  irq_cnt_last_sec  max_irq_cnt_per_sec  total_irq_cnt
0          17              17              181
```

```
cur_irq_time    max_irq_time    irq_time_last_sec    max_irq_time_per_sec
                8                24                180                180
total_irq_time
                2445

-----svp_nnn runtime info-----
device_id    hw_status    last_stream_id    last_task_node_id    net_seg_idx
              0              0              0              0              446
net_seg_num
              610

timeout_err_cnt    hw_err_cnt    aacpu_err_cnt
0                0                0
last_hw_task_time    hw_utilization    total_running_time
                  13                0%                897
```

【调试信息分析】

记录当前 SVP 图像分析引擎模块参数信息，资源信息，被调用 stream 信息，中断信息和运行时状态信息。

【参数说明】

参数		描述
svp_nnn module param SVP 图像分析引擎 模块参数	nnn_save_power	低功耗标志。 0：关闭低功耗； 1：打开低功耗。
	max_task_node_num	最大任务节点个数，范围[1, 4096]，默认为 512 个，用户可通过模块参数 svp_nnn_max_task_node_num 配置。配置方式是在加载 svp_nnn 的 ko 时，使用 "svp_nnn_max_task_node_num=512" 参数并修改"512"的值，将其配置成需要设置的数值。
svp_nnn resource info SVP 图像分析引擎资源信息	free_model_num	剩余可加载的模型个数。
	device_id	设备 ID。
	free_stream_num	剩余可申请的 Stream 个数。

参数		描述
	free_report_num	剩余可申请的 report 个数。
	free_task_node_num	剩余可申请的 task 节点个数。
svp_nnn busy stream info SVP 图像分析 引擎运行 stream 的信息	device_id	设备 ID。
	stream_id	Stream ID。
	report_id	Stream 注册的 report ID。如果为-1, 则 Stream 没有注册 report。
	block_type	阻塞类型。 0: 没有被阻塞; 1: 被逻辑任务阻塞; 2: 被 模式识别/CPU 任务阻塞; 3: 被 Callback 任务阻塞。
	send_task_num	下发到 Stream 上的任务个数。
	timeout_err_cnt	Stream 上发生超时错误的个数。
	hw_err_cnt	Stream 上发生逻辑错误的个数。
	aacpu_err_cnt	Stream 上发生 模式识别/CPU 任务执行错误的个数。
	model_task_handle	模型推理任务 handle。
	model_task_handle_wrap	模型推理任务 handle 发生回写次数。
	model_task_finish	已完成模型推理任务个数。
	model_task_finish_wrap	已完成模型推理任务数发生回写的次数。
	callback_task_handle	Callback 任务 handle。
	callback_task_handle_wrap	Callback 任务 handle 发生回写次数。
	callback_task_finish	已完成 Callback 任务个数。
	callback_task_finish_wrap	已完成 Callback 任务数发生回写的次

参数		描述
	p	数。
svp_nnn irq info SVP 图像分析引擎中断信息	device_id	设备 ID。
	irq_cnt_last_sec	最近 1 秒内中断执行次数。
	max_irq_cnt_per_sec	历史上的 1 秒内最大的中断执行次数。
	total_irq_cnt	产生中断的总次数。
	cur_irq_time	最近一次执行中断的执行耗时。 单位：us。
	max_irq_time	执行一次中断的最大耗时。 单位：us。
	irq_time_last_sec	最近一秒执行中断的执行耗时。 单位：us。
	max_irq_time_per_sec	历史上一秒内执行中断的最大耗时。 单位：us。
	total_irq_time	中断处理总时间。 单位：us。
svp_nnn sync model task info SVP 图像分析引擎同步模型任务信息	device_id	设备 ID。
	task_send_num	下发任务个数。
	task_send_num_wrap	下发任务数发生回绕的次数。
	task_finish_num	完成任务个数。
	task_finish_num_wrap	完成任务数发生回绕的次数。
svp_nnn runtime info SVP 图像分析引擎设备运行信息	device_id	设备 ID。
	hw_status	硬件所处状态。 0：空闲。 1：工作。

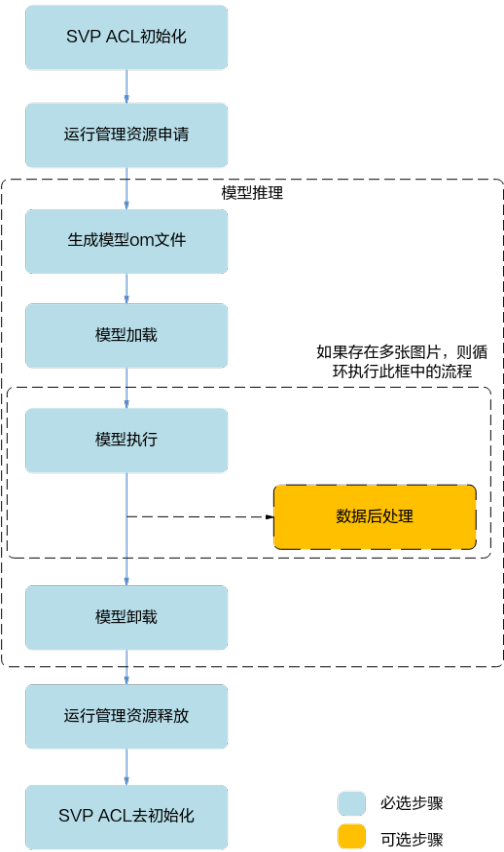
参数		描述
	last_stream_id	最近一次下发任务到图像分析引擎的 Stream id。
	last_task_node_id	最近一次在图像分析引擎上执行的任务节点 id。
	net_seg_idx	最近一次图像分析引擎任务被执行的网络段索引。
	net_seg_net	最近一次图像分析引擎任务含有的网络段数。
	timeout_err_cnt	设备上发生超时错误的个数。
	hw_err_cnt	设备上发生逻辑错误的个数。
	aacpu_err_cnt	设备上发生模式识别/CPU 任务执行错误的个数
	last_hw_task_time	最近一次图像分析引擎执行任务的耗时。
	hw_utilization	最近 1s 内硬件执行时间所占的比例。
	total_running_time	设备运行总时间。 单位：s

备注：loadSS626V100/loadSS928V100 脚本位置在版本包目录/mpp/out/ko/路径下。其中 loadSS928V100 脚本默认加载 PQP 的 ko，PQP 与 svp_nnn 是互斥的，不能同时加载，需要手动修改 loadSS928V100 脚本加载 svp_nnn 的 ko。

4 接口调用流程介绍

4.1 主要接口调用流程

图4-1 接口调用流程图



上图根据应用开发中的典型功能抽象出主要的接口调用流程，例如，如果需要通过模型推理的功能，则需要先加载模型，模型推理结束后，则需要卸载模型；如果模型推

理后，需要从推理结果中查找最大置信度的类别标识对图片分类，则需要数据后处理。

1. SVP ACL 初始化：调用 `svp_acl_init` 接口实现初始化 SVP ACL。
2. 运行管理资源申请：依次申请运行管理资源：`Device`、`Context`、`Stream`。具体流程，请参见“4.2 运行管理资源申请”。
3. 模型推理。
 - a. 生成模型 om 文件：需使用 ATC 工具将第三方网络（例如，Caffe ResNet-50 网络）转换为适配 SoC 的离线模型（*.om 文件），请参见《ATC 工具使用指南》。
 - b. 模型加载：模型推理前，需要先将对应的模型加载到系统中。具体流程，请参见“4.3 模型加载”。
 - c. 模型执行：使用模型实现图片分类、目标识别等功能，目前 SVP ACL 提供同步推理接口和异步推理接口，支持动态 Batch 场景。具体流程，请参见“4.4 模型执行”。
 - d. （可选）数据后处理：处理模型推理的结果，此处根据用户的实际需求来处理推理结果，例如用户可以将获取到的推理结果写入文件、从推理结果中找到每张图片最大置信度的类别标识等。
 - e. 模型卸载：调用 `svp_acl_md1_unload` 接口卸载模型。
4. 运行管理资源释放：所有数据处理都结束后，需要依次释放运行管理资源：`Stream`、`Context`、`Device`。具体流程，请参见“4.6 运行管理资源释放”。
5. SVP ACL 去初始化：调用 `svp_acl_finalize` 接口实现 SVP ACL 去初始化。

说明

在应用开发过程中，各环节都涉及内存的申请与释放、数据传输（通过内存复制实现）、数据类型的创建与销毁，因此未在图中一一标识，关于内存申请与释放、内存复制的接口请参见“9.7 内存管理”，数据类型的创建与销毁的接口请参见“9.12 数据类型及其操作接口”。

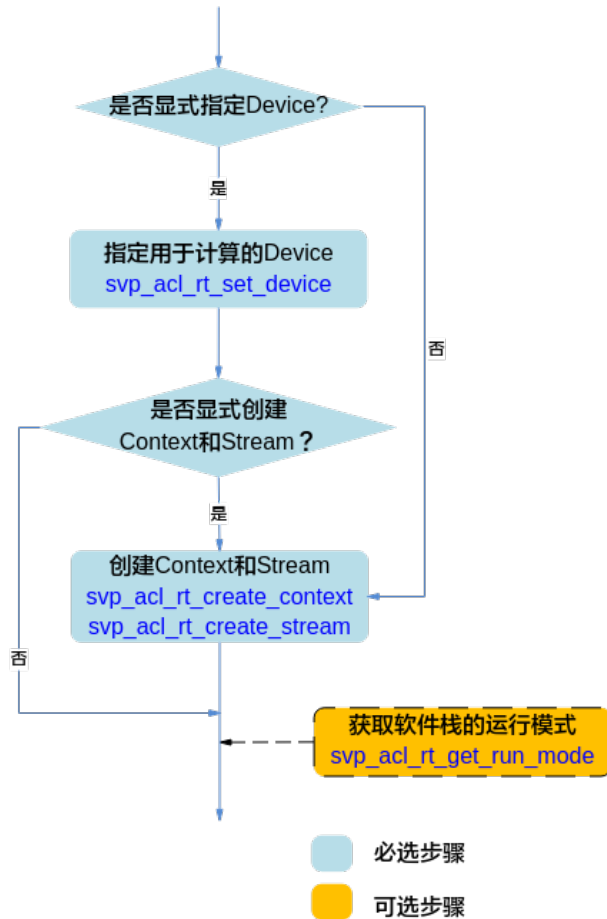
4.2 运行管理资源申请

您需要依次申请运行管理资源，包括：`Device`、`Context`、`Stream`。其中创建 `Context`、`Stream` 的方式分为隐式创建和显式创建。

- 隐式创建 `Context` 和 `Stream`：适合简单、无复杂交互逻辑的应用，但缺点在于，在多线程编程中，每个线程都使用默认 `Context` 或默认 `Stream`，默认 `Stream` 中任务的执行顺序取决于操作系统线程调度的顺序。

- 显式创建 Context 和 Stream：**推荐显式**，适合大型、复杂交互逻辑的应用，且便于提高程序的可读性、可维护性。

图4-2 运行管理资源申请流程



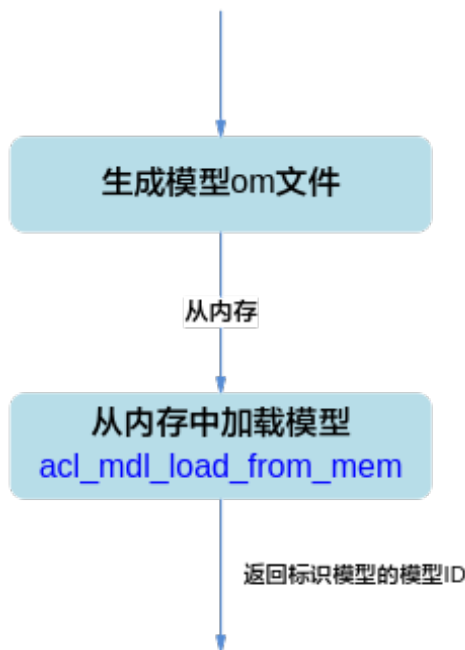
1. 申请运行管理资源时，需按顺序依次申请：Device、Context、Stream。
 - 调用 `svp_acl_rt_set_device` 接口显式指定用于运算的 Device。
 - 调用 `svp_acl_rt_create_context` 接口显式创建 Context，调用 `svp_acl_rt_create_stream` 接口显式创建 Stream。
 - 不显式创建 Context 和 Stream，系统会使用默认 Context、默认 Stream，该默认 Context、默认 Stream 是在调用 `svp_acl_rt_set_device` 接口时隐式创建的。
默认 Stream 作为接口入参时，直接传 NULL。
 - 不显式指定用于运算的 Device。

调用 `svp_acl_rt_create_context` 接口显式创建 Context，调用 `svp_acl_rt_create_stream` 接口显式创建 Stream。系统在显式创建 Context 时，系统内部会调用 `svp_acl_rt_set_device` 接口指定运行的 Device，Device ID 通过 `svp_acl_rt_create_context` 接口传入。

2. （可选）调用 `svp_acl_rt_get_run_mode` 接口获取软件栈的运行模式，根据运行模式来判断后续的内存申请接口调用逻辑，如果查询结果为 `SVP_ACL_HOST`，则数据传输时涉及申请 Host 上的内存；如果查询结果为 `SVP_ACL_DEVICE`，则数据传输时不涉及申请 Host 上的内存，仅需申请 Device 上的内存。

4.3 模型加载

图4-3 模型加载流程

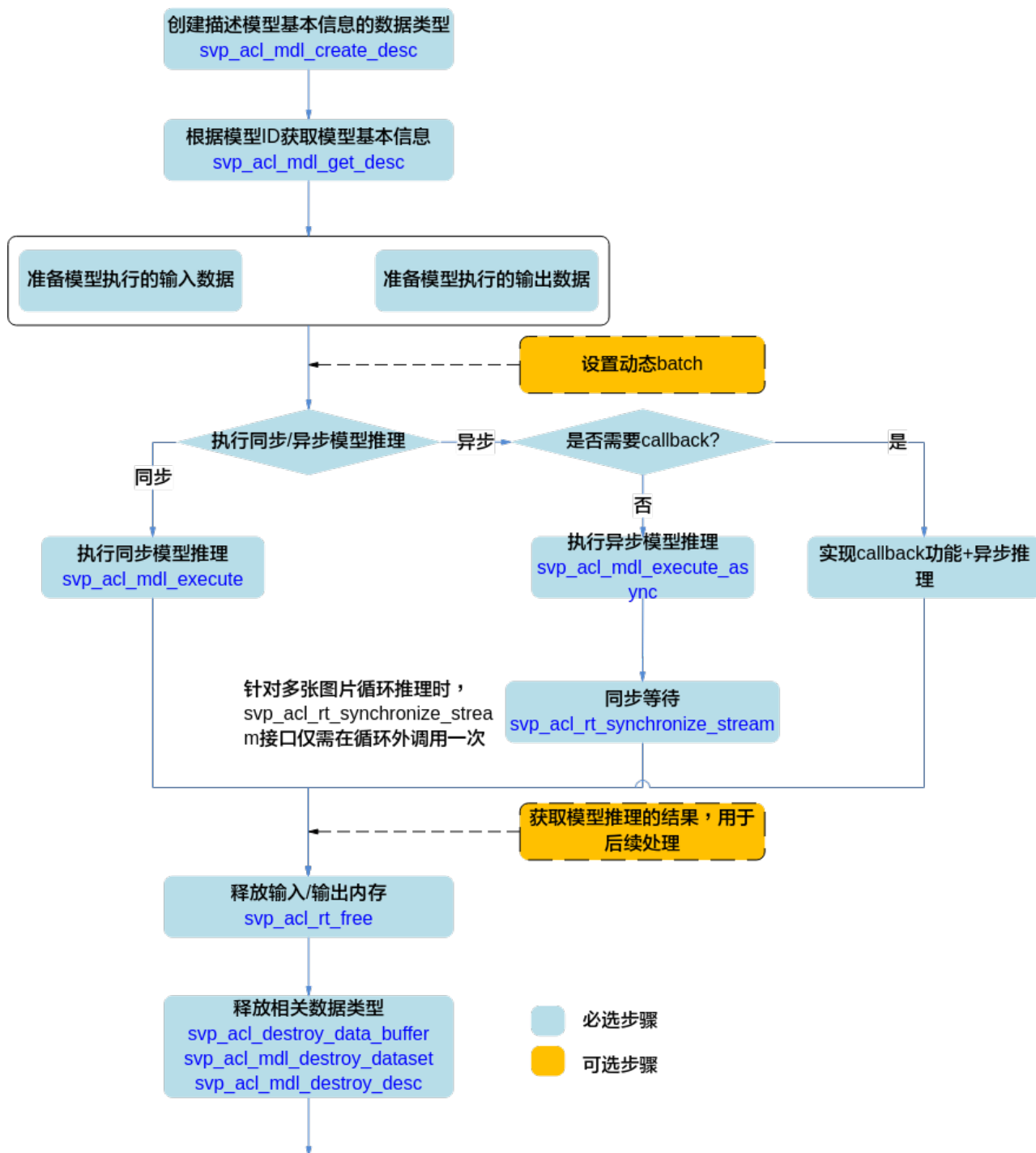


- 在模型加载前，需使用 ATC 工具将第三方网络（例如，Caffe ResNet-50 网络）转换为适配 SoC 的离线模型（*.om 文件），请参见《ATC 工具使用指南》。
- 支持以下方式加载模型，模型加载成功后，返回标识模型的模型 ID：
`svp_acl_mdload_from_mem`：从内存加载离线模型数据，由系统内部管理内存。

4.4 模型执行

4.4.1 基本的模型执行流程

图4-4 基本的模型推理流程



模型执行的关键流程说明如下：

1. 调用 `sva_acl_md1_create_desc` 接口创建描述模型基本信息的数据类型。
2. 调用 `sva_acl_md1_get_desc` 接口根据“4.3 模型加载”中返回的模型 ID 获取模型基本信息。
3. 准备模型执行的输入、输出数据，具体流程，请参见“4.4.3 准备模型执行的输入/输出数据”。
4. 设置动态 Batch 的具体流程，请参见“4.4.2 设置动态 Batch 和 Total_t”。
5. 执行模型推理。

当前系统支持模型的同步推理和异步推理：

- 同步推理时调用 `sva_acl_md1_execute` 接口
- 异步推理时调用 `sva_acl_md1_execute_async` 接口

对于异步接口，还需调用 `sva_acl_rt_synchronize_stream` 接口阻塞 Host 运行，直到指定 Stream 中的所有任务都完成。

如果同时需要实现 Callback 功能，请参见“4.5.3 Callback 场景”。

6. 获取模型推理的结果，用于后续处理。

对于同步推理，直接获取模型推理的输出数据即可。

对于异步推理，在实现 Callback 功能时，在回调函数内获取模型推理的结果，供后续使用。

7. 释放内存。

调用 `sva_acl_rt_free` 接口释放 Device 上的内存。

8. 释放相关数据类型的数据。

在模型推理结束后，需及时调用 `sva_acl_destroy_data_buffer` 接口和 `sva_acl_md1_destroy_dataset` 接口释放描述模型输入的数据，且先调用 `sva_acl_destroy_data_buffer` 接口，再调用 `sva_acl_md1_destroy_dataset` 接口。如果存在多个输入、输出，需调用多次 `sva_acl_destroy_data_buffer` 接口。

4.4.2 设置动态 Batch 和 Total_t

须知

- Recurrent 网络只支持设置动态 batch 为 1，支持设置 Total_t（要处理的总帧数）；非 Recurrent 网络只支持设置动态 batch，不支持设置 Total_t。
- 图片输入时最大 batch 数为 256。非图片输入时，最大 batch 数为 5000。

1. 准备模型推理的动态 Batch 输入的数据，详细流程请参见“4.4.3 准备模型执行的输入/输出数据”。
 - a. 设置动态 batch 之前需要先调用 `svp_acl_md1_get_input_index_by_name` 接口根据输入名称获取模型中标识动态 Batch 输入的 index，动态 Batch 输入的名称固定为 SVP_ACL_DYNAMIC_TENSOR_NAME，然后再调用 `svp_acl_md1_set_dynamic_batch_size` 接口根据 index 设置动态 batch。
 - b. 申请动态 Batch 输入对应的内存前，需要先调用 `svp_acl_md1_get_num_inputs` 获取输入个数，然后调用 `svp_acl_md1_get_input_size_by_index` 接口根据 index 获取单 batch 所需内存大小，乘以用户要设置的动态 batch 值得到需要的内存大小(task_buf 和 work_buf 不需要乘以动态 batch 值，详细请参见“4.4.3 准备模型执行的输入/输出数据”)。

说明

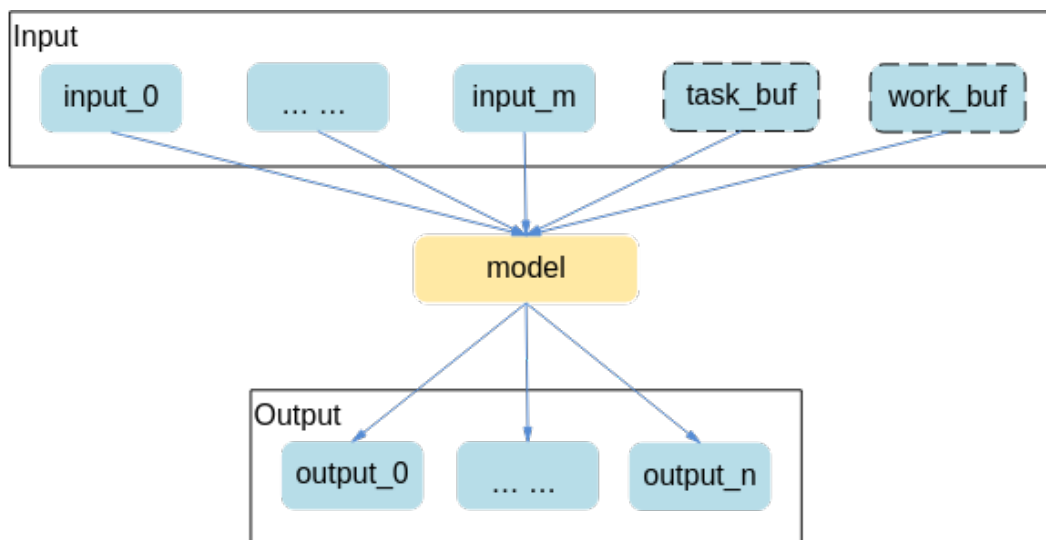
SVP_ACL_DYNAMIC_TENSOR_NAME 是一个宏，宏的定义如下：

```
#define SVP_ACL_DYNAMIC_TENSOR_NAME "svp_acl_mbatch_shape_data"
```

2. 在成功加载模型之后，执行模型之前，设置动态 Batch 数或者设置 Total_t。
 - a. 设置动态 Batch 数
调用 `svp_acl_md1_set_dynamic_batch_size` 接口设置动态 Batch 数。
 - b. 设置 Total_t
调用 `svp_acl_md1_set_total_t` 接口设置 Recurrent 网络推理的帧数。

4.4.3 准备模型执行的输入/输出数据

图4-5 模型输入/输出数据

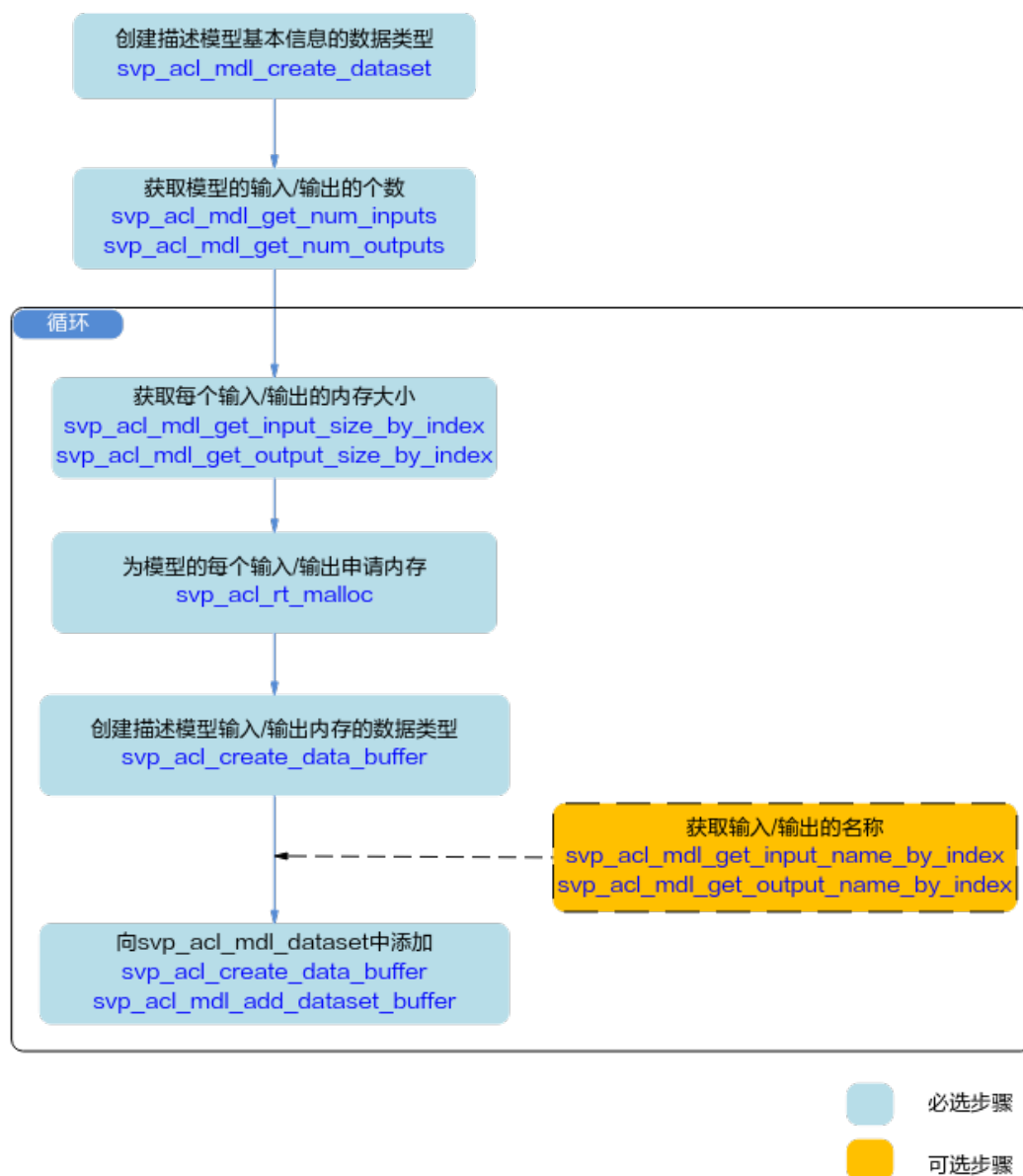


准备输入内存时需要额外输入两块内存，分别为 task_buf，work_buf，对应的输入索引为 M-2 和 M-1（M 是通过 [svp_acl_md1_get_num_inputs](#) 接口获取的输入个数）。在申请动态 batch 内存时，这两块内存不需要乘以 batch 值。

说明

- task_buf 用于填充模型推理的任务信息，当前模型推理完之前不能被修改或被其他模型推理任务使用，task_buf 的大小不为 0。
- work_buf 用于任务执行时存储中间结果，一个 stream 上如果会执行不同的模型，开辟一个最大的 work_buf 即可，这些模型在该 stream 上推理时可以共用，不同 stream 间不能共用，work_buf 大小不为 0。

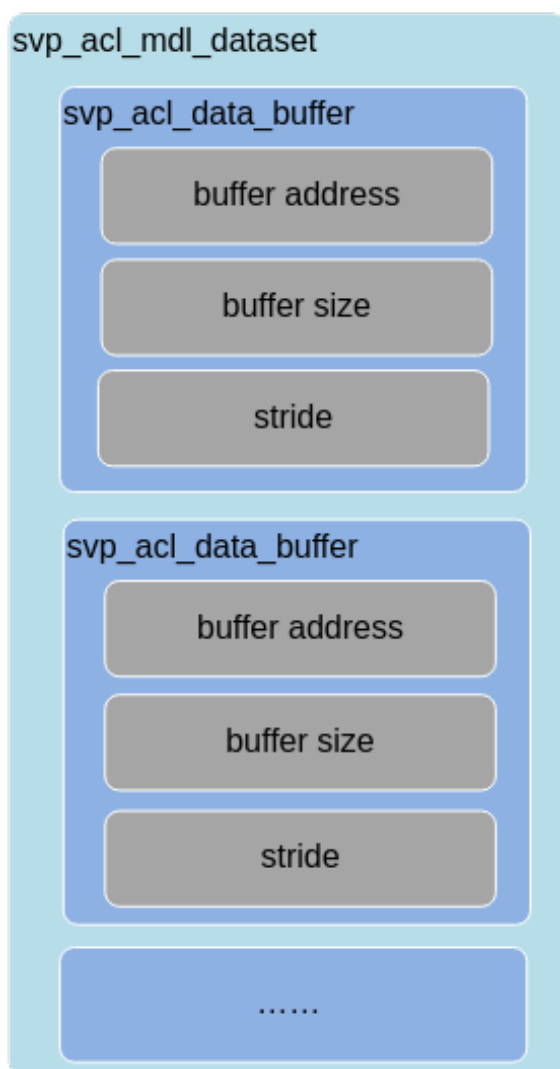
图4-6 模型执行的输入/输出数据准备流程



使用 `svp_acl_md1_desc` 类型的数据描述模型基本信息（例如输入/输出的个数、名称、数据类型、Format、维度信息等），使用 `svp_acl_md1_dataset` 类型的数据描述模

型的输入/输出数据，模型可能存在多个输入、多个输出，每个输入/输出的内存地址、内存大小用 `sva_acl_data_buffer` 类型的数据来描述。

图4-7 `sva_acl_md1_dataset` 类型与 `sva_acl_data_buffer` 类型的关系

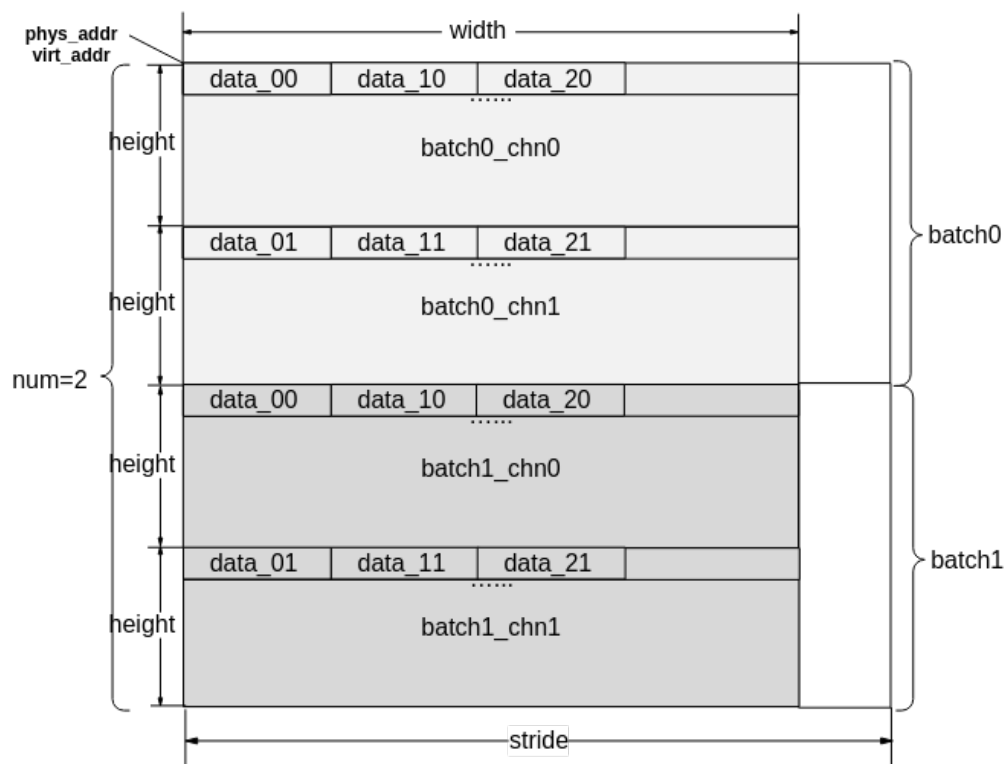


模型存在多个输入、输出时，用户在向 `sva_acl_md1_dataset` 中添加 `sva_acl_create_data_buffer` 时，为避免顺序出错，可以先获取输入、输出的名称，根据输入、输出名称所对应的 `index` 的顺序添加。

📖 说明

在创建 `sva_acl_data_buffer` 类型的数据时，内存大小可以通过调用 `sva_acl_md1_get_input_size_by_index` 接口根据 `index` 获取。

图4-8 模型输入/输出数据排布 (通道为 2, batch 为 2 示意图)

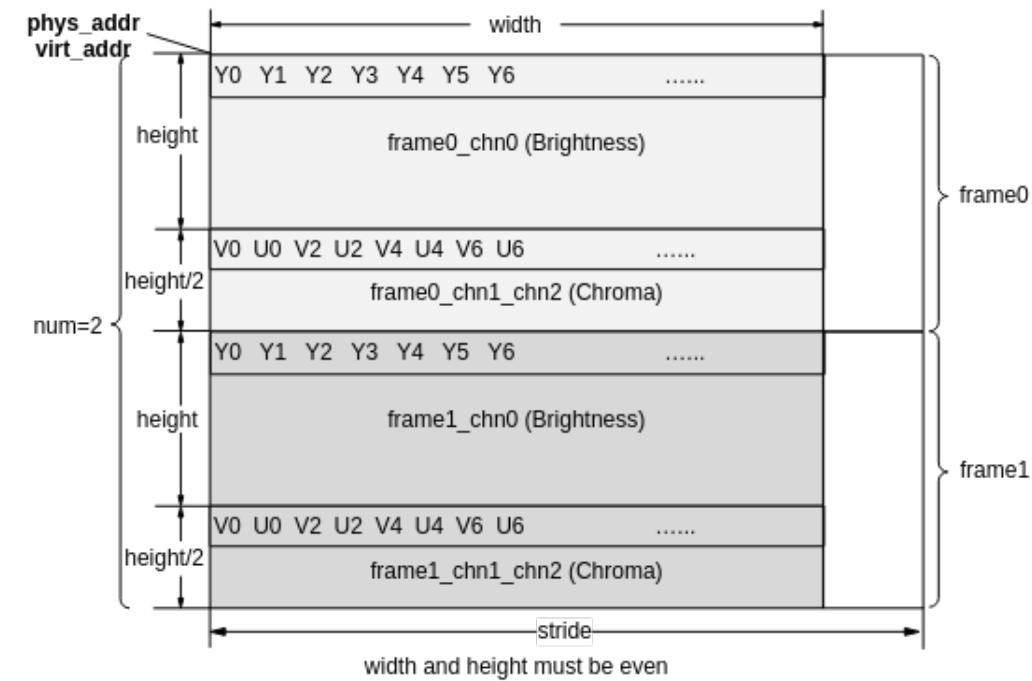


SVP_ACL 输入输出数据 stride 都是按照最后一维对齐, 除 YVU420SP 与 YUV420SP 输入格式外, 数据排布都如图 4-8 所示。

- 如果是 RGB_PACKAGE 格式, data_xx 数据类型为 U24, 通道数为 1。
- 如果是 XRGB_PACKAGE 格式, data_xx 数据类型为 U32, 通道数为 1。
- 如果是 XBGR_PACKAGE 格式, data_xx 数据类型为 U32, 通道数为 1。
- 如果是 RGBX_PACKAGE 格式, data_xx 数据类型为 U32, 通道数为 1。
- 如果是 BGRX_PACKAGE 格式, data_xx 数据类型为 U32, 通道数为 1。
- 如果是 BGR_PACKAGE 格式, data_xx 数据类型为 U24, 通道数为 1。
- 如果是 BGR_PLANAR 格式, data_xx 数据类型为 U8, 通道数为 3。
- 如果是 RGB_PLANAR 格式, data_xx 数据类型为 U8, 通道数为 3。
- 如果是 XRGB_PLANAR 格式, data_xx 数据类型为 U8, 通道数为 3。
- 如果是 XBGR_PLANAR 格式, data_xx 数据类型为 U8, 通道数为 3。
- 如果是 RGBX_PLANAR 格式, data_xx 数据类型为 U8, 通道数为 3。
- 如果是 BGRX_PLANAR 格式, data_xx 数据类型为 U8, 通道数为 3。
- 如果是 RAW_RGGB 格式, data_xx 数据类型为 U8/U12/U14/U16, 通道数为 1。

- 如果是 RAW_GRBG 格式，data_xx 数据类型为 U8/U12/U14/U16，通道数为 1。
- 如果是 RAW_GBRG 格式，data_xx 数据类型为 U8/U12/U14/U16，通道数为 1。
- 如果是 RAW_BGGR 格式，data_xx 数据类型为 U8/U12/U14/U16，通道数为 1。

图4-9 YVU420SP/YUV420SP 数据排布 (通道为 2, frame 为 2 示意图)



SVP_ACL 支持检测网阈值通过 data 层传入，阈值输入固定长度为 4，分别是 nms_threshold, score_threshold, min_height, min_width，同时将检测网络输出的检测框结果数据排布进行了统一，检测网阈值和检测框结果排布格式如图 4-11 所示。

图4-10 检测网阈值输入数据排布

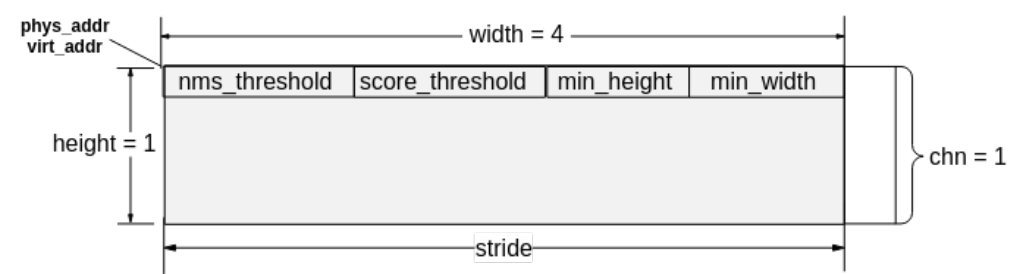
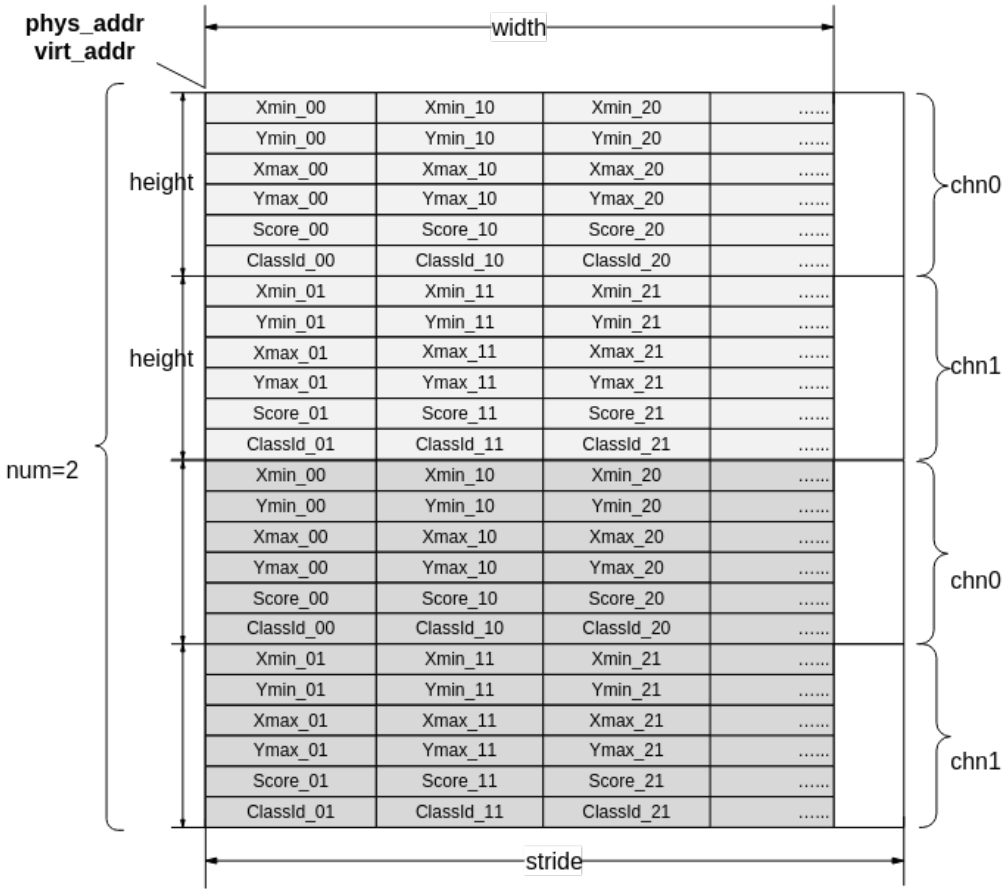


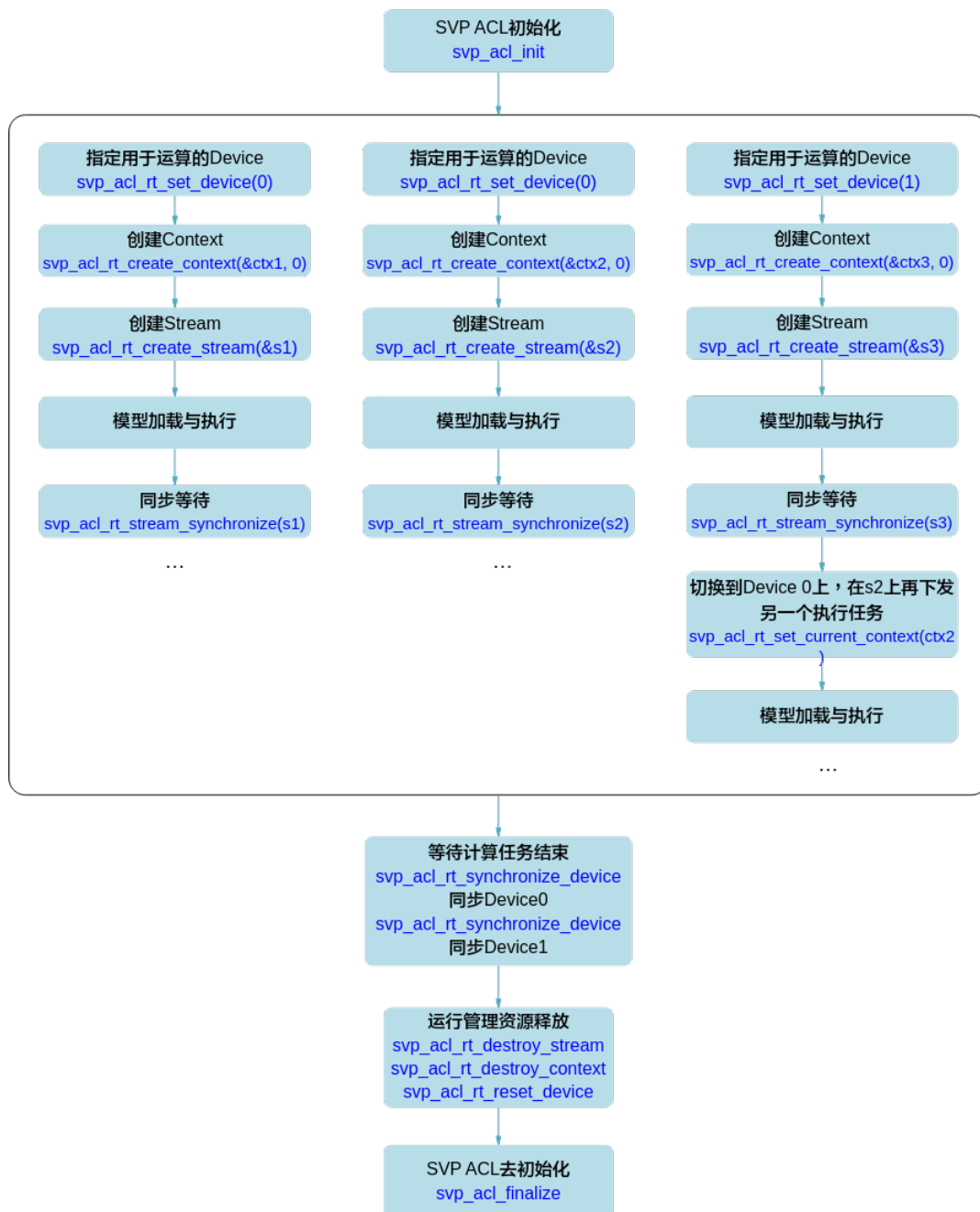
图4-11 检测框结果数据排布 (通道为 2, chn 为 2 示意图)



4.5 同步等待

4.5.1 多 Device 场景

图4-12 同步等待流程多 Device 场景

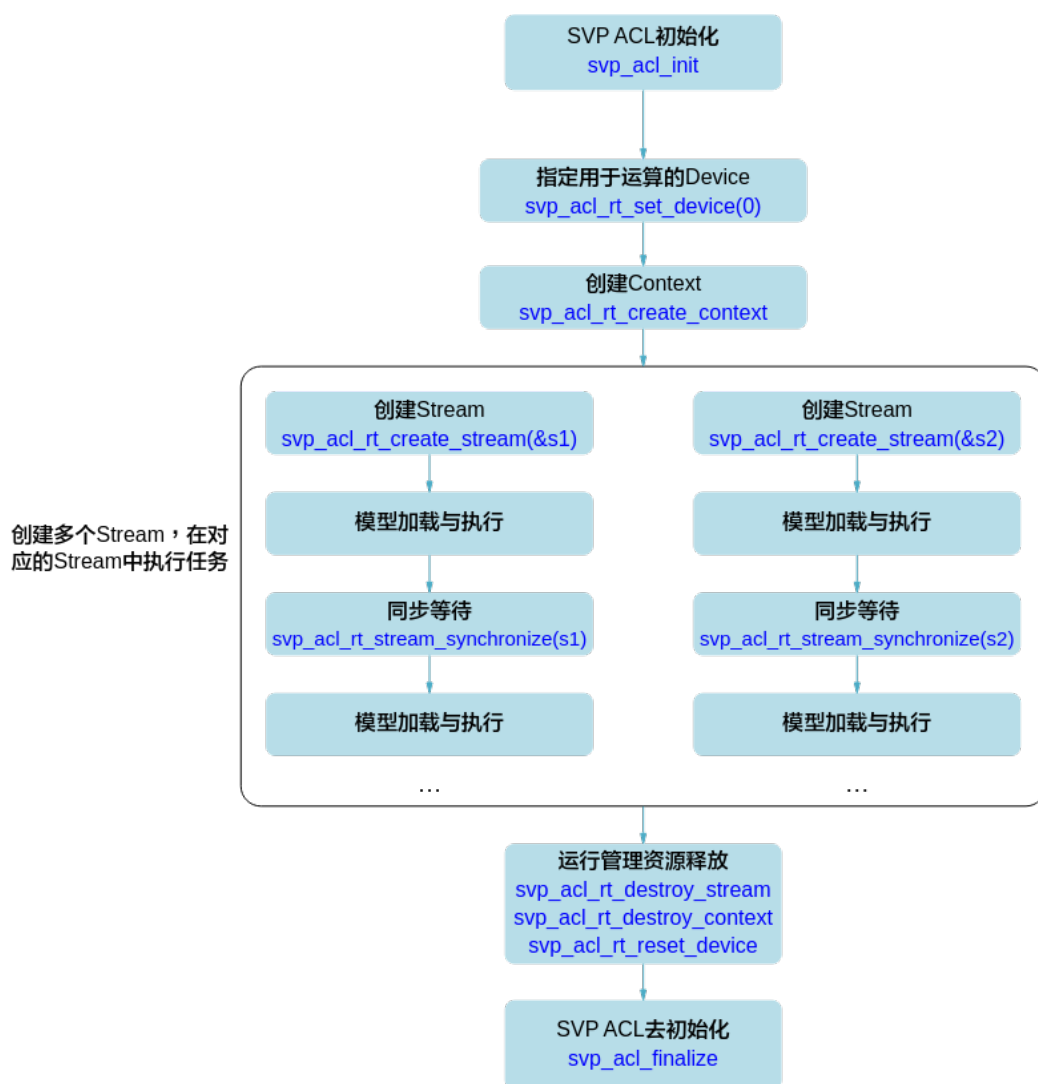


- 在多 Device 时，利用 Context 切换（调用 `svp_acl_rt_set_current_context` 接口）来切换 Device，比使用 `svp_acl_rt_set_device` 接口效率高。

- 调用 `svp_acl_rt_synchronize_device` 接口等待 Device 上的计算任务结束。
- 模型加载的流程请参见“4.3 模型加载”，模型执行的流程请参见“4.4 模型执行”。

4.5.2 多 Stream 场景

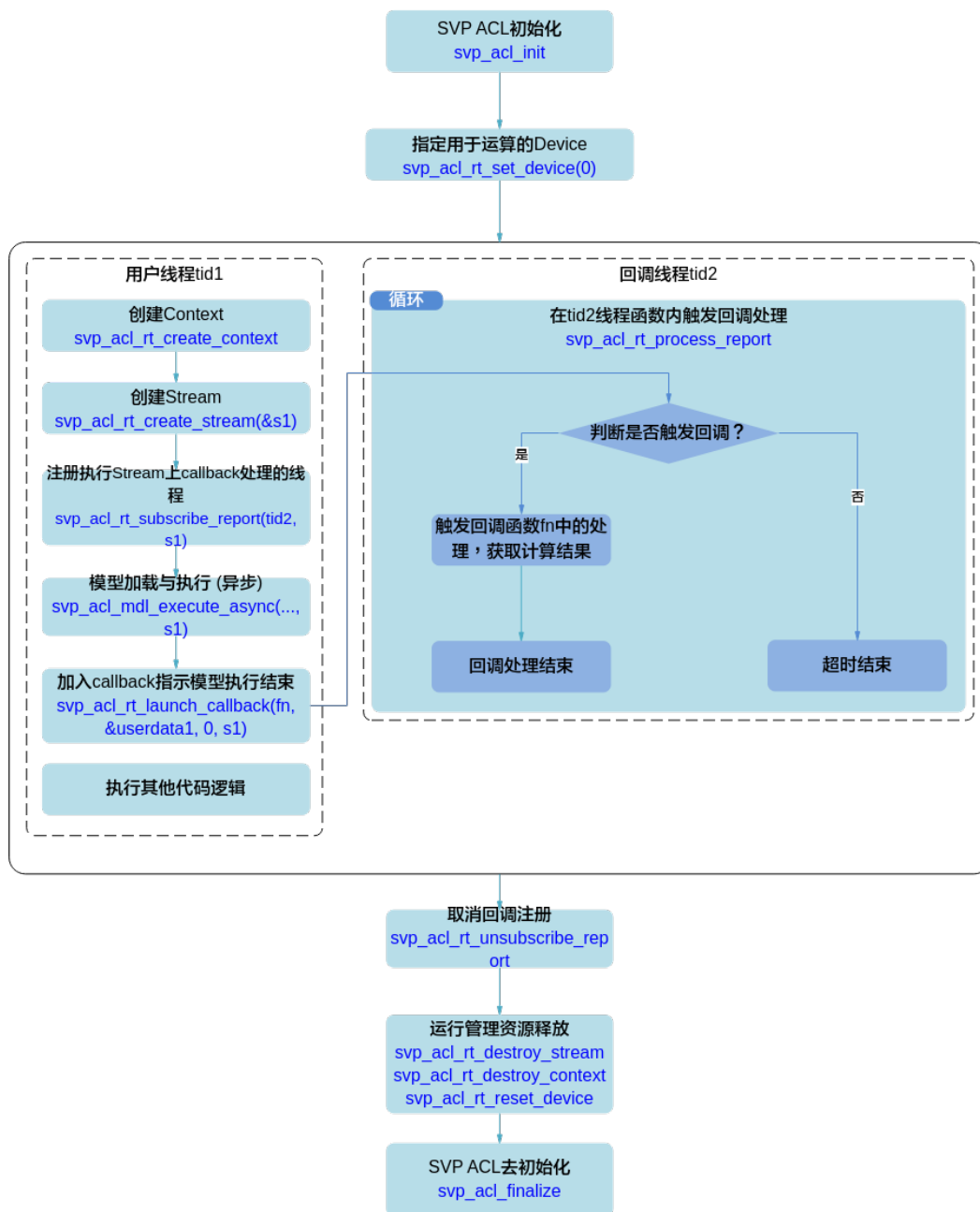
图4-13 同步等待流程多 Stream 场景



模型加载的流程请参见“4.3 模型加载”，模型执行的流程请参见“4.4 模型执行”。

4.5.3 Callback 场景

图4-14 同步等待流程 Callback 场景模型推理



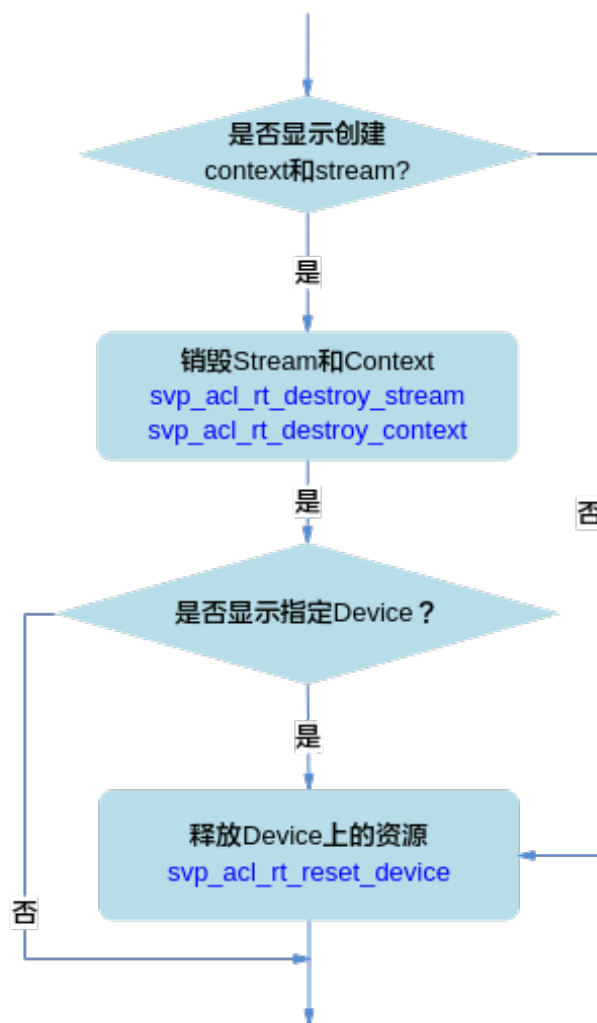
Callback 流程中关键流程说明如下：

- 回调函数 fn 需由用户提前创建，用于获取并处理模型推理或算子执行的结果。
- 线程 tid2 需由用户提前创建，并自定义线程函数，在线程函数内调用 `svp_acl_rt_process_report` 接口，等待指定时间后，触发回调函数。

- 调用 `sva_acl_rt_unsubscribe_report` 接口：指定处理 Stream 上回调函数的线程，线程与 tid2 保持一致。
- 调用 `sva_acl_rt_launch_callback` 接口：在 Stream 的任务队列中增加一个需要在板端环境上执行的回调函数，回调函数与 fn 保持一致。
- 调用 `sva_acl_rt_unsubscribe_report` 接口：取消线程注册（Stream 上的回调函数不再由指定线程处理）。
- 如果是异步推理 Callback 场景，为确保 Stream 中所有任务都完成、模型推理的结果数据都经过 Callback 函数处理，在 Stream 销毁前，需要调用一次 `sva_acl_rt_synchronize_device` 接口。

4.6 运行管理资源释放

图4-15 运行管理资源释放流程

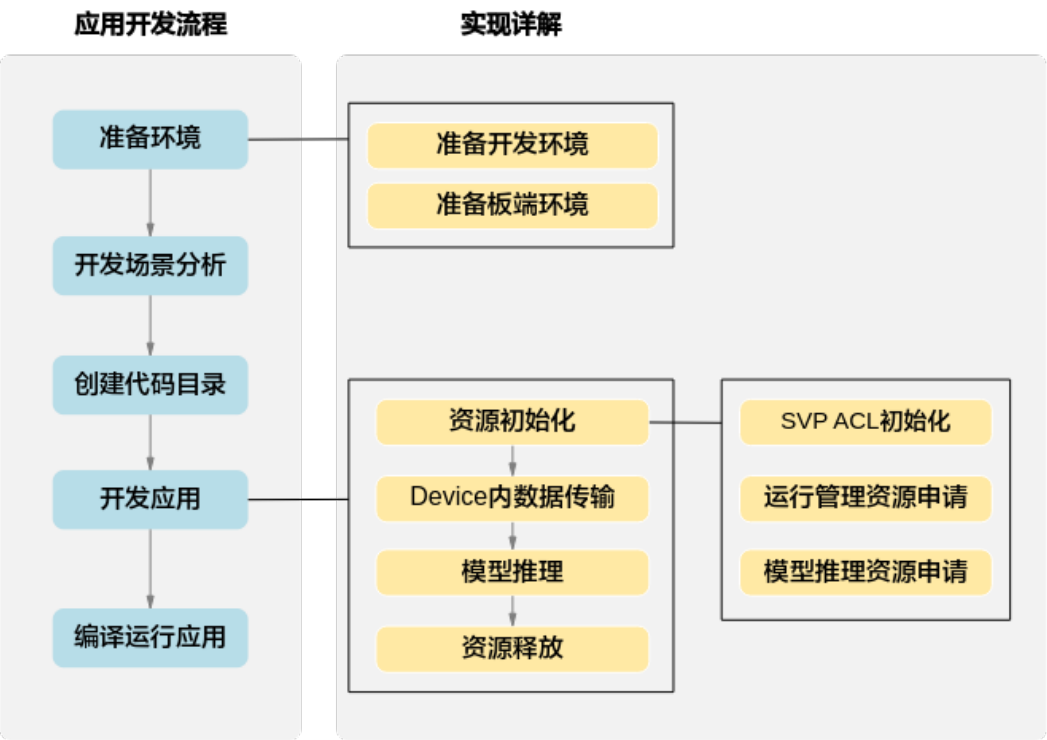


释放运行管理资源时，需按顺序依次释放：[Stream](#)、[Context](#)、[Device](#)。

- 显式创建 Context 和 Stream 时，需调用 [svp_acl_rt_destroy_stream](#) 接口释放 Stream，再调用 [svp_acl_rt_destroy_context](#) 接口释放 Context。若显式调用 [svp_acl_rt_set_device](#) 接口指定运算的 Device 时，还需调用 [svp_acl_rt_reset_device](#) 接口释放 Device 上的资源。
- 不显式创建 Context 和 Stream 时，仅需调用 [svp_acl_rt_reset_device](#) 接口释放 Device 上的资源。

5 开发流程

图5-1 开发流程（不包含数据预处理）



步骤 1 准备环境，包括开发环境和板端环境。

步骤 2 开发场景分析。

根据开发场景分析涉及哪些功能（例如，数据传输、模型推理等）的开发，确定功能后，再明确涉及的命令或接口，请参见“7.1 开发场景分析”。

步骤 3 创建代码目录。

在开发应用前，您需要先创建目录，存放代码文件、编译脚本、测试图片数据、模型文件等，请参见“[7.2 创建代码目录](#)”。

步骤 4 开发应用。

1. 资源初始化，包括 SVP ACL 初始化、运行管理资源申请、模型推理资源申请等，请参见“[7.3.1 资源初始化](#)”。

使用 SVP ACL 接口开发应用时，必须先调用 `svp_acl_init` 接口进行 SVP ACL 初始化，否则可能会导致后续系统内部资源初始化出错，进而导致其它业务异常。

2. 数据传输，请参见“[7.3.2 读入图片数据](#)”。
3. 执行模型推理。请参见“[7.3.3 模型推理](#)”。

模型推理结束后，需及时释放相关资源。

4. 若需要处理模型推理的结果，还需要进行数据后处理，例如对于图片分类应用，通过数据后处理从推理结果中查找最大置信度的类别标识。
5. 所有数据处理结束后，需及时释放运行管理资源。

步骤 5 编译运行应用，包括模型转换、编译代码、运行应用，请参见“[7.4 编译及运行应用](#)”。

---结束

6 准备环境

6.1 准备开发环境

- 您需要参见《驱动和开发环境安装指南》安装开发环境，获取以下文件：
 - 从“SVP ACLlib 组件的安装目录/acllib/include/acl”目录下获取调用 SVP ACL 接口所需的头文件。
 - 从“SVP ACLlib 组件的安装目录/acllib/lib64/stub”目录下获取编译 SVP ACL 接口所需的库文件。
- 在安装开发环境后，您需要参见《驱动和开发环境安装指南》中的“安装后处理 ” 章节安装交叉编译器，并配置对应环境变量。

须知

- 本文以如下安装路径示例来说明操作步骤，实际编译、运行应用前，**请务必**获取这些组件的实际安装路径，以便后续操作时使用，其中，\$HOME 表示安装用户的家目录：

以非 root 用户安装 ACLlib 组件安装包，安装路径示例为\$HOME/acl，在该路径下，包括“acllib”目录。

- 本文中的操作步骤（包括模型转换、编译代码等）需以运行用户登录开发环境后再执行，**请务必**获取各组件的运行用户，以便后续操作时使用。
- 用户使用 export 命令在当前终端窗口下声明环境变量，关闭 Shell 终端或切换用户时环境变量失效。

6.2 准备运行环境

- 如果您需要运行应用，需完成板端环境的配置、软件包的部署等，请参见《驱动和开发环境安装指南》。
- 板端环境的操作系统为 Linux 时，在部署与调试后，使用过程中，若板端环境上的空间不足时，您可以使用 mount 命令将 NFS 服务器上的目录挂载到板端环境的指定目录。

- a. 以 root 用户登录 Linux 服务器（Ubuntu 操作系统），安装 NFS 服务并配置共享目录。

确保环境连网且源可用的前提下，安装 NFS 服务，可参考如下命令安装，如果安装过程中提示已安装 NFS 服务，则无需重复安装：

```
apt-get install nfs-kernel-server
```

在“/etc/exports”文件中配置共享目录，配置完成后，执行 **/etc/init.d/nfs-kernel-server restart** 命令重启 NFS 服务使配置生效。可参考如下配置段（添加在文件的末尾），斜体部分请根据实际情况替换，*服务器绝对路径*表示 Linux 服务器的共享目录，如不存在，请提前创建。

```
服务器绝对路径 *(rw, sync, root_squash, anonuid=id*, anongid=gid*)
```

- b. 命令示例如下，其中，*path* 表示板端环境上的目录，需根据实际情况修改，例如：/root。

```
mount -t nfs -o nolock, tcp NFS 服务器 IP 地址:服务器绝对路径 path
```

7 开发首个应用

7.1 开发场景分析

7.1.1 开发场景

开发图片分类应用，对 2 张分辨率为 1024*683 的*.jpg 图片分类。

7.1.2 场景分析

您可从以下几方面入手分析该应用需包含哪些功能，这些功能点对应哪些 SVP ACL 接口。

场景列表	实现分析	涉及的命令或关键接口
模型推理	本文介绍的场景是图片分类，因此需要选取开源的分类网络，此处选择的是 Caffe resnet-50 网络，将开源的 resnet-50 网络转换为适配图像分析引擎的离线模型（*.om 文件），使用该离线模型推理图片所属的类别。 resnet-50 网络对输入图片宽高的要求为 224*224，且要求输入图片格式为 RGB。但当前输入图片是*.jpg 格式，因此下文的样例中提供了一个 Python 脚本先转换图片。	• 使用 ATC（Advanced Tensor Compiler）工具将 resnet-50 网络转换为*.om 文件（适配图像分析引擎处理器的离线模型）。 请参见《ATC 工具使用指南》。 • 模型加载 svp_acl_md1_load_from_mem • 模型执行 svp_acl_md1_execute • 模型卸载 svp_acl_md1_unload

7.2 创建代码目录

在开发应用前，您需要先创建目录，存放代码文件、编译脚本、测试图片数据、模型文件等。

以开发环境的安装用户，在开发环境的任意目录下创建存放应用代码的目录，如下仅是示例，可参考：

```
└─App 名称
├─caffe_model    //该目录下存放模型转换相关的文件
│   └─xxx.caffemodel
├─model          //该目录下存放转化后的 om 模型
│   └─xxx.om
├─data
│   └─xxx.jpg    //原始测试数据
├─inc            //该目录下存放声明函数的头文件
│   └─xxx.h
├─out            //该目录下存放输出结果
├─src            //该目录下存放系统初始化的配置文件、编译脚本、函数的实现文件
│   └─xxx.json   //系统初始化的配置文件
│   └─CMakeLists.txt //编译脚本
│   └─xxx.cpp    //实现文件
```

7.3 开发应用

7.3.1 资源初始化

7.3.1.1 SVP ACL 初始化

基本原理：必须调用 `svp_acl_init` 接口初始化 SVP ACL，配置文件内容为 json 格式，当前支持以下配置：

dump 信息配置，示例、配置说明及约束请参见《MindStudio 用户指南》中的“精度比对->推理场景数据准备”。默认不启动 dump 配置。

📖 说明

如果当前的默认配置已满足需求，无需修改，可向 `svp_acl_init` 接口中传入 `NULL`，或者可将配置文件配置为空 json 串（即配置文件中只有 {}）。

向 `svp_acl_init` 接口中传入空指针的示例如下：

```
svp_acl_error ret = svp_acl_init(NULL);
```

示例代码：可以从 [acl_resnet50](#) 样例中查看完整样例代码。

调用接口后，增加了异常处理的分支，同时通过 ERROR_LOG 记录报错日志、通过 INFO_LOG 记录各动作的提示日志，示例代码中不——列举。以下是关键步骤的代码示例，不可以直接拷贝编译运行，仅供参考。

```
#include "acl/svp_acl.h"
//.....
//初始化基本配置。
//此处的..表示相对路径，相对可执行文件所在的目录
//例如，编译出来的可执行文件存放在 out 目录下，此处的..就表示 out 目录的上一级目录
const char * acl_config_path = "../src/acl.json";
svp_acl_error ret = svp_acl_init(acl_config_path);
//.....
```

7.3.1.2 运行管理资源申请（单进程+单线程+单 Stream）

多线程、多 Stream 的场景请参见 [8.1 Stream 管理](#)。

基本原理：需要按顺序依次申请如下资源：[Device](#)、[Context](#)、[Stream](#)，确保可以使用这些资源执行运算、管理任务。

- 关于 Context 和 Stream
 - 显式创建 Context 时，调用 [svp_acl_rt_create_context](#) 接口，此时需显式调用 [svp_acl_rt_destroy_context](#) 接口释放。
 - 显式创建 Stream 时，调用 [svp_acl_rt_create_stream](#) 接口，此时需显式调用 [svp_acl_rt_destroy_stream](#) 接口释放。
 - 如果不显式创建 Context 和 Stream，您可以使用 [svp_acl_rt_set_device](#) 接口隐式创建的默认 Context 和默认 Stream，但默认 Context 和默认 Stream 存在如下限制：
 - 一个 Device 对应一个默认 Context，默认 Context 不能通过 [svp_acl_rt_destroy_context](#) 接口来释放。
 - 一个 Device 对应一个默认 Stream，默认 Stream 不能通过 [svp_acl_rt_destroy_stream](#) 接口来释放。默认 Stream 作为接口入参时，直接传 NULL。
- 关于单进程、单线程、单 Stream 场景
 - 单进程：一个应用程序对应一个进程。
 - 单线程：不显式创建多个线程时，默认只有一个线程。
 - 单 Stream：整个开发的过程中使用同一个 Stream。

对于同一个 Stream 中的异步任务，SVP ACL 会按照应用程序中任务的顺序执行任务，确保异步任务执行的顺序。

示例代码：可以从 [acl_resnet50](#) 样例中查看完整样例代码。

调用接口后，增加了异常处理的分支，同时通过 ERROR_LOG 记录报错日志、通过 INFO_LOG 记录各动作的提示日志，示例代码中不一一列举。以下是关键步骤的代码示例，不可以直接拷贝编译运行，仅供参考。

```
#include "acl/svp_acl.h"
//.....

//1.初始化变量
extern bool g_is_device;
//.....

//2.指定运算的Device，SoC场景下，当前只有一个Device
ret = svp_acl_rt_set_device(device_id);

//3.显式创建一个Context，用于管理Stream对象
ret = svp_acl_rt_create_context(&context, device_id);

//4.显式创建一个Stream
//用于维护一些异步操作的执行顺序，确保按照应用程序中的代码调用顺序执行任务
ret = svp_acl_rt_create_stream(&stream);

//5.获取当前svp_acl软件栈的运行模式，根据不同的运行模式，后续的接口调用方式不同
//SoC场景下,该接口的查询结果为SVP_ACL_DEVICE，表示svp_acl软件栈运行在SoC的CPU上，只
需申请SoC上的内存
svp_acl_rt_run_mode run_mode;
ret = svp_acl_rt_get_run_mode(&run_mode);
g_is_device = (run_mode == SVP_ACL_DEVICE);

//.....
```

7.3.1.3 模型推理资源申请

基本原理：在模型推理前，需要从离线模型文件（适配 SoC 的离线模型）中加载模型数据到内存中。

- 关于模型转换

在模型加载前，需要将第三方网络（例如，Caffe ResNet-50 网络）转换为适配 SoC 的离线模型，请参见《ATC 工具使用指南》。

- 关于模型加载

加载模型数据方式如下：

`svp_acl_mdload_from_mem`：从内存加载离线模型数据，由系统内部管理内存。

- 关于模型的输入、输出数据

`svp_acl_mdload_dataset` 主要用于描述模型推理时的输入数据/输出数据，模型可能存在多个输入/多个输出，每个输入/输出的内存地址、内存大小用

`svp_acl_data_buffer` 类型的数据来描述。

示例代码：可以从 `acl_resnet50` 样例中查看完整样例代码。

以下是关键步骤的代码示例，不可以直接拷贝编译运行，仅供参考。

```
#include "acl/svp_acl.h"
//.....

//1.初始化变量。

//此处的..表示相对路径，相对可执行文件所在的目录

//例如，编译出来的可执行文件存放在out目录下，此处的..就表示out目录的上一级目录
const char* om_model_path = "../model/resnet50.om"
//.....

//2.申请用于存储.om文件内存，model_mem_size为.om文件大小。
ret = svp_acl_rt_malloc(&model_mem_ptr, model_mem_size,
SVP_ACL_MEM_MALLOC_HUGE_FIRST);
//.....

//3.加载离线模型文件（适配SoC的离线模型）。

//模型加载成功，返回标识模型的ID。
ret = svp_acl_mdload_from_mem(model_mem_ptr, model_mem_size, &model_id);

//4.根据5中加载成功的模型的ID，获取该模型描述信息。

// model_desc为svp_acl_mdload_desc类型。
model_desc = svp_acl_mdload_create_desc();
ret = svp_acl_mdload_get_desc(model_desc, model_id);
```

```
//5.创建svp_acl_model_dataset类型的数据，描述模型推理的输出。

//output为svp_acl_md1_dataset类型
output = svp_acl_md1_create_dataset();

//6.1 获取模型的输出个数.
size_t output_size = svp_acl_md1_get_num_outputs(model_desc);

//6.2 循环为每个输出申请内存，并将每个输出添加到svp_acl_model_dataset类型的数据中.
for (size_t i = 0; i < output_size; ++i) {
    size_t buffer_size = svp_acl_md1_get_output_size_by_index(model_desc, i);
    void *output_buffer = nullptr;
    svp_acl_error ret = svp_acl_rt_malloc(&output_buffer, buffer_size,
SVP_ACL_MEM_MALLOC_NORMAL_ONLY);
    svp_acl_data_buffer* output_data = svp_acl_create_data_buffer(output_buffer, buffer_size);
    ret = svp_acl_md1_add_dataset_buffer(output, output_data);
}
//.....
```

7.3.2 读入图片数据

基本原理：

- 根据“7.3.1.2 运行管理资源申请（单进程+单线程+单 Stream）”中 [svp_acl_rt_get_run_mode](#) 接口的查询结果（SVP_ACL_DEVICE，表示软件栈运行在 SoC 的图像分析引擎上），在自定义函数 ReadBinFile 内部，直接调用 [svp_acl_rt_malloc](#) 申请 Device 上的内存，再调用 C++标准库的 read 函数将图片数据读入 Device 的内存中。
- 读入图片数据（即将输入图片数据传输到 Device 上，用于推理）。

示例代码：可以从 [acl_resnet50](#) 样例中查看完整样例代码。

以下是关键步骤的代码示例，不可以直接拷贝编译运行，仅供参考。

```
#include <iostream>
#include <fstream>
#include <cstring>
#include "acl/svp_acl.h"
//.....

//以下数据用于描述图片的路径及图片名称picName。
string testFile[] = {
```

```
        "../data/dog1_1024_683.bin",
        "../data/dog2_1024_683.bin"
    };

//循环处理每张图片
for (size_t index = 0; index < sizeof(testFile) / sizeof(testFile[0]); ++index) {

    //自定义函数，调用C++标准库std::ifstream中的函数读取图片文件。

    //并输出图片文件占用的内存大小fileSize，返回值为图片文件的内存地址inputBuff。

    void *inputBuff = nullptr;
    uint32_t fileSize = 0;
    auto ret = Utils::ReadBinFile(fileName, inputBuff, fileSize);

    //TODO：模型推理
}
//.....
```

7.3.3 模型推理

基本原理：同步执行模型推理。模型推理结束后，需及时释放相关资源。

- 关于模型推理

调用 `svp_acl_mdل_execute` 接口实现同步模型推理，该接口的入参之一是模型 ID，在“7.3.1.3 模型推理资源申请”阶段，如果加载模型成功，则会返回标识模型的 ID。

- 关于模型输出数据的处理

“7.3.3.1 示例代码”中的代码逻辑是：获取模型推理的输出数据，将输出数据转换为 float 类型后，再输出其中 top5 置信度的类别编号。

- 关于资源释放

- `svp_acl_mdل_dataset` 主要用于描述模型推理时的输入数据、输出数据，模型可能存在多个输入、多个输出，每个输入/输出的内存地址、内存大小用 `svp_acl_data_buffer` 类型的数据来描述。在模型推理结束后，需及时调用 `svp_acl_destroy_data_buffer` 接口和 `svp_acl_mdل_destroy_dataset` 接口释放描述模型输入的数据，且先调用 `svp_acl_destroy_data_buffer` 接口，再调用 `svp_acl_mdل_destroy_dataset` 接口。如果存在多个输入，需调用多次 `svp_acl_destroy_data_buffer` 接口。

- 在模型推理结束后，还需要通过 `svp_acl_md1_unload` 接口卸载模型。

7.3.3.1 示例代码

您可以从 `acl_resnet50` 样例中查看完整样例代码。

调用接口后，增加了异常处理的分支，同时通过 `ERROR_LOG` 记录报错日志、通过 `INFO_LOG` 记录各动作的提示日志，示例代码中不一一列举。以下是关键步骤的代码示例，不可以直接拷贝编译运行，仅供参考。

```
#include <iostream>
#include <map>
#include <sstream>
#include <algorithm>
#include "acl/svp_acl.h"
//.....

//以下数据用于描述图片的路径及图片名称picName。
string testFile[] = {
    "../data/dog1_1024_683.bin",
    "../data/dog2_1024_683.bin"
};

//循环处理每张图片
for (size_t index = 0; index < sizeof(testFile) / sizeof(testFile[0]); ++index) {

    //1. 创建模型推理的输入数据

    //创建svp_acl_md1_dataset类型的数据input_，用于描述模型推理时的输入数据（图片数据集）

    //输入的内存地址、内存大小用svp_acl_data_buffer类型的数据来描述
    input_ = svp_acl_md1_create_dataset();
    //示例中的模型只有1个输入，将“读入图片数据”阶段的图片数据作为模型推理的输入

    //inputDataBuffer表示图片数据在Device中的内存，bufferSize表示内存大小
    svp_acl_data_buffer* inputData = svp_acl_create_data_buffer(inputDataBuffer, bufferSize);
    aclError ret = svp_acl_md1_add_dataset_buffer(input_, inputData);

    //2. 执行模型推理

    //modelId_表示模型ID，在“模型推理资源初始化”时成功加载模型后，会返回标识模型的ID

    //output_表示模型推理的输出数据，在“模型推理资源初始化”时已定义
    aclError ret = svp_acl_md1_execute(modelId_, input_, output_)
```

```

//获取模型推理的输出，做数据后处理，输出top5置信度的类别编号
for (size_t i = 0; i < svp_acl_md1_get_dataset_num_buffers (output_); ++i) {
    //获取每个输出的内存地址和内存大小
    svp_acl_data_buffer* dataBuffer = svp_acl_md1_get_dataset_buffer(output_, i);
    void* data = svp_acl_get_data_buffer_addr(dataBuffer);

    size_t len = svp_acl_get_data_buffer_size(dataBuffer);

    //将内存中的数据转换为float类型
    float *outData = NULL;
    outData = reinterpret_cast<float*>(data);

    //屏显每张图片的top5置信度的类别编号
    map<float, int, greater<float> > resultMap;
    for (int j = 0; j < len / sizeof(float); ++j) {
        resultMap[*outData] = j;
        outData++;
    }
    int cnt = 0;
    for (auto it = resultMap.begin(); it != resultMap.end(); ++it) {
        // print top 5
        if (++cnt > 5) {
            break;
        }

        INFO_LOG("top %d: index[%d] value[%lf]", cnt, it->second, it->first);
    }

    //3. 模型推理结束后，释放模型推理的输入内存picDevBuffer
    svp_acl_rt_free(picDevBuffer);

    //4. 释放资源

    //如果模型存在多个输入，则需要多次调用svp_acl_destroy_data_buffer释放描述每个输入的
    svp_acl_md1_dataset类型数据
    for (size_t i = 0; i < svp_acl_md1_get_dataset_num_buffers(input_); ++i) {
        svp_acl_data_buffer* dataBuffer = svp_acl_md1_get_dataset_buffer(input_, i);
        void* data = svp_acl_get_data_buffer_addr(dataBuffer);
        (void)svp_acl_rt_free(data);
        svp_acl_destroy_data_buffer(dataBuffer);
    }
}

```

```
}

    svp_acl_md5_destroy_dataset(input_);

    //.....
}

//TODO: 释放运行管理资源
```

7.3.4 运行管理资源释放与 SVP ACL 去初始化

基本原理：

所有数据处理都结束后，需要释放运行管理资源，包括 [Stream](#)、[Context](#)、[Device](#)。释放资源时，需要按顺序释放，先释放 Stream，再释放 Context，最后再释放 Device。

- 如果使用默认 Stream，则不能通过 [svp_acl_rt_destroy_stream](#) 接口来释放。
- 如果使用默认 Context，则不能通过 [svp_acl_rt_destroy_context](#) 接口来释放。
- 默认 Context、默认 Stream，是在调用 [svp_acl_rt_reset_device](#) 接口后自动释放。
- 调用 [svp_acl_finalize](#) 接口实现 SVP ACL 去初始化。

示例代码：可以从 [acl_resnet50](#) 样例中查看完整样例代码。

以下是关键步骤的代码示例，不可以直接拷贝编译运行，仅供参考。

```
#include "acl/svp_acl.h"
//.....

svp_acl_error ret = svp_acl_rt_destroy_stream(stream);
ret = svp_acl_rt_destroy_context(context);
ret = svp_acl_rt_reset_device(device_id);
ret = svp_acl_finalize();
//.....
```

7.4 编译及运行应用

编译运行应用的步骤，请参见“[10.2 基于 Caffe ResNet-50 网络实现图片分类（同步推理）](#)”。

相关注意点：

- 模型转换，详细说明请参见《ATC 工具使用指南》。
- 编译代码时，编译脚本的修改点如下。

您可以从 [acl_resnet50](#) 样例中获取编译脚本 CMakeLists.txt，在该编译脚本的基础上修改如下参数。

- include_directories：添加头文件所在的目录。

示例如下：

```
include_directories(  
    directoryPath1  
    directoryPath2  
)
```

- link_directories：添加库文件所在的目录。

示例如下：

```
link_directories(  
    directoryPath3  
    directoryPath4  
)
```

- add_executable：修改可执行文件的名称（例如：*main*）、添加*.cpp 文件所在的目录。

示例如下：

```
add_executable(main  
    directoryPath5  
    directoryPath6)
```

- target_link_libraries：修改可执行文件的名称（与 [add_executable](#) 中保持一致）、添加可执行文件依赖的库文件。

示例如下：

```
target_link_libraries(main  
    svp_acl  
    libName1  
    libName2)
```

依赖的库文件与接口所在的头文件有关，具体对应关系如下：

表7-1 头文件与库文件的对应关系

头文件	库文件
Acclib 组件的安装目录 /acclib/include/acl/svp_acl_base.h	Acclib 组件的安装目录 /acclib/lib64/stub/libsvp_acl.so

头文件	库文件
Acllib 组件的安装目录 /acllib/include/acl/svp_acl.h	
Acllib 组件的安装目录 /acllib/include/acl/svp_acl_md1.h	
Acllib 组件的安装目录 /acllib/include/acl/svp_acl_rt.h	
Acllib 组件的安装目录 /acllib/include/acl/svp_acl_ext.h	
Acllib 组件的安装目录 /acllib/include/acl/svp_acl_prof.h	

说明

- 编译通过后，运行应用时，通过配置环境变量，应用会链接到运行环境上“Acllib 组件的安装目录/acllib/lib64”目录下的*.so 库，运行时会自动链接到依赖其它组件的*.so 库。
- 编译基于 SVP ACL 接口的代码逻辑时，请按照引用的头文件，依赖对应的 so 文件，引用多余的 so 文件可能导致后续版本升级时存在兼容性问题。

编译选项修改可执行文件的名称（与 `add_executable` 保持一致），以及可执行文件的安装目录。

示例如下，表示 *main* 安装在`${CMAKE_INSTALL_PREFIX}/out` 目录下，

`${CMAKE_INSTALL_PREFIX}`变量定义的路径是相对路径，相对 `cmake` 命令执行的路径：

```
set(CMAKE_RUNTIME_OUTPUT_DIRECTORY "../out")
install(TARGETS main DESTINATION ${CMAKE_RUNTIME_OUTPUT_DIRECTORY})
```

- 关于 `cmake` 参数的详细介绍，请参见 <https://cmake.org/cmake/help/latest/guide/tutorial/index.html>，选择对应的版本后查看参数。
- 运行可执行文件时，需将 SVP ACL 初始化配置文件（`acl.json`）所在的目录、可执行文件所在的目录、测试图片所在的目录、*.om 文件所在的目录都上传到板端环境的同一个目录下。

如果在 SVP ACL 初始化阶段，在 `svp_acl_init` 接口中传入空指针，则无需将 SVP ACL 初始化配置文件（`acl.json`）所在的目录上传到板端环境。

8

开发典型功能点的介绍

8.1 Stream 管理

8.1.1 原理介绍

在 SVP ACL 中，Stream 是一个任务队列，应用程序通过 Stream 来管理任务的并行，一个 Stream 内部的任务保序执行，即 Stream 根据发送过来的任务依次执行；不同 Stream 中的任务并行执行。一个默认 Context 下会挂一个默认 Stream，如果不显式创建 Stream，可使用默认 Stream。默认 Stream 作为接口入参时，直接传 NULL。

8.1.2 单线程单 Stream

调用接口后，需增加异常处理的分支，示例代码中不一一列举。以下是关键步骤的代码示例，不可以直接拷贝编译运行，仅供参考。

```
#include "acl/svp_acl.h"
//.....

//显式创建一个Stream

svp_acl_rt_stream stream;
svp_acl_rt_create_stream(&stream);
uint32_t model_id = 0;

//调用触发任务的接口，例如异步模型推理，任务下发在stream1

svp_acl_md_dataset *input;
svp_acl_md_dataset *output;

svp_acl_md_execute_async(model_id, input, output, stream);

//调用svp_acl_rt_synchronize_stream接口，阻塞应用程序运行，直到指定Stream中的所有任务都完成。
```

```
svp_acl_rt_synchronize_stream(stream);

//Stream使用结束后，显式销毁Stream
svp_acl_rt_destroy_stream(stream);
//.....
```

8.1.3 单线程多 Stream

调用接口后，需增加异常处理的分支，示例代码中不一一列举。以下是关键步骤的代码示例，不可以直接拷贝编译运行，仅供参考。

```
#include "acl/svp_acl.h"
//.....
int32_t device_id = 0 ;
uint32_t model_id1 = 0;
uint32_t model_id2 = 1;
svp_acl_rt_context context;
svp_acl_rt_stream stream1;
svp_acl_rt_stream stream2;

//如果只创建了一个Context，线程默认将这个Context作为线程当前的Context;
//如果是多个Context，则需要调用svp_acl_rt_set_current_context接口设置当前线程的Context
svp_acl_rt_create_context(&context, device_id);

svp_acl_rt_create_stream(&stream1);
//调用触发任务的接口，例如异步模型推理，任务下发在stream1
svp_acl_md_dataset *input1;
svp_acl_md_dataset *output1;
svp_acl_md_execute_async(model_id1, input1, output1, stream1);

svp_acl_rt_create_stream(&stream2);
//调用触发任务的接口，例如异步模型推理，任务下发在stream2
svp_acl_md_dataset *input2;
svp_acl_md_dataset *output2;
svp_acl_md_execute_async(model_id2, input1, output2, stream2);

// 流同步
svp_acl_rt_synchronize_stream(stream1);
```

```
svp_acl_rt_synchronize_stream(stream2);

//释放资源
svp_acl_rt_destroy_stream(stream1);
svp_acl_rt_destroy_stream(stream2);
svp_acl_rt_destroy_context(context);
//....
```

8.1.4 多线程多 Stream

调用接口后，需增加异常处理的分支，示例代码中不一一列举。以下是关键步骤的代码示例，不可以直接拷贝编译运行，仅供参考。

```
#include "acl/svp_acl.h"
//.....
void run_thread(svp_acl_rt_stream stream) {
    int32_t device_id = 0;
    svp_acl_rt_context context;

    //如果只创建了一个Context，线程默认将这个Context作为线程当前的Context；
    //如果是多个Context，则需要调用svp_acl_rt_set_current_context接口设置当前线程的Context
    svp_acl_rt_create_context(&context, device_id);
    svp_acl_rt_create_stream(&stream);

    //调用触发任务的接口
    //....

    //释放资源
    svp_acl_rt_destroy_stream(stream);
    svp_acl_rt_destroy_context(context);
}

svp_acl_rt_stream stream1;
svp_acl_rt_stream stream2;

//创建2个线程，每个线程对应一个Stream
std::thread t1(runThread, stream1);
std::thread t2(runThread, stream2);

//显式调用join函数确保结束线程
```

```
t1.join();
t2.join();
```

8.2 同步等待

8.2.1 原理介绍

SVP ACL 提供以下几种同步机制：

- Stream 内任务的同步等待：调用 `svp_acl_rt_synchronize_stream` 接口，阻塞应用程序运行，直到指定 Stream 中的所有任务都完成。
- Device 的同步等待：调用 `svp_acl_rt_synchronize_device` 接口，阻塞应用程序运行，直到正在运算中的 Device 完成运算。多 Device 场景下，调用该接口等待的是当前 Context 对应的 Device。

8.2.2 关于 Stream 内任务的同步等待

调用接口后，需增加异常处理的分支，示例代码中不——列举。以下是关键步骤的代码示例，不可以直接拷贝编译运行，仅供参考。

```
#include "acl/svp_acl.h"
//.....

//显式创建一个Stream
svp_acl_rt_stream stream;
svp_acl_rt_create_stream(&stream);
uint32_t model_id = 0;

//调用触发任务的接口，例如异步模型推理，任务下发在stream1
svp_acl_md_dataset *input;
svp_acl_md_dataset *output;
svp_acl_md_execute_async(model_id, input, output, stream);

//调用svp_acl_rt_synchronize_stream接口，阻塞应用程序运行，直到指定Stream中的所有任务都完成。
svp_acl_rt_synchronize_stream(stream);

//Stream使用结束后，显式销毁Stream
svp_acl_rt_destroy_stream(stream);
//.....
```

8.2.3 关于 Device 的同步等待

调用接口后，需增加异常处理的分支，示例代码中不一一列举。以下是关键步骤的代码示例，不可以直接拷贝编译运行，仅供参考。

```
#include "acl/svp_acl.h"
//.....
//指定device
svp_acl_rt_set_device(0);

//创建context
svp_acl_rt_context ctx;
svp_acl_rt_create_context(&ctx, 0);

//创建stream
svp_acl_rt_stream stream;
svp_acl_rt_create_stream(&stream);

//阻塞应用程序运行，直到正在运算中的Device完成运算
svp_acl_rt_synchronize_device();

//资源销毁
svp_acl_rt_destroy_stream(stream);
svp_acl_rt_destroy_context(ctx);
svp_acl_rt_reset_device(0);
```

8.3 模型推理

8.3.1 单 Batch+固定 shape+单模型同步推理

1. 若涉及色域转换（转换图像格式）、图像归一化（减均值/乘系数）等，在模型加载前，需要先参见《ATC 工具使用指南》转换模型。
2. 在模型推理前，需要从离线模型文件（适配 SoC 的离线模型）中加载模型数据到内存中，并创建 `svp_acl_md_dataset` 类型的数据描述模型的输出。请参见“[7.3.1.3 模型推理资源申请](#)”。
3. 加载模型后，再同步执行模型推理。请参见“[7.3.3 模型推理](#)”。

8.3.2 异步推理+callback 回调处理

基本原理：

异步推理场景下，关键接口的调用流程如下：

1. 调用 `sva_acl_init` 接口初始化 SVP ACL。
2. 按顺序调用 `sva_acl_rt_set_device`、`sva_acl_rt_create_context`、`sva_acl_rt_create_stream` 接口依次申请运行管理资源：Device、Context、Stream，确保可以使用这些资源执行运算、管理任务。

如果不显式创建 Context 和 Stream，您可以使用 `sva_acl_rt_set_device` 接口隐式创建的默认 Context 和默认 Stream，但默认 Context 和默认 Stream 存在如下限制：

- 一个 Device 对应一个默认 Context，默认 Context 不能通过 `sva_acl_rt_destroy_context` 接口来释放。
- 一个 Device 对应一个默认 Stream，默认 Stream 不能通过 `sva_acl_rt_destroy_stream` 接口来释放。默认 Stream 作为接口入参时，直接传 NULL。

3. 申请模型推理资源。

- a. 加载模型。

`sva_acl_mdload_from_mem`：从内存加载离线模型数据。

- b. 调用 `sva_acl_mdload_get_desc` 接口获取成功加载的模型的描述信息。
- c. 初始化内存，用于存放模型推理的输入数据、输出数据。

此处需要用户可自行定义函数实现以下关键点：

- i. 调用 `sva_acl_mdload_create_dataset` 接口创建 `sva_acl_mdload_dataset` 类型的数据（描述模型的输入/输出）。
- ii. 模型可能存在多个输入/输出，调用 `sva_acl_create_data_buffer` 接口创建 `sva_acl_data_buffer` 类型的数据（描述每个输入/输出数据的内存地址、内存大小）。

模型输入数据的内存大小，根据实际读入的图片数据的大小来确定。

模型输出数据的内存大小，可调用

`sva_acl_mdload_get_output_size_by_index` 接口根据 3.b 中模型描述信息获取每个输出需占用的内存大小。

- iii. 调用 `sva_acl_mdload_add_dataset_buffer` 接口向 `sva_acl_mdload_dataset` 中增加 `sva_acl_data_buffer`。

4. 执行模型异步推理和 callback（用于处理模型推理的结果）。

此处需要用户可自行定义函数实现以下关键点：

- a. 创建新线程（例如 t1），在线程函数内调用 `svp_acl_rt_process_report` 接口，等待指定时间后，触发回调函数（例如 `CallBackFunc`，用于处理模型推理结果）。
 - b. 调用 `svp_acl_rt_subscribe_report` 接口，指定处理 Stream 上回调函数（`CallBackFunc`）的线程（t1），回调函数需要提前创建，用于处理模型推理的结果（例如，将推理结果写入文件、从推理结果中获取 topn 置信度的类别标识等）。
 - c. 调用 `svp_acl_md1_execute_async` 接口执行异步模型推理。
 - d. 调用 `svp_acl_rt_launch_callback` 接口，在 Stream 的任务队列中增加一个需要在板端环境中执行的回调函数（`CallBackFunc`）。
 - e. 调用 `svp_acl_rt_synchronize_stream` 接口，阻塞应用程序运行，直到指定 Stream 中的所有任务都完成。
 - f. 调用 `svp_acl_rt_unsubscribe_report` 接口，取消线程注册，Stream 上的回调函数（`CallBackFunc`）不再由指定线程（t1）处理。
 - g. 模型推理结束后，调用 `svp_acl_md1_unload` 接口卸载模型。
5. 所有数据处理结束后，按顺序调用 `svp_acl_rt_destroy_stream`、`svp_acl_rt_destroy_context`、`svp_acl_rt_reset_device` 接口依次释放运行管理资源，包括 Stream、Context、Device。
6. 调用 `svp_acl_finalize` 接口实现 SVP ACL 去初始化。

示例代码：

您可以从 `acl_resnet50_async` 样例中查看完整样例代码，以下是关键步骤的代码示例，不可以直接拷贝编译运行，仅供参考。

在 `acl_resnet50_async` 样例中：

- 运行可执行文件，不带参数时：
 - 执行模型异步推理的次数默认为 10 次，对应代码中的 `g_executeTimes` 变量；
 - callback 间隔默认为 1，表示 1 次异步推理后，下发一次 callback 任务，对应代码中的 `g_callbackInterval` 变量；
 - 内存池中的内存块的个数默认为 10 个，对应代码中的 `g_memoryPoolSize`。
- 运行可执行文件，带参数时：

- 第一个参数表示执行模型异步推理的次数；
- 第二个参数表示下发 callback 间隔，参数值为 0 时表示不下发 callback 任务，参数值为非 0 值（例如 m）时表示 m 次异步推理后下发一次 callback 任务；
- 第三个参数表示内存池中内存块的个数，内存块个数需大于等于模型异步推理的次数。用户可根据输入图片数量，来调整内存块的个数，例如有 2 张输入图片、内存块个数为 2 时，则 1 张图片 1 个内存块；例如有 3 张输入图片、内存块个数为 10 时，则执行 10 次循环，每（10/3 取整）个内存块对应同一张图片，剩下 1 个内存块随机对应 1 张图片。内存块中存放是模型推理的输入数据、输出数据，若多个内存块对应的是同一张图片，则多个内存块中存放的是相同的输入数据、输出数据，用于输入图片少但又想模拟大量图片数据的场景。

```
#include "acl/svp_acl.h"
//.....
//1. SVP ACL 初始化
//此处的..表示相对路径，相对可执行文件所在的目录
//例如，编译出来的可执行文件存放在 out 目录下，此处的..就表示 out 目录的上一级目录
const char *aclConfigPath = "../src/acl.json";
svp_acl_error ret = svp_acl_init(aclConfigPath);

//2. 申请运行管理资源
extern bool g_isDevice;
ret = svp_acl_rt_set_device(deviceId_);
ret = svp_acl_rt_create_context(&context_, deviceId_);
ret = svp_acl_rt_create_stream(&stream_);
//获取当前 svp_acl 软件栈的运行模式，根据不同的运行模式，后续的内存申请、内存复制等接口调用方式不同
svp_acl_rt_run_mode run_mode;
ret = svp_acl_rt_get_run_mode(&runMode);
g_isDevice = (runMode == ACL_DEVICE);

//3. 申请模型推理资源
//此处的..表示相对路径，相对可执行文件所在的目录
//例如，编译出来的可执行文件存放在 out 目录下，此处的..就表示 out 目录的上一级目录
const char* omModelPath = "../model/resnet50.om"

//3.1 加载模型
//根据模型文件获取模型执行时所需的权值内存大小、工作内存大小，并申请权值内存、工作内存
ret = svp_acl_rt_malloc(&modelMemPtr_, modelMemSize_,
SVP_ACL_MEM_MALLOC_NORMAL_ONLY);

//加载离线模型文件，模型加载成功，返回标识模型的 ID。
ret = svp_acl_mdload_from_mem(modelPath, &modelId_, modelMemPtr_,
modelMemSize_);

//3.2 根据模型的 ID，获取该模型描述信息
```



```

modelDesc_ = svp_acl_md1_create_desc();
ret = svp_acl_md1_get_desc(modelDesc_, modelId_);

//3.3 自定义函数 InitMemPool, 初始化内存池, 存放模型推理的输入数据、输出数据
//-----自定义函数 InitMemPool 内部的关键实现-----
string testFile[] = {
    "../data/dog1_1024_683.bin",
    "../data/dog2_1024_683.bin"
};
size_t fileNum = sizeof(testFile) / sizeof(testFile[0]);
for (size_t i = 0; i < g_memoryPoolSize; ++i) {
    size_t index = i % (sizeof(testFile) / sizeof(testFile[0]));
    // model process
    uint32_t devBufferSize;
    //自定义函数 GetDeviceBufferOfFile, 完成以下功能:
    //获取 Host 上存放输入图片数据的内存及内存大小、将图片数据从 Host 传输到 Device、获取
    Device 上存放输入图片数据的内存及内存大小
    void *picDevBuffer = Utils::GetDeviceBufferOfFile(testFile[index], devBufferSize);
    aclmdlDataset *input = nullptr;
    //自定义函数 CreateInput, 创建 aclmdlDataset 类型的数据 input, 用于存放模型推理的输入数据
    Result ret = CreateInput(picDevBuffer, devBufferSize, input);
    svp_acl_md1_dataset *output = nullptr;
    //自定义函数 CreateOutput, 创建 svp_acl_md1_dataset 类型的数据 output, 用于存放模型推理
    的输出数据, modelDesc 表示模型描述信息
    CreateOutput(output, modelDesc);
    {
        std::lock_guard<std::recursive_mutex> lk(freePoolMutex_);
        freeMemoryPool_[input] = output;
    }
}
//-----自定义函数 InitMemPool 内部的关键实现-----

//4 模型推理
//4.1 创建线程 tid, 并将该 tid 线程指定为处理 Stream 上回调函数的线程
//其中 ProcessCallback 为线程函数, 在该函数内调用 svp_acl_rt_process_report 接口, 等待指定时间后, 触发回调函数处理
pthread_t tid;
(void)pthread_create(&tid, nullptr, ProcessCallback, &s_isExit);

//4.2 指定处理 Stream 上回调函数的线程
svp_acl_error aclRt = svp_acl_rt_subscribe_report(tid, stream_);

//4.2 创建回调函数, 用户处理模型推理的结果, 由用户自行定义
void ModelProcess::CallBackFunc(void *arg)
{
    std::map<svp_acl_md1_dataset *, svp_acl_md1_dataset *> *dataMap =
        (std::map<svp_acl_md1_dataset *, svp_acl_md1_dataset *> *)arg;

    svp_acl_md1_dataset *input = nullptr;
    svp_acl_md1_dataset *output = nullptr;
    MemoryPool *memPool = MemoryPool::Instance();

```

```

        for (auto& data : *dataMap) {
            ModelProcess::OutputModelResult(data.second);
            memPool->FreeMemory(data.first, data.second);
        }

        delete dataMap;
    }

//4.3 自定义函数 ExecuteAsync, 执行模型推理
//-----自定义函数 ExecuteAsync 内部的关键实现-----
bool isCallback = (g_callbackInterval != 0);
size_t callbackCnt = 0;
std::map<sva_acl_md1_dataset *, sva_acl_md1_dataset *> *dataMap = nullptr;
sva_acl_md1_dataset *input = nullptr;
sva_acl_md1_dataset *output = nullptr;
MemoryPool *memPool = MemoryPool::Instance();
for (uint32_t cnt = 0; cnt < g_executeTimes; ++cnt) {
    if (memPool->mallocMemory(input, output) != SUCCESS) {
        ERROR_LOG("get free memory failed");
        return FAILED;
    }
    //执行异步推理
    sva_acl_error ret = sva_acl_md1_execute_async(modelId_, input, output, stream_);

    if (isCallback) {
        if (dataMap == nullptr) {
            dataMap = new std::map<sva_acl_md1_dataset *, sva_acl_md1_dataset *>;
            if (dataMap == nullptr) {
                ERROR_LOG("malloc list failed, modelId is %u", modelId_);
                memPool->FreeMemory(input, output);
                return FAILED;
            }
        }
        (*dataMap)[input] = output;
        callbackCnt++;
        if ((callbackCnt % g_callbackInterval) == 0) {
            //在 Stream 的任务队列中增加一个需要在 Host 上执行的回调函数
            ret = sva_acl_rt_launch_callback(CallBackFunc, (void *)dataMap,
            SVA_ACL_CALLBACK_BLOCK, stream_);
            if (ret != ACL_SUCCESS) {
                ERROR_LOG("launch callback failed, index=%zu", callbackCnt);
                memPool->FreeMemory(input, output);
                delete dataMap;
                return FAILED;
            }
            dataMap = nullptr;
        }
    }
}

//-----自定义函数 ExecuteAsync 内部的关键实现-----

//4.4 对于异步推理, 需阻塞应用程序运行, 直到指定 Stream 中的所有任务都完成
sva_acl_rt_synchronize_stream(stream_);

```

```
//4.5 取消线程注册，Stream 上的回调函数不再由指定线程处理
aclRt = svp_acl_rt_unsubscribe_report(static_cast<uint64_t>(tid), stream_);
s_isExit = true;
(void)pthread_join(tid, nullptr);

//5 释放运行管理资源
svp_acl_error ret = svp_acl_rt_destroy_stream(stream_);
ret = svp_acl_rt_destroy_context(context_);
ret = svp_acl_rt_reset_device(deviceId_);

//6 SVP ACL 去初始化
ret = svp_acl_finalize();
//.....
```

8.3.3 多模型推理

多模型推理的基本流程与单模型类似，请参见 [8.3.1 单 Batch+固定 shape+单模型同步推理](#)。

多模型推理与单模型推理的不同点如下：

- 关于模型加载，如果涉及多个模型，需调用多次模型加载接口。
[svp_acl_mdload_from_mem](#)：从内存加载离线模型数据。
- 关于模型推理，如果涉及多个模型，需调用多次模型推理接口。
调用 [svp_acl_mdexecute](#) 接口实现同步模型推理。

8.4 带 cache 属性的内存管理

基本原理：

通过 [svp_acl_rt_malloc_cached](#) 接口申请的内存支持 cache 缓存，对于频繁使用的内存，建议使用本接口分配内存，这样可以提高 cpu 的读写效率，提升系统性能，但需要用户处理 cpu 与硬件设备（如[图像分析引擎](#)）之间的数据一致性问题，SVP ACL 提供了 [svp_acl_rt_mem_flush](#) 接口将 cache 中的数据刷新到 ddr 中，从而确保硬件设备（如[图像分析引擎](#)）可以从 ddr 中获取最新数据进行处理，SVP ACL 还提供了 [svp_acl_rt_mem_invalidate](#) 接口将 cache 中的数据置为无效（如果 cache 被 CPU 写操作，调用该接口功能会将数据刷到 ddr），从而确保 cpu 可以获取到 ddr 中的经[图像分析引擎](#)处理后的结果数据。

如果任务（例如，模型加载）不涉及 cpu 与硬件设备（如[图像分析引擎](#)）之间的数据交互，则不涉及两者之间的数据一致性问题，因此就不需要调用 [svp_acl_rt_mem_flush](#) 接口和 [svp_acl_rt_mem_invalidate](#) 接口。

示例代码：

调用接口后，需增加异常处理的分支，示例代码中不——列举。以下是关键步骤的代码示例，不可以直接拷贝编译运行，仅供参考。

示例中，运行管理资源申请与释放请参见[“4.2 运行管理资源申请”](#)和[“4.6 运行管理资源释放”](#)，模型加载的接口调用流程请参见[“4.3 模型加载”](#)，模型推理的接口调用流程、准备模型推理的输入/输出数据的接口调用流程请参见[“4.4 模型执行”](#)。

```
//1.申请运行管理资源，包括设置用于计算的 Device、创建 Context、创建 Stream
//.....

//2.模型加载，加载成功后，返回标识模型的 model_id
//.....

//3.创建 svp_acl_mdl_dataset 类型的数据，用于描述模型的输入数据 input、输出数据 output
void *input_dev_buffer = null;
void *output_dev_buffer = null;
size_t input_size;
size_t output_size;

//3.1 申请支持 cache 缓存的内存：input_dev_buffer 用于存放模型推理的输入数据
svp_acl_error ret = svp_acl_rt_malloc_cached(&input_dev_buffer, input_size,
SVP_ACL_MEM_MALLOC_NORMAL_ONLY);
svp_acl_mdl_dataset *input = svp_acl_mdl_create_dataset();
svp_acl_data_buffer* input_data = svp_acl_create_data_buffer(input_dev_buffer, input_size);
ret = svp_acl_mdl_add_dataset_buffer(input, input_data);
//将输入数据读入内存中，该自定义函数 ReadFile 由用户实现
read_file(file_name, input_dev_buffer, input_size);

//3.2 申请支持 cache 缓存的内存：output_dev_buffer 用于存放模型推理的输出数据
ret = svp_acl_rt_malloc_cached(&output_dev_buffer, output_size,
SVP_ACL_MEM_MALLOC_NORMAL_ONLY);
svp_acl_mdl_dataset *output = svp_acl_mdl_create_dataset();
svp_acl_data_buffer* output_data = svp_acl_create_data_buffer(output_dev_buffer,
size);
ret = svp_acl_mdl_add_dataset_buffer(output, output_data);

//4.把对应 cache 中的数据刷新到 ddr
//示例以获取第一个输入为例，如果有多个输入，可调用 svp_acl_mdl_get_dataset_num_buffers 接口获取输入个数
svp_acl_data_buffer *input_data_buffer = svp_acl_mdl_get_dataset_buffer(input, 0);
void *input_data_buffer_addr = svp_acl_get_data_buffer_addr(input_data_buffer);
size_t input_data_buffer_size = svp_acl_get_data_buffer_size(input_data_buffer);
ret = svp_acl_rt_mem_flush(input_data_buffer_addr, input_data_buffer_size);

//5.执行模型
```

```
ret = svp_acl_mdl_execute(model_id, input, output);

//6.执行 svp_acl_rt_mem_invalidate 接口，把对应 cache 中的数据置为无效，确保 cpu 获取到的是 ddr 中的数据
//示例以获取第一个输出为例，如果有多个输出，可调用 svp_acl_mdl_get_dataset_num_buffers 接口获取输出个数
svp_acl_data_buffer *out_data_buffer = svp_acl_mdl_get_dataset_buffer(output, 0);
void *output_data_buffer_addr = svp_acl_get_data_buffer_addr(out_data_buffer);
size_t output_data_buffer_size = svp_acl_get_data_buffer_size(out_data_buffer);
ret = svp_acl_rt_mem_invalidate(output_data_buffer_addr, output_data_buffer_size);

//7.处理模型推理结果
//TODO

//8.释放内存
svp_acl_rt_free(input_dev_buffer);
svp_acl_rt_free(output_dev_buffer);

//9.释放描述模型输入/输出信息、内存等资源，卸载模型
//.....

//10.释放运行管理资源
//.....
```

8.5 推理使用模式识别 CPU 算子

基本原理：

SoC 推理场景下，如果模型中有 *模式识别* Core 不支持的算子，需要通过 *模式识别* CPU 算子来实现，这时，异步推理场景下可以调用 `svp_acl_ext_process_aacpu_task` 接口来接收 *模式识别* CPU 算子并在 CPU 上执行，同步推理场景下无需用户做特殊处理。下面就异步推理关于该接口的使用方式进行进一步介绍。

模式识别 CPU 算子在 CPU 侧执行时可能会引入一些问题，因此用户需要关注以下几点：

1. 为了不影响整个推理业务，需要创建 *模式识别* CPU 工作线程对 *模式识别* CPU 算子进行处理，确保 *模式识别* CPU 算子处理和推理业务同时在 CPU 侧运行，且可保证各自的执行效率。
2. 需要用户在 *模式识别* CPU 工作线程内，调用 `svp_acl_ext_process_aacpu_task` 接口来接收 *模式识别* CPU 算子并执行。
3. 用户场景差异比较大，需要根据自己的业务需求来部署 *模式识别* CPU 工作线程，例如线程数量，是否绑核。

示例代码：

调用接口后，需增加异常处理的分支，示例代码中不一一列举。以下是关键步骤的代码示例，不可以直接拷贝编译运行，仅供参考。

示例中，异步推理流程请参见“[8.3.2 异步推理+callback 回调处理](#)”。

```
//1.实现模式识别/CPU工作线程处理函数
static bool thread_exit_flag = false;
void *cpu_func_async_test(void *param)
{
    while (thread_exit_flag == false) {
        svp_acl_ext_process_aacpu_task(1000);
    }
    return NULL;
}

//2.主线程创建模式识别/CPU工作线程
static pthread_t g_tid;
pthread_create(&g_tid, NULL, cpu_func_async_test, NULL);

//3.异步推理流程
//.....

//4.释放线程
thread_exit_flag = true;
pthread_join(g_tid, NULL);
```

须知

编译代码的时候，需要匹配对应的头文件 svp_acl_ext.h，目前文件（如 SS626V100，分别在板端 SDK 版本的 amp/a55_linux/mpp/out 路径的 include/svp_nnn 和 lib/svp_nnn 目录下，其他解决方案类似参考）。

8.6 Profiling 性能数据采集

基本原理：

该章节下的接口用于 Profiling 采集性能数据，实现方式支持以下两种：

- 方式一：将采集到的 Profiling 数据写入文件，再使用 Profiling 工具解析该文件。
- 方式二：将采集到的 Profiling 数据解析后写入管道，由用户读入内存，再由用户调用 ACL 的接口获取性能数据。

实现方式详细说明请参考“[9.9.1 功能及约束说明](#)”。

示例代码 1：

调用接口后，需增加异常处理的分支，并记录报错日志、提示日志，此处不一一列举。以下是关键步骤的代码示例，不可以直接拷贝编译运行，仅供参考。

示例中，运行管理资源申请与释放、模型加载的接口调用流程、模型推理的接口调用流程、准备模型推理的输入/输出数据的接口调用流程请参见“[8.3.2 异步推理+callback 回调处理](#)”。

```
//1.SVP ACL 资源初始化、模型加载、数据创建

//2.创建管道（UNIX 操作系统下需要引用 C++标准库头文件 unistd.h），用于读取以及写入模型订阅的数据
int sub_fd[2];
// 读管道指针指向 sub_fd[0]，写管道指针指向 sub_fd[1]
pipe(sub_fd);

//3.创建模型订阅的配置并且进行模型订阅
svp_acl_prof_subscribe_config *config = svp_acl_prof_create_subscribe_config(1, 0, &sub_fd[1]);
//模型订阅需要传入模型的 model_id
svp_acl_prof_model_subscribe(model_id, config);

//4.实现管道读取订阅数据的函数
//4.1 自定义函数，实现从用户内存中读取订阅数据的函数
void get_model_info(void *data, uint32_t len) {
    uint32_t op_number = 0;
    uint32_t data_len = 0;
    //通过 SVP ACL 接口读取算子信息个数
    svp_acl_prof_get_op_num(data, len, &op_number);
    //遍历用户内存的算子信息
    for (int32_t i = 0; i < op_number; i++){
        //获取算子的模型 id
        uint32_t model_id = svp_acl_prof_get_model_id(data, len, i);
        //获取算子的类型名称长度
        size_t op_type_len = 0;
        svp_acl_prof_get_op_type_len(data, len, i, &op_type_len);
        //获取算子的类型名称
        char op_type[op_type_len];
        svp_acl_prof_get_op_type(data, len, i, op_type, op_type_len);
        //获取算子的详细名称长度
        size_t op_name_len = 0;
        svp_acl_prof_get_op_name_len(data, len, i, &op_name_len);
        //获取算子的详细名称
        char op_name[op_name_len];
        svp_acl_prof_get_op_name(data, len, i, op_name, op_name_len);
        //获取算子的执行开始时间
        uint64_t op_start = svp_acl_prof_get_op_start(data, len, i);
        //获取算子的执行结束时间
        uint64_t op_end = svp_acl_prof_get_op_end(data, len, i);
        uint64_t op_duration = svp_acl_prof_get_op_duration(data, len, i);
    }
}

//4.2 自定义函数，实现从管道中读取数据到用户内存的函数
```

```

void *prof_data_read(void *fd) {
    //设置每次从管道中读取的算子信息个数
    uint64_t N = 10;
    //获取单位算子信息的大小 (Byte)
    uint64_t buffer_size = 0;
    svp_acl_prof_get_op_desc_size(&buffer_size);
    //计算存储算子信息的内存的大小, 并且申请内存
    uint64_t read_buf_len = buffer_size * N;
    char *read_buf = (char *)malloc(sizeof(char) * read_buf_len);
    //从管道中读取数据到申请的内存中, 读取到的实际数据大小 data_len 可能小于 buffer_size * N, 如果管道中
    //没有数据, 默认会阻塞直到读取到数据为止
    uint32_t data_len = read(*(int*)fd, read_buf, read_buf_len);
    //读取数据到 read_buf 成功
    while (data_len > 0) {
        //调用 5.1 实现的函数解析内存中的数据
        get_model_info(read_buf, data_len);
        memset(read_buf, 0, buffer_size);
        data_len = read(*(int*)fd, read_buf, read_buf_len);
    }
    free(read_buf);
}

//5. 启动线程读取管道数据并解析
pthread_t sub_tid = 0;
pthread_create(&sub_tid, NULL, prof_data_read, &sub_fd[0]);

//6. 执行模型
ret = svp_acl_md_execute(model_id, input, output);

//7. 处理模型推理结果、释放资源、卸载模型、等待线程数据获取完毕
//8. 取消订阅, 释放订阅相关资源
svp_acl_prof_model_unsubscribe(model_id);
close(sub_fd[1]);
pthread_join(sub_tid, NULL);
close(sub_fd[0]);
//释放 config 指针
svp_acl_prof_destroy_subscribe_config(config);

//9. 释放运行管理资源, SVP ACL 去初始化
//.....

```

示例代码 2:

调用接口后, 需增加异常处理的分支, 示例代码中不一一列举。以下是关键步骤的代码示例, 不可以直接拷贝编译运行, 仅供参考。

示例中, 运行管理资源申请与释放、模型加载的接口调用流程、模型推理的接口调用流程、准备模型推理的输入/输出数据的接口调用流程请参见“[8.3.2 异步推理+callback 回调处理](#)”。

```
//1.SVP ACL 资源初始化、模型加载、数据创建
```



```

//2.profilng 初始化
//设置数据落盘路径
const char *prof_path = "./";
svp_acl_prof_init(prof_path, strlen(prof_path));

//3.进行 profiling 配置
uint32_t device_id_list[1] = {0};
//创建配置结构体
svp_acl_prof_config *config = svp_acl_prof_create_config(device_id_list, 1, 0, NULL,
SVP_ACL_PROF_AACORE_METRICS | SVP_ACL_PROF_AACPU);
svp_acl_prof_start(config);

//4.执行模型
ret = svp_acl_md_execute(model_id, input, output);

//5.处理模型推理结果、释放资源、卸载模型

//6.关闭 profiling 配置, 释放配置资源, 释放 profiling 组件资源
svp_acl_prof_stop(config);
svp_acl_prof_destroy_config(config);
svp_acl_prof_finalize();

//7.释放运行管理资源, SVP ACL 去初始化.
//.....

```

JSON 文件格式:

通过 `svp_acl_init` 指定带有 Profiling 配置的 json 文件也可以实现将采集到的 Profiling 数据写入文件的功能, 文件格式如下:

```

{
  "profiler": {
    "switch": "on",
    "output": "output",
    "interval": "20",
    "aac_metrics": "ArithmeticUtilization",
    "aacpu": "on",
    "acl_api": "on"
  }
}

```

profiler 参数配置说明:

- switch: Profiling 开关, 取值 on 或 off。可选参数。
on 表示开启 Profiling, off 表示关闭 Profiling; 如果缺失该参数或参数值不为 on, 则表示关闭 Profiling。
- output: Profiling 性能数据在板端环境上的落盘路径。可选参数。
Profiling 采集结束后, 在该目录下生成 JOB 开头目录, 存放 Profiling 采集的性能原始数据。支持配置绝对路径或相对路径 (相对执行命令行时的当前路径):

- 绝对路径配置以“/”开头，例如：/home/output。
- 相对路径配置直接以目录名开始，例如：output。
- 如果该处设置的目录不存在，默认存放采集结果数据到应用工程可执行文件所在目录（确保安装时配置的运行用户具有该目录的读写权限）。

⚠ 注意

该参数指定的目录需要提前创建且确保安装时配置的运行用户具有读写权限。

- interval: Profiling 采集间隔，代表每 interval 次推理保存一次 Profiling 数据，此配置建议不要设置太小，否则可能由于落盘数据太慢而引起数据丢失。
- aac_metrics: 模式识别 Core 采集事件，配置为 ArithmeticUtilization 代表采集模式识别 Core 性能数据，否则为不采集。
- aacpu: 模式识别 CPU 采集事件，配置为 on 代表采集模式识别 CPU 性能数据，否则为不开启。
- acl_api: ACL API 采集事件，配置为 on 代表采集主要 ACL API 接口性能数据，否则为不开启。

9 SVP ACL API 参考

9.1 接口列表

一级目录	二级目录或接口或数据类型	三级目录或接口	是否支持
系统配置	svp_acl_init	-	支持
	svp_acl_finalize	-	支持
Device 管理	svp_acl_rt_set_device	-	支持
	svp_acl_rt_reset_device	-	支持
	svp_acl_rt_get_device	-	支持
	svp_acl_rt_get_run_mode	-	支持
	svp_acl_rt_get_device_count	-	支持
Context 管理	svp_acl_rt_create_context	-	支持
	svp_acl_rt_destroy_context	-	支持
	svp_acl_rt_set_current_context	-	支持
	svp_acl_rt_get_current_context	-	支持
Stream 管理	svp_acl_rt_create_stream	-	支持
	svp_acl_rt_destroy_stream	-	支持
同步等待	svp_acl_rt_synchronize_device	-	支持
	svp_acl_rt_synchronize_stream	-	支持

一级目录	二级目录或接口或数据类型	三级目录或接口	是否支持
	svp_acl_rt_subscribe_report	-	支持
	svp_acl_rt_launch_callback	-	支持
	svp_acl_rt_process_report	-	支持
	svp_acl_rt_unsubscribe_report	-	支持
	svp_acl_rt_set_op_wait_timeout	-	支持
内存管理	svp_acl_rt_malloc	-	支持
	svp_acl_rt_malloc_cached	-	支持
	svp_acl_rt_mem_flush	-	支持
	svp_acl_rt_mem_invalidate	-	支持
	svp_acl_rt_free	-	支持
	svp_acl_rt_malloc_host	-	支持
	svp_acl_rt_free_host	-	支持
模型加载与执行	svp_acl_mdl_load_from_mem	-	支持
	svp_acl_mdl_execute	-	支持
	svp_acl_mdl_execute_async	-	支持
	svp_acl_mdl_unload	-	支持
	svp_acl_mdl_set_dynamic_batch_size	-	支持
	svp_acl_mdl_set_total_t	-	支持
	svp_acl_mdl_init_dump	-	支持
	svp_acl_mdl_set_dump	-	支持
	svp_acl_mdl_finalize_dump	-	支持
	svp_acl_mdl_set_config_opt	-	支持
	svp_acl_mdl_load_with_config	-	支持

一级目录	二级目录或接口或数据类型	三级目录或接口	是否支持
	svp_acl_mdl_get_first_aapp_info	-	支持
	svp_acl_mdl_get_dynamic_batch	-	支持
Profiling 配置	svp_acl_prof_init	-	支持
	svp_acl_prof_start	-	支持
	svp_acl_prof_stop	-	支持
	svp_acl_prof_finalize	-	支持
	svp_acl_prof_model_subscribe	-	支持
	svp_acl_prof_model_unsubscribe	-	支持
	svp_acl_prof_get_op_desc_size	-	支持
	svp_acl_prof_get_op_num	-	支持
	svp_acl_prof_get_op_type_len	-	支持
	svp_acl_prof_get_op_type	-	支持
	svp_acl_prof_get_op_name_len	-	支持
	svp_acl_prof_get_op_name	-	支持
	svp_acl_prof_get_op_start	-	支持
	svp_acl_prof_get_op_stop	-	支持
	svp_acl_prof_get_op_duration	-	支持
	svp_acl_prof_get_model_id	-	支持
获取数据大小	svp_acl_data_type_size	-	支持
SoC 扩展接口	svp_acl_ext_process_aacpu_task	-	支持
	svp_acl_ext_get_mdl_aacpu_task_num	-	支持
	svp_acl_ext_get_mdl_net_type	-	支持
数据类型及其操作	svp_acl_error	-	支持

一级目录	二级目录或接口或数据类型	三级目录或接口	是否支持
接口	svp_acl_data_type	-	支持
	svp_acl_float16	-	支持
	svp_acl_format	-	支持
	svp_acl_rt_context	-	支持
	svp_acl_rt_stream	-	支持
	svp_acl_mdl_io_dims	-	支持
	svp_acl_data_buffer	svp_acl_create_data_buffer	支持
		svp_acl_destroy_data_buffer	支持
		svp_acl_get_data_buffer_addr	支持
		svp_acl_get_data_buffer_size	支持
		svp_acl_get_data_buffer_stride	支持
		svp_acl_update_data_buffer	支持
	svp_acl_mdl_dataset	svp_acl_mdl_create_dataset	支持
		svp_acl_mdl_destroy_dataset	支持
		svp_acl_mdl_add_dataset_buffer	支持
		svp_acl_mdl_get_dataset_num_buffers	支持
		svp_acl_mdl_get_dataset_buffer	支持
	svp_acl_mdl_desc	svp_acl_mdl_create_desc	支持
		svp_acl_mdl_destroy_desc	支持
		svp_acl_mdl_get_desc	支持
		svp_acl_mdl_get_num_i	支持

一级目录	二级目录或接口或数据类型	三级目录或接口	是否支持
		nputs	
		svp_acl_mdl_get_num_outputs	支持
		svp_acl_mdl_get_input_size_by_index	支持
		svp_acl_mdl_get_output_size_by_index	支持
		svp_acl_mdl_get_input_dims	支持
		svp_acl_mdl_get_output_dims	支持
		svp_acl_mdl_get_input_name_by_index	支持
		svp_acl_mdl_get_output_name_by_index	支持
		svp_acl_mdl_get_input_format	支持
		svp_acl_mdl_get_output_format	支持
		svp_acl_mdl_get_input_data_type	支持
		svp_acl_mdl_get_output_data_type	支持
		svp_acl_mdl_get_input_index_by_name	支持
		svp_acl_mdl_get_output_index_by_name	支持
		svp_acl_mdl_get_input_default_stride	支持
		svp_acl_mdl_get_output_default_stride	支持
	svp_acl_prof_config	svp_acl_prof_create_config	支持
		svp_acl_prof_destroy_config	支持
	svp_acl_prof_subscribe_config	svp_acl_prof_create_subscribe_config	支持
		svp_acl_prof_destroy_subscribe_config	支持

一级目录	二级目录或接口或数据类型	三级目录或接口	是否支持
	svp_acl_mdl_config_attr	-	支持
	svp_acl_mdl_config_handle	svp_acl_mdl_create_config_handle	支持
		svp_acl_mdl_destroy_config_handle	支持
	svp_acl_aapp_input_format svp_acl_aapp_info	svp_acl_mdl_get_first_aapp_info	支持

9.2 系统配置

9.2.1 `svp_acl_init`

函数功能：SVP ACL 初始化函数，同步接口。

约束说明：

- 一个进程内只能调用一次 `svp_acl_init` 接口。
- 使用 SVP ACL 接口开发应用时，必须先调用 `svp_acl_init` 接口，进行内部资源初始化，否则可能会导致业务异常。
- 进程退出时需要调用 `svp_acl_finalize` 接口释放内部资源。

函数原型：

```
svp_acl_error svp_acl_init(const char *config_path)
```

参数说明：

参数名	输入/输出	说明
config_path	输入	配置文件所在的路径，包含文件名，配置文件内容为 json 格式（json 文件内的“{”的层级最多为 10，“[”的层级最多为 10）。如果以下的默认配置已满足需求，无需修改，可向 <code>aclInit</code> 接口中传入 NULL，或者可将配置文件配置为空 json 串（即配置文件中只有 {}）。

参数名	输入/输出	说明
		配置文件格式为 json 格式，当前支持以下配置： • dump 信息配置，示例、配置说明及约束请参见《MindStudio 用户手册》中的“精度比对->推理场景数据准备”。默认不启动 dump 配置。dump 功能开启除了配置 json 文件外，OM 需要选择开启 dump 模式，如果 OM 开启 dump 后，json 未配置 dump，运行会报错，开启 dump 模式请参考《ATC 工具使用指南》。 • profiling 信息配置，配置说明请参见“8.6 Profiling 性能数据采集”。默认不启动 Profiling 配置。

返回值说明：返回 0 表示成功，返回其它值表示失败。

9.2.2 svp_acl_finalize

函数功能：SVP ACL 去初始化函数，用于释放进程内的 SVP ACL 内部管理的相关资源，同步接口。

约束说明：

- 用户申请的资源需要在调用 svp_acl_finalize 接口前通过调用对应的释放接口释放，防止未释放的资源造成内存泄露或者后续被误用而导致业务异常。
- 应用的进程退出前，应显式调用该接口实现 SVP ACL 去初始化，否则可能会导致异常，例如应用进程退出时有异常报错。
- 不建议在析构函数中调用 svp_acl_finalize 接口，否则在进程退出时可能由于单例析构顺序未知而导致进程异常退出的问题。

函数原型：

```
svp_acl_error svp_acl_finalize()
```

参数说明：无

返回值说明：返回 0 表示成功，返回其它值表示失败。

9.3 Device 管理

9.3.1 `svp_acl_rt_set_device`

函数功能：

- 指定用于运算的 Device，同时隐式创建默认 Context，该默认 Context 中包含 1 个默认 Stream，同步接口。
- 如果多次调用 `svp_acl_rt_set_device` 接口而不调用 `svp_acl_rt_reset_device` 接口释放本进程使用的 Device 资源，功能上不会有问题，因为在进程退出时也会释放本进程使用的 Device 资源。建议 `svp_acl_rt_set_device` 接口和 `svp_acl_rt_reset_device` 接口配对使用，在不使用 Device 上资源时，通过调用 `svp_acl_rt_reset_device` 接口及时释放本进程使用的 Device 资源。

支持以下使用场景：

- 在不同进程或线程中可指定同一个 Device 用于运算。
- 在某一进程中指定 Device，该进程内的多个线程可共用此 Device 显式创建 Context (`svp_acl_rt_create_context` 接口)。
- 多 Device 场景下，可在进程中通过 `svp_acl_rt_set_device` 接口切换到其它 Device。但利用 Context 切换 (调用 `svp_acl_rt_set_current_context` 接口) 来切换 Device，比使用 `svp_acl_rt_set_device` 接口效率高。

函数原型：

```
svp_acl_error svp_acl_rt_set_device(int32_t device_id)
```

参数说明：

参数名	输入/输出	说明
device_id	输入	Device ID。 用户调用 <code>svp_acl_rt_get_device_count</code> 接口获取可用的 Device 数量后，这个 Device ID 的取值范围：[0, (可用的 Device 数量-1)]

返回值说明：返回 0 表示成功，返回其它值表示失败。

9.3.2 `svp_acl_rt_reset_device`

函数功能：复位当前运算的 Device，释放 Device 上的资源，包括默认 Context、默认 Stream 以及默认 Context 下创建的所有 Stream，同步接口。若默认 Context 或默认 Stream 下的任务还未完成，会删除任务，不再执行。

约束说明：

- 若要复位的 Device 上存在显式创建的 Context、Stream，在复位前，建议遵循如下接口调用顺序，否则可能会导致业务异常。
- 接口调用顺序：**调用 `svp_acl_rt_destroy_stream` 接口释放显式创建的 Stream-->调用 `svp_acl_rt_destroy_context` 释放显式创建的 Context-->调用 `svp_acl_rt_reset_device` 接口

函数原型：

```
svp_acl_error svp_acl_rt_reset_device(int32_t device_id)
```

参数说明：

参数名	输入/输出	说明
device_id	输入	Device ID。

返回值说明：返回 0 表示成功，返回其它值表示失败。

9.3.3 `svp_acl_rt_get_device`

函数功能：获取当前正在使用的 Device 的 ID，同步接口。

约束说明：如果没有调用 `svp_acl_rt_set_device` 接口显式指定 Device 或没有调用 `svp_acl_rt_create_context` 接口隐式指定 Device，则调用 `svp_acl_rt_get_device` 接口时，返回错误。

函数原型：

```
svp_acl_error svp_acl_rt_get_device(int32_t *device_id)
```

参数说明：

参数名	输入/输出	说明
device_id	输出	Device ID。

返回值说明：返回 0 表示成功，返回其它值表示失败。

9.3.4 svp_acl_rt_get_run_mode

函数功能：获取当前 svp_acl 软件栈的运行模式，同步接口。

函数原型：

```
svp_acl_error svp_acl_rt_get_run_mode(svp_acl_rt_run_mode *run_mode)
```

参数说明：

参数名	输入/输出	说明
run_mode	输出	表示运行模式。 <pre>typedef enum svp_acl_rt_run_mode { //软件栈运行在SoC的CPU上 SVP_ACL_DEVICE, //SoC不支持 SVP_ACL_HOST, } svp_acl_rt_run_mode;</pre>

返回值说明：返回 0 表示成功，返回其它值表示失败。

9.3.5 svp_acl_rt_get_device_count

函数功能：获取可用 Device 的数量，同步接口。

函数原型：

```
svp_acl_error svp_acl_rt_get_device_count(uint32_t *count)
```

参数说明：

参数名	输入/输出	说明
count	输出	Device 数量。

返回值说明：返回 0 表示成功，返回其它值表示失败。

9.4 Context 管理

9.4.1 svp_acl_rt_create_context

函数功能：显式创建一个 Context，该 Context 中包含 1 个默认 Stream，同步接口。

支持以下使用场景：

- 若不调用 svp_acl_rt_create_context 接口显式创建 Context，那系统会使用默认 Context，该默认 Context 是在调用 [svp_acl_rt_set_device](#) 接口时隐式创建的。
 - 隐式创建 Context：适合简单、无复杂交互逻辑的应用，但缺点在于，在多线程编程中，执行结果取决于线程调度的顺序。
 - 显式创建 Context：**推荐显式**，适合大型、复杂交互逻辑的应用，且便于提高程序的可读性、可维护性。
- 若在某一进程内创建多个 Context（Context 的数量与 Stream 相关，Stream 数量有限，请参见 [svp_acl_rt_create_stream](#)），当前线程在同一时刻内只能使用其中一个 Context，建议通过 [svp_acl_rt_set_current_context](#) 接口明确指定当前线程的 Context，增加程序的可维护性。

函数原型：

```
svp_acl_error svp_acl_rt_create_context(svp_acl_rt_context *context, int32_t device_id)
```

参数说明：

参数名	输入/输出	说明
device_id	输入	指定需要创建 context 的 Device 的 ID。
context	输出	创建的 context 的指针。

返回值说明：返回 0 表示成功，返回其它值表示失败。

9.4.2 svp_acl_rt_destroy_context

函数功能：销毁一个 Context，释放 Context 的资源，同步接口。只能销毁通过 [svp_acl_rt_create_context](#) 接口创建的 Context。

函数原型：

```
svp_acl_error svp_acl_rt_destroy_context(svp_acl_rt_context context)
```

参数说明：

参数名	输入/输出	说明
context	输入	指定需要销毁的 context。

返回值说明：返回 0 表示成功，返回其它值表示失败。

9.4.3 svp_acl_rt_set_current_context

函数功能：设置线程的 Context，同步接口。

支持以下场景：

- 如果在某线程（例如：thread1）中调用 [svp_acl_rt_create_context](#) 接口显式创建一个 Context（例如：ctx1），则可以不调用 [svp_acl_rt_set_current_context](#) 接口指定该线程的 Context，系统默认将 ctx1 作为 thread1 的 Context。
- 如果没有调用 [svp_acl_rt_create_context](#) 接口显式创建 Context，则系统将默认 Context 作为线程的 Context，此时，不能通过 [svp_acl_rt_destroy_context](#) 接口来释放默认 Context。
- 如果多次调用 [svp_acl_rt_set_current_context](#) 接口设置线程的 Context，以最后一次为准。

约束说明：

- 若给线程设置的 Context 所对应的 Device 已经被复位，则不能将该 Context 设置为线程的 Context，否则会导致业务异常。
- 推荐在某一线程中创建的 Context，在该线程中使用。若在线程 A 中调用 [svp_acl_rt_create_context](#) 接口创建 Context，在线程 B 中使用该 Context，则需由用户自行保证两个线程中同一个 Context 下同一个 Stream 中任务执行的顺序。

函数原型：

```
svp_acl_error svp_acl_rt_set_current_context(svp_acl_rt_context context)
```

参数说明：

参数名	输入/输出	说明
context	输入	线程当前的 context。

返回值说明：返回 0 表示成功，返回其它值表示失败。

9.4.4 `svp_acl_rt_get_current_context`

函数功能：

- 获取线程的 Context，同步接口。
- 如果用户多次调用 `svp_acl_rt_set_current_context` 接口设置当前线程的 Context，则获取的是最后一次设置的 Context。

函数原型：

```
svp_acl_error svp_acl_rt_get_current_context(svp_acl_rt_context *context)
```

参数说明：

参数名	输入/输出	说明
context	输出	线程当前的 context。

返回值说明：返回 0 表示成功，返回其它值表示失败。

9.5 Stream 管理

9.5.1 `svp_acl_rt_create_stream`

函数功能：创建一个 Stream，同步接口。

约束说明：

- SoC-cs 形态，硬件资源最多支持 128 个 Stream，如果已存在多个默认 Stream，只能显式创建 N 个 Stream (N=128-默认 Stream 个数)，例如，若已存在一个默认 Stream，则只能显式创建 127 个 Stream。
- 每个 Context 对应一个默认 Stream，该默认 Stream 是调用 `svp_acl_rt_set_device` 接口或 `svp_acl_rt_create_context` 接口隐式创建的。推荐调用 `svp_acl_rt_create_stream` 接口显式创建 Stream。
- 隐式创建 Stream：适合简单、无复杂交互逻辑的应用，但缺点在于，在多线程编程中，执行结果取决于线程调度的顺序。
- 显式创建 Stream：**推荐显式**，适合大型、复杂交互逻辑的应用，且便于提高程序的可读性、可维护性。

函数原型：

```
svp_acl_error svp_acl_rt_create_stream(svp_acl_rt_stream *stream)
```

参数说明：

参数名	输入/输出	说明
stream	输出	新创建的 stream 的指针。

返回值说明：返回 0 表示成功，返回其它值表示失败。

9.5.2 svp_acl_rt_destroy_stream

函数功能：销毁指定 Stream，只能销毁通过 [svp_acl_rt_create_stream](#) 接口创建的 Stream，同步接口。

约束说明：

- 在调用 [svp_acl_rt_destroy_stream](#) 接口销毁指定 Stream 前，需要先调用 [svp_acl_rt_synchronize_stream](#) 接口确保 Stream 中的任务都已完成。
- 调用 [svp_acl_rt_destroy_stream](#) 接口销毁指定 Stream 时，需确保该 Stream 在当前 Context 下。
- 在调用 [svp_acl_rt_destroy_stream](#) 接口销毁指定 Stream 时，需确保其它接口没有正在使用该 Stream。

函数原型：

```
svp_acl_error svp_acl_rt_destroy_stream(svp_acl_rt_stream stream)
```

参数说明：

参数名	输入/输出	说明
stream	输入	待销毁的 stream。

返回值说明：返回 0 表示成功，返回其它值表示失败。

9.6 同步等待

9.6.1 svp_acl_rt_synchronize_device

函数功能：

- 阻塞应用程序运行，直到正在运算中的 Device 完成运算，同步接口。
- 多 Device 场景下，调用该接口等待的是当前 Context 对应的 Device。

约束说明：一个进程内多线程场景下，不支持多线程并发调用
svp_acl_rt_synchronize_device 接口。

函数原型：

```
svp_acl_error svp_acl_rt_synchronize_device(void)
```

参数说明：无

返回值说明：返回 0 表示成功，返回其它值表示失败。

9.6.2 svp_acl_rt_synchronize_stream

函数功能：阻塞应用程序运行，直到指定 Stream 中的所有任务都完成，同步接口。

函数原型：

```
svp_acl_error svp_acl_rt_synchronize_stream(svp_acl_rt_stream stream)
```

参数说明：

参数名	输入/输出	说明
stream	输入	指定需要完成所有任务的 stream，如果为 NULL 则调用默认 stream。

返回值说明：返回 0 表示成功，返回其它值表示失败。

9.6.3 svp_acl_rt_subscribe_report

函数功能：注册处理 Stream 上回调函数的线程。同步接口。

约束说明：

- 支持多次调用 svp_acl_rt_subscribe_report 接口给多个 Stream ((仅支持同一 Device 内的多个 Stream)) 注册同一个处理回调函数的线程；
- 为确保 Stream 内的任务按调用顺序执行，不支持调用
svp_acl_rt_subscribe_report 接口给同一个 Stream 注册多个处理回调函数的线程；
- 单进程内调用 svp_acl_rt_subscribe_report 接口注册的线程数量如果超过 128 个，则接口返回失败；

- 考虑操作系统的线程切换性能开销，建议调用 `svp_acl_rt_subscribe_report` 接口注册的线程数量控制在 32 个以下（包括 32）；
- 同一个进程内，在不同的 Device 上注册回调函数的线程时，不能指定同一个线程 ID。

函数原型：

```
svp_acl_error svp_acl_rt_subscribe_report(uint64_t thread_id, svp_acl_rt_stream stream)
```

参数说明：

参数名	输入/输出	说明
threadId	输入	指定线程的 ID。
stream	输入	指定 Stream。

返回值说明：返回 0 表示成功，返回其它值表示失败。

9.6.4 svp_acl_rt_launch_callback

函数功能：在 Stream 的任务队列中增加一个需要在板端环境上执行的回调函数。异步接口。

函数原型：

```
svp_acl_error svp_acl_rt_launch_callback(svp_acl_rt_callback fn, void *user_data, svp_acl_rt_callback_block_type blockType, svp_acl_rt_stream stream)
```

参数说明：

参数名	输入/输出	说明
fn	输入	指定要增加的回调函数。 回调函数的函数原型为： <code>typedef void (*svp_acl_rt_callback)(void *user_data)</code>
user_data	输入	待传递给回调函数的用户数据，若 fn 不需要入参，user_data 可以为空。
block_type	输入	指定回调函数调用是否阻塞 Device 运行。 <code>typedef enum svp_acl_rt_callback_block_type { SVP_ACL_CALLBACK_NO_BLOCK, //非</code>

参数名	输入/输出	说明
		阻塞 SVP_ACL_CALLBACK_BLOCK, //阻塞 } svp_acl_rt_callback_block_type;
stream	输入	指定 Stream, 如果为 NULL 则调用默认 stream。

约束说明：用户需要保证传入的 fn 和 user_data 的正确性，以及 user_data 所指向内存空间大小是否足够，否则处理回调函数时会导致程序异常。

返回值说明：返回 0 表示成功，返回其它值表示失败。

9.6.5 svp_acl_rt_process_report

函数功能：

- 在 timeout 时间内，触发回调处理，由 svp_acl_rt_subscribe_report 接口指定的线程处理回调。同步接口。
- 用户需新建一个线程，在线程函数内调用 svp_acl_rt_process_report 接口。

函数原型：

```
svp_acl_error svp_acl_rt_process_report(int32_t timeout)
```

参数说明：

参数名	输入/输出	说明
timeout	输入	超时时间，单位为 ms。 取值范围： -1：表示无限等待 - 大于等于 0：表示等待的时间

返回值说明：返回 0 表示成功，返回其它值表示失败。

9.6.6 svp_acl_rt_unsubscribe_report

函数功能：取消线程注册，Stream 上的回调函数不再由指定线程处理。同步接口。

函数原型：

```
svp_acl_error svp_acl_rt_unsubscribe_report(uint64_t thread_id, svp_acl_rt_stream stream)
```

参数说明：

参数名	输入/输出	说明
threadId	输入	指定线程的 ID。
stream	输入	指定 Stream，如果为 NULL 则调用默认 stream。

返回值说明：返回 0 表示成功，返回其它值表示失败。

9.6.7 svp_acl_rt_set_op_wait_timeout

函数功能：

- 设置等待超时时间。同步接口。
- 对于需要等待 Stream、Device 上任务完成的场景，可以调用本接口设置等待超时时间。设置超时时间后，如果等待任务完成的时间超过所设置的时间，则 SVP ACL 会返回报错。不调用本接口设置等待超时时间，则默认超时时间是 120 秒。

函数原型：

```
svp_acl_error svp_acl_rt_set_op_wait_timeout(uint32_t timeout)
```

参数说明：

参数名	输入/输出	说明
timeout	输入	设置超时时间，单位为秒。 将该参数设置为 0 时，表示不超时。 取值范围: [0, 4000000]。

返回值说明：返回 0 表示成功，返回其它值表示失败。

9.7 内存管理

9.7.1 svp_acl_rt_malloc

函数功能：

- 申请 Device 上的内存，同步接口。
- 在 Device 上申请 size 大小的线性内存，通过*dev_ptr 返回已分配内存的虚拟地址指针。
- 通过该接口申请的 Device 内存不支持 cache 缓存，如果用户需要申请支持 cache 缓存的内存，请参见 [svp_acl_rt_malloc_cached](#)。

约束说明：

- 使用 svp_acl_rt_malloc 接口申请的内存，需要通过 [svp_acl_rt_free](#) 接口释放内存。
- 频繁调用 svp_acl_rt_malloc 接口申请内存、调用 [svp_acl_rt_free](#) 接口释放内存，会损耗性能，建议用户提前做内存预先分配或二次管理，避免频繁申请/释放内存。

函数原型：

```
svp_acl_error svp_acl_rt_malloc(void **dev_ptr, size_t size, svp_acl_rt_mem_malloc_policy policy)
```

参数说明：

参数名	输入/输出	说明
size	输入	申请内存的大小，单位 Byte。 size 不能为 0。
policy	输入	内存分配规则。 <pre>typedef enum svp_acl_rt_mem_malloc_policy { SVP_ACL_MEM_MALLOC_HUGE_FIRST, // SOC不支持申请大页内存，使用该内存 分配规则，申请的是普通页的内存 SVP_ACL_MEM_MALLOC_HUGE_ONLY, // SOC不支持，使用该内存分配规则，申请 的是普通页的内存 SVP_ACL_MEM_MALLOC_NORMAL_ON</pre>

参数名	输入/输出	说明
		LY, // 仅申请普通页 } svp_acl_rt_mem_malloc_policy;
dev_ptr	输出	指向 Device 上已分配内存的指针的指针。

返回值说明：返回 0 表示成功，返回其它值表示失败。

9.7.2 svp_acl_rt_malloc_cached

函数功能：

- 申请 Device 上的内存，同步接口，该接口在任何场景下申请的内存都是支持 cache 缓存。
- 在 Device 上申请 size 大小的线性内存，通过*dev_ptr 返回已分配内存的虚拟地址指针。
- 通过本接口分配的内存支持 cache 缓存，对于频繁使用的内存，建议使用本接口分配内存，这样可以提高 cpu 的读写效率，提升系统性能。当 cpu 访问此接口申请的内存时，会将内存中的数据放在 cache 中，而硬件设备（如图像分析引擎）只能访问物理内存，不能访问 cache 中内容，对于这种 cpu 和硬件会共同操作的内存，需要调用 svp_acl_rt_mem_flush 接口做好数据同步。

约束说明：其它约束与 svp_acl_rt_malloc 接口相同，但使用本接口申请的支持 cache 缓存的内存，需要用户显式处理 cpu 与图像分析引擎之间 cache 的一致性。

函数原型：

```
svp_acl_error svp_acl_rt_malloc_cached(void **dev_ptr, size_t size,
svp_acl_rt_mem_malloc_policy policy)
```

参数说明：

参数名	输入/输出	说明
dev_ptr	输出	指向 Device 上已分配内存的指针的指针。
size	输入	申请内存的大小，单位 Byte。 size 不能为 0。

参数名	输入/输出	说明
policy	输入	内存分配规则。 <pre>typedef enum svp_acl_rt_mem_malloc_policy { SVP_ACL_MEM_MALLOC_HUGE_FIRST, // SOC不支持申请大页内存, 使用该内存分 配规则, 申请的是普通页的内存 SVP_ACL_MEM_MALLOC_HUGE_ONLY, // SOC不支持, 使用该内存分配规则, 申请的 是普通页的内存 SVP_ACL_MEM_MALLOC_NORMAL_ONL Y, // 仅申请普通页 } svp_acl_rt_mem_malloc_policy;</pre>

返回值说明：返回 0 表示成功，返回其它值表示失败。

9.7.3 svp_acl_rt_mem_flush

函数功能：

- 将 cache 中的数据刷新到 ddr 中，并将 cache 中的内容设置成无效。
- 当 cache 里的数据为最新数据时，为了保证不能直接访问 cache 的硬件（如图像分析引擎）在访问内存时能够得到正确的数据，此时需要先调用本接口将 cache 里的内容更新到内存，这样，当硬件访问内存时，保证了数据的一致性和正确性。
- 此接口应与 [svp_acl_rt_malloc_cached](#) 接口配套使用。

函数原型：

```
svp_acl_error svp_acl_rt_mem_flush(const void *dev_ptr, size_t size)
```

参数说明：

参数名	输入/输出	说明
dev_ptr	输入	要 flush 的 ddr 内存起始地址。
size	输入	要 flush 的 ddr 内存大小，单位 Byte。 size 不能为 0。

返回值说明：返回 0 表示成功，返回其它值表示失败。

9.7.4 svp_acl_rt_mem_invalidate

函数功能：

- 将 cache 中的数据设置成无效。
- 针对硬件输出的内存，如果此时 cache 中有预读引起的脏数据，硬件计算完成后，为了保证 CPU 后处理能获取到正确的数据，此时需要将 cache 中预读数据置无效。如果 CPU 对该内存执行过写操作，调用该接口会将 cache 中的数据同步到 ddr。
- 此接口应与 [svp_acl_rt_malloc_cached](#) 接口配套使用。

函数原型：

```
svp_acl_error svp_acl_rt_mem_invalidate(const void *dev_ptr, size_t size)
```

参数说明：

参数名	输入/输出	说明
devPtr	输入	需要指定 ddr 内存对应的 cache 无效。
size	输入	ddr 内存大小，单位 Byte。 size 不能为 0。

返回值说明：返回 0 表示成功，返回其它值表示失败。

9.7.5 svp_acl_rt_free

函数功能：

- 释放 Device 上的内存，同步接口。
- svp_acl_rt_free 接口只能释放通过 [svp_acl_rt_malloc](#) 接口或 [svp_acl_rt_malloc_cached](#) 接口申请的内存。如果应用在 Devcie 侧使用 [svp_acl_rt_malloc_host](#) 接口申请内存，也可以通过 svp_acl_rt_free 接口释放。

函数原型：

```
svp_acl_error svp_acl_rt_free(const void *dev_ptr)
```

参数说明：

参数名	输入/输出	说明
dev_ptr	输入	待释放内存的指针。

返回值说明：返回 0 表示成功，返回其它值表示失败。

9.7.6 `svp_acl_rt_malloc_host`

函数功能：申请 Host 或 Device 上的内存，Device 上的内存按普通页申请。同步接口。

约束说明：

- 使用 `svp_acl_rt_malloc_host` 接口申请的内存，需要通过 [svp_acl_rt_free_host](#) 接口释放内存。
- 应用在 Host 上运行时，调用该接口申请的是 Host 内存，由系统保证内存首地址 64 字节对齐。应用在 Device 上运行时，调用该接口申请的是 Device 内存，由系统保证内存首地址 64 字节对齐。
- 频繁调用 `svp_acl_rt_malloc_host` 接口申请内存、调用 [svp_acl_rt_free_host](#) 接口释放内存，会损耗性能，建议用户提前做内存预先分配或二次管理，避免频繁申请/释放内存。

函数原型：

```
svp_acl_error svp_acl_rt_malloc_host(void **host_ptr, size_t size)
```

参数说明：

参数名	输入/输出	说明
size	输入	申请内存的大小，单位 Byte。 size 不能为 0。
host_ptr	输出	“已分配内存的指针”的指针。

返回值说明：返回 0 表示成功，返回其它值表示失败。

9.7.7 `svp_acl_rt_free_host`

函数功能：

- 释放 Host 或 Device 上的内存，同步接口。
- 应用在 Host 侧，`svp_acl_rt_free_host` 接口只能释放通过 `svp_acl_rt_malloc_host` 接口申请的内存。应用在 Device 侧，`svp_acl_rt_free_host` 接口既可以释放 `svp_acl_rt_malloc_host` 接口申请的内存，也可以释放 `svp_acl_rt_malloc` 接口申请的内存。

函数原型：

```
svp_acl_error svp_acl_rt_free_host(const void *host_ptr)
```

参数说明：

参数名	输入/输出	说明
host_ptr	输入	待释放内存的指针。

返回值说明：返回 0 表示成功，返回其它值表示失败。

9.8 模型加载与执行

9.8.1 svp_acl_mdload_from_mem

函数功能：

- 从内存加载离线模型数据，加载的模型是独立的，不依赖于 device，contxt 等资源，可以被下发到不同的 stream 上执行，同步接口。
- 系统完成模型加载后，返回的模型 ID，作为后续操作时用于识别模型的标志。

约束说明：

- 最多可加载 64 个模型。
- 输入/输出所有维度乘积值不能超过 0xFFFFFFFF。
- 生成.om 文件时配置的[internal_stride]项（详细参见《ATC 工具使用指南》）必须大于 0 且为 16 的倍数（倍数须为 2 的非负整数次幂），最大不能超过 256。
- Recurrent 网络生成.om 文件时配置的[recurrent_max_total_t]项（详细参见《ATC 工具使用指南》）必须大于 0，最大不能超过 1024。
- 输入/输出默认 stride 按照生成.om 文件时配置的[internal_stride]的值对齐后不能超过 0xFFFFFFFF（详细输入/输出默认 stride 计算方法请参见

[svp_acl_md1_get_input_default_stride](#) 和 [svp_acl_md1_get_output_default_stride](#))。

- 输入/输出按照默认 stride 计算得到的单 batch 内存大小不能超过 0xFFFFFFFF (详细输入/输出内存计算方法请参见 [svp_acl_md1_get_input_size_by_index](#) 和 [svp_acl_md1_get_output_size_by_index](#))。
- Recurrent 网络 data 层个数范围：[2, 5]。
- Recurrent 网络不支持前后接模式识别/CPU 算子。
- 模型每个执行段 data 层和 report 层个数总和不能超过 32。

函数原型：

```
svp_acl_error svp_acl_md1_load_from_mem(const void* model,size_t model_size, uint32_t* model_id)
```

参数说明：

参数名	输入/输出	说明
model	输入	模型数据的内存地址。用户要保证内存中存储的模型数据的完整性和正确性，在调用 svp_acl_md1_unload 接口卸载模型之前内存不能被释放且内存中的模型数据不能被修改。
model_size	输入	内存中的模型数据长度，单位 Byte。
model_id	输出	系统完成模型加载后生成的模型 ID。

返回值说明：返回 0 表示成功，返回其它值表示失败。

9.8.2 svp_acl_md1_execute

函数功能：执行模型推理，直到返回推理结果，同步接口。

约束说明：

- input 中的 task_buf (见“[4.4.3 准备模型执行的输入/输出数据](#)”) 在任务模型推理完成前不能被改写也不能被其他任务使用。
- input 中的 work_buf (见“[4.4.3 准备模型执行的输入/输出数据](#)”) 不能被其他 stream 上的任务使用。

- 在模型推理完成之前，不能调用 `svp_acl_md5_destroy_dataset` 接口释放 input 或 output，不能调用 `svp_acl_update_data_buffer` 接口更新 input 或者 output。

函数原型：

```
svp_acl_error svp_acl_md5_execute(uint32_t model_id, const svp_acl_md5_dataset *input, const svp_acl_md5_dataset *output)
```

参数说明：

参数名	输入/输出	说明
model_id	输入	指定需要执行推理的模型的 ID。 调用 <code>svp_acl_md5_load_from_mem</code> 接口加载模型成功后，会返回模型 ID。
input	输入	模型推理的输入数据。 若用户使用 <code>svp_acl_rt_malloc</code> 或 <code>svp_acl_rt_malloc_host</code> 接口申请大块内存并自行划分、管理内存时，用户在管理内存时，模型输入数据的内存有对齐和补齐要求，首地址和 stride 需要 16 字节对齐。
output	输出	模型推理的输出数据。 若用户使用 <code>svp_acl_rt_malloc</code> 接口申请大块内存并自行划分、管理内存时，用户在管理内存时，模型输出数据的内存有对齐和补齐要求，首地址和 stride 需要 16 字节对齐。

返回值说明：返回 0 表示成功，返回其它值表示失败。

9.8.3 svp_acl_md5_execute_async

函数功能：执行模型推理，异步接口。

约束说明：

- input 中的 task_buf（见“4.4.3 准备模型执行的输入/输出数据”）在任务模型推理完成前不能被改写也不能被其他任务使用。
- input 中的 work_buf（见“4.4.3 准备模型执行的输入/输出数据”）不能被其他 stream 上的任务使用。
- 在模型推理完成之前，不能调用 `svp_acl_md5_destroy_dataset` 接口释放 input 或 output，不能调用 `svp_acl_update_data_buffer` 接口更新 input 或者 output。

函数原型：

```
svp_acl_error svp_acl_md5_execute_async(uint32_t model_id, const svp_acl_md5_dataset *input,
const svp_acl_md5_dataset *output, svp_acl_rt_stream stream)
```

参数说明：

参数名	输入/输出	说明
model_id	输入	指定需要执行推理的模型的 ID。 调用 <code>svp_acl_md5_load_from_mem</code> 接口加载模型成功后，会返回模型 ID。
input	输入	模型推理的输入数据。 若用户使用 <code>svp_acl_rt_malloc</code> 或 <code>svp_acl_rt_malloc_host</code> 接口申请大块内存并自行划分、管理内存时，用户在管理内存时，模型输入数据的内存有对齐和补齐要求，首地址和 stride 需要 16 字节对齐。
output	输出	模型推理的输出数据。 若用户使用 <code>svp_acl_rt_malloc</code> 接口申请大块内存并自行划分、管理内存时，用户在管理内存时，模型输出数据的内存有对齐和补齐要求，首地址和 stride 需要 16 字节对齐。
stream	输入	指定 stream。stream 为 NULL 则使用默认 stream。

返回值说明：返回 0 表示成功，返回其它值表示失败。

9.8.4 svp_acl_md1_unload

函数功能：系统完成模型推理后，可调用该接口卸载模型，释放资源，同步接口。

约束说明：

- 在调用 svp_acl_md1_unload 接口卸载指定模型时，需确保其它接口没有正在使用该模型。
- 不建议在析构函数中调用 svp_acl_md1_unload 接口，否则在进程退出时可能由于单例析构顺序未知而导致进程异常退出的问题。

函数原型：

```
svp_acl_error svp_acl_md1_unload(uint32_t model_id)
```

参数说明：

参数名	输入/输出	说明
model_id	输入	指定需卸载的模型的 ID。

返回值说明：返回 0 表示成功，返回其它值表示失败。

9.8.5 svp_acl_md1_set_dynamic_batch_size

函数功能：用于设置模型推理时的批量大小 Batch（每次处理图片的数量）。

约束说明：

- 配置 batch_size 时，batch_size 需大于 0，若输入为图片类型，不能超过 256；若输入为非图片数据，不能超过 5000。如果执行模型前没有调用该接口配置 batch_size 到 dataset，则将按照 dataset 中默认的 batch 值（默认值为 1）执行。
- batch_size 值约束请查看《ATC 工具使用指南》中 batch_num 参数说明。

函数原型：

```
svp_acl_error svp_acl_md1_set_dynamic_batch_size(uint32_t model_id, svp_acl_md1_dataset  
*dataset, size_t index, uint64_t batch_size)
```

参数说明：

参数名	输入/输出	说明
model_id	输入	模型 ID。 调用 svp_acl_md1_load_from_mem 接口加载模型成功后，会返回模型 ID。
dataset	输入&输出	表示模型的输入数据。 使用 svp_acl_md1_dataset 类型的数据描述模型推理时的输入数据，输入的内存地址、内存大小用 svp_acl_data_buffer 类型的数据来描述。
index	输入	标识动态 Batch 输入的输入 index，需调用 svp_acl_md1_get_input_index_by_name 接口获取，输入名称固定为 SVP_ACL_DYNAMIC_TENSOR_NAME。
batch_size	输入	指定模型推理时的批量大小 Batch。

返回值说明：返回 0 表示成功，返回其它值表示失败。

9.8.6 svp_acl_md1_set_total_t

函数功能：用于设置 Recurrent 模型推理时要处理的总帧数。

约束说明：配置的 total_t 不能为 0，也不能超过编译 om 时[recurrent_max_total_t]项配置的值（详细参见《ATC 工具使用指南》）。执行 Recurrent 网络前可以调用该接口将 total_t 配置到 dataset 中，如果没有配置，将把编译 om 时[recurrent_max_total_t]项配置的值作为 T 维度长度进行内存校验和网络执行。

函数原型：

```
svp_acl_error svp_acl_md1_set_total_t(uint32_t model_id, svp_acl_md1_dataset *dataset, uint64_t total_t)
```

参数说明：

参数名	输入/输出	说明
model_id	输入	模型 ID。 调用 svp_acl_mdl_load_from_mem 接口加载模型成功后，会返回模型 ID。
dataset	输入&输出	表示模型的输入数据。 使用 svp_acl_mdl_dataset 类型的数据描述模型推理时的输入数据，输入的内存地址、内存大小用 svp_acl_data_buffer 类型的数据来描述。
total_t	输入	指定 Recurrent 推理时的总帧数。

返回值说明：返回 0 表示成功，返回其它值表示失败。

9.8.7 svp_acl_mdl_init_dump

函数功能：

- Dump 初始化。
- [svp_acl_mdl_init_dump](#) 接口需要与 [svp_acl_mdl_set_dump](#) 接口、[svp_acl_mdl_finalize_dump](#) 接口配合使用，用于配置 Dump 信息。
- 接口的调用顺序如下，以配置两个模型的 Dump 信息为例：[svp_acl_init](#) 接口-->[svp_acl_mdl_init_dump](#) 接口-->[svp_acl_mdl_set_dump](#) 接口-->模型 1 加载-->[svp_acl_mdl_finalize_dump](#) 接口-->[svp_acl_mdl_init_dump](#) 接口-->[svp_acl_mdl_set_dump](#) 接口-->模型 2 加载-->[svp_acl_mdl_finalize_dump](#) 接口-->执行其它任务-->模型卸载-->[svp_acl_finalize](#) 接口

约束说明：

- 如果已经通过 [svp_acl_init](#) 接口配置了 dump 信息，则调用 [svp_acl_mdl_init_dump](#) 接口时会返回失败。
- 必须在调用 [svp_acl_init](#) 接口之后、模型加载接口之前调用 [svp_acl_mdl_init_dump](#) 接口。

函数原型：


```
svp_acl_error svp_acl_md1_init_dump()
```

参数说明：无

返回值说明：返回 0 表示成功，返回其它值表示失败。

9.8.8 svp_acl_md1_set_dump

函数功能：

- 设置 dump 参数。
- svp_acl_md1_set_dump 接口需要与 [svp_acl_md1_init_dump](#) 接口、[svp_acl_md1_finalize_dump](#) 接口配合使用，用于配置 Dump 信息。
- 接口的调用顺序如下，以配置两个模型的 Dump 信息为例：[svp_acl_init](#) 接口-->[svp_acl_md1_init_dump](#) 接口-->[svp_acl_md1_set_dump](#) 接口-->模型 1 加载-->[svp_acl_md1_finalize_dump](#) 接口-->[svp_acl_md1_init_dump](#) 接口-->[svp_acl_md1_set_dump](#) 接口-->模型 2 加载-->[svp_acl_md1_finalize_dump](#) 接口-->执行其它任务-->模型卸载-->[svp_acl_finalize](#) 接口

约束说明：

- 只有在调用本接口之后加载模型，配置的 Dump 信息有效。在调用本接口之前已经加载的模型不受影响，除非用户在调用本接口后重新加载该模型。

例如以下接口调用顺序中，加载的模型 1 不受影响，配置的 Dump 信息仅对加载的模型 2 有效：

[svp_acl_md1_init_dump](#) 接口-->模型 1 加载-->[svp_acl_md1_set_dump](#) 接口-->模型 2 加载-->[svp_acl_md1_finalize_dump](#) 接口

- 多次调用本接口对同一个模型配置了 Dump 信息，系统内处理时会采用覆盖策略。

例如以下接口调用顺序中，第二次调用本接口配置的 Dump 信息会覆盖第一次配置的 Dump 信息：

[svp_acl_md1_init_dump](#) 接口-->[svp_acl_md1_set_dump](#) 接口-->[svp_acl_md1_set_dump](#) 接口-->模型 1 加载-->[svp_acl_md1_finalize_dump](#) 接口

- Dump 功能开启除了配置 json 文件，OM 需要选择开启 Dump 模式的 OM，开启 Dump 模式请参考《MindStudio 用户手册》

函数原型：

```
svp_acl_error svp_acl_md1_set_dump(const char *dump_cfg_path)
```

参数说明：

参数名	输入/输出	说明
dump_cfg_path	输入	配置文件所在的路径，包含文件名。 配置文件格式为 json 格式，当前可配置 dump 数据的相关信息，示例请参见 配置文件示例 ，详细配置说明请参见《精度比对工具使用指导》中的 比对数据准备>准备离线模型 Dump 数据 ；如果不涉及配置信息，需向 <code>svp_acl_md1_set_dump</code> 接口中传入 NULL。

配置文件示例：

以 Caffe ResNet-50 网络为例，若需要比对 Caffe ResNet-50 网络与基于 Caffe ResNet-50 转换成的适配 SoC 的离线模型中某些层算子的输出结果，可以在配置文件中配置如下内容：

```
{
  "dump":{
    "dump_list":[
      {
        "model_name":"ResNet-50",
        "layer":[
          "conv1conv1_relu",
          "res2a_branch2ares2a_branch2a_relu",
          "res2a_branch1",
          "pool1"
        ]
      }
    ],
    "dump_path":"/MyApp20/dump",
    "dump_mode":"output"
  }
}
```

返回值说明：返回 0 表示成功，返回其它值表示失败。

9.8.9 svp_acl_md1_finalize_dump

函数功能：

- Dump 去初始化。

- `svp_acl_mdl_finalize_dump` 接口需要与 `svp_acl_mdl_init_dump` 接口、`svp_acl_mdl_set_dump` 接口配合使用，用于配置 Dump 信息。
- 接口的调用顺序如下，以配置两个模型的 Dump 信息为例：`svp_acl_init` 接口-->`svp_acl_mdl_init_dump` 接口-->`svp_acl_mdl_set_dump` 接口-->模型 1 加载-->`svp_acl_mdl_finalize_dump` 接口-->`svp_acl_mdl_init_dump` 接口-->`svp_acl_mdl_set_dump` 接口-->模型 2 加载-->`svp_acl_mdl_finalize_dump` 接口-->执行其它任务-->模型卸载-->`svp_acl_finalize` 接口

函数原型：

```
svp_acl_error svp_acl_mdl_finalize_dump()
```

参数说明：无

返回值说明：返回 0 表示成功，返回其它值表示失败。

9.8.10 svp_acl_mdl_set_config_opt

函数功能：

设置模型加载的配置对象中的各属性的取值，包括模型执行的优先级、模型的内存地址、内存大小等。

约束说明：

按如下接口调用顺序可实现模型加载的功能：

1. 调用 `svp_acl_mdl_create_config_handle` 接口创建模型加载的配置对象。
2. 多次调用 `svp_acl_mdl_set_config_opt` 接口设置配置对象中每个属性的值。
3. 调用 `svp_acl_mdl_load_with_config` 接口指定模型加载时需要的配置信息，并进行模型加载。
4. 模型加载成功后，调用 `svp_acl_mdl_destroy_config_handle` 接口销毁。
5. 值得注意的是通过 `svp_acl_mdl_load_with_config` 接口加载的模型同样需要调用 `svp_acl_mdl_unload` 接口进行模型卸载。

函数原型：

```
svp_acl_error svp_acl_mdl_set_config_opt(svp_acl_mdl_config_handle *handle,  
svp_acl_mdl_config_attr attr, const void *attr_value, size_t value_size)
```

参数说明：

参数名	输入/输出	说明
handle	输入	指定模型加载的配置对象。需提前调用 svp_acl_mdl_create_config_handle 接口创建该对象。
attr	输入	指定需设置的属性。
attr_value	输入	指向属性值的指针，attr 对应的属性取值。 如果属性值本身是指针，则传入该指针的地址。
value_size	输入	attr_value 部分的数据长度。 用户可使用 C/C++ 标准库的函数 <code>sizeof(*attr_value)</code> 查询数据长度。

返回值说明：返回 0 表示成功，返回其它值表示失败。

9.8.11 svp_acl_mdl_load_with_config

函数功能：

指定模型加载时需要的配置信息，并进行模型加载。

约束说明：

1. 调用 [svp_acl_mdl_create_config_handle](#) 接口创建模型加载的配置对象。
2. 多次调用 [svp_acl_mdl_set_config_opt](#) 接口设置配置对象中每个属性的值。
3. 调用 [svp_acl_mdl_load_with_config](#) 接口指定模型加载时需要的配置信息，并进行模型加载。
4. 模型加载成功后，调用 [svp_acl_mdl_destroy_config_handle](#) 接口销毁。

函数原型：

```
svp_acl_error svp_acl_mdl_load_with_config(const svp\_acl\_mdl\_config\_handle *handle, uint32_t *model_id)
```

参数说明：

参数名	输入/输出	说明
handle	输入	指定模型加载的配置对象。需提前调用 svp_acl_mdl_create_config_handle 接口创建该对象，与 svp_acl_mdl_set_config_opt 中的 handle 保持一致。
model_id	输出	系统完成模型加载后生成的模型 ID。

返回值说明：返回 0 表示成功，返回其它值表示失败。

9.8.12 svp_acl_mdl_get_first_aapp_info

函数功能：

获取模型 AAPP 的配置信息。

约束说明：

使用本接口获取模型中 AAPP 的信息，只能获取 [svp_acl_aapp_info](#) 结构体中 inputFormat 的值，其他的值无法获取。

函数原型：

```
svp_acl_error svp_acl_mdl_get_first_aapp_info(uint32_t model_id, size_t index, svp_acl_aapp_info *aapp_info)
```

参数说明：

参数名	输入/输出	说明
model_id	输入	系统完成模型加载后生成的模型 ID。
index	输入	模型中输入的 index。不支持获取 task_buf 与 work_buf 的 aapp 配置信息。
aapp_info	输出	获取指定输入上 AAPP 的配置信息。当前只能获取输入的 format 类型。

返回值说明：返回 0 表示成功，返回其它值表示失败。

9.8.13 svp_acl_md1_get_dynamic_batch

函数功能：
根据模型描述信息获取模型生成时配置的 Batch 信息。同步接口。

函数原型：

```
svp_acl_error svp_acl_md1_get_dynamic_batch(const svp_acl_md1_desc *model_desc,
svp_acl_md1_batch *batch);
```

参数说明：

参数名	输入/输出	说明
model_desc	输入	svp_acl_md1_desc 类型数据的指针。
batch	输出	#define SVP_ACL_MAX_BATCH_NUM 1 typedef struct svp_acl_md1_batch { size_t batch_count; /* batch array count, only support 1 */ uint64_t batch[SVP_ACL_MAX_BATCH_NUM]; /* batch data array */ } svp_acl_md1_batch; 当前只支持获取生成模型时配置的 Batch 信息。Batch 值约束请查看《ATC 工具使用指南》中 batch_num 参 数说明。

返回值说明：返回 0 表示成功，返回其它值表示失败。

9.9 Profiling 配置

9.9.1 功能及约束说明

功能：
该章节下的接口用于 Profiling 采集性能数据，实现方式支持以下两种：

- 方式一：将采集到的 Profiling 数据写入文件，再使用 Profiling 工具解析该文件，并展示性能分析数据
包括以下两种接口调用方式：

- `svp_acl_prof_init` 接口、`svp_acl_prof_start` 接口、`svp_acl_prof_stop` 接口、`svp_acl_prof_finalize` 接口配合使用，实现该方式的性能数据采集。该方式可获取 SVP ACL 的接口性能数据、*模式识别* Core 上算子的执行时间、*模式识别* CPU 算子的执行时间等。目前这些接口为进程级控制，表示在进程内任意线程调用该接口，其它线程都会生效。

一个进程内，可以根据需求多次调用这些接口，基于不同的 Profiling 采集配置，采集数据。

- 调用 `svp_acl_init` 接口，在 SVP ACL 初始化阶段，通过*.json 文件传入要采集的 Profiling 数据。该方式可获取 SVP ACL 的接口性能数据、*模式识别* Core 上算子的执行时间、*模式识别* CPU 算子的执行时间等。

一个进程内，只能调用一次 `svp_acl_init` 接口，如果要修改 Profiling 采集配置，需修改*.json 文件中的配置。详细使用说明请参见接口处的说明，不在本章节描述。

- 方式二：将采集到的 Profiling 数据解析后写入管道，由用户读入内存，再由用户调用 SVP ACL 的接口获取性能数据

`svp_acl_prof_model_subscribe` 接口、`svp_acl_prof_get*`接口、`svp_acl_prof_model_unsubscribe` 接口配合使用，实现该方式的性能数据采集，当前支持获取网络模型中*模式识别* Core 算子的性能数据，包括算子名称、算子类型名称、算子执行时间等。

总体约束：两种方式的 Profiling 性能数据采集接口不能交叉调用。

- `svp_acl_prof_init` 接口和 `svp_acl_prof_finalize` 接口之间不能调用 `svp_acl_prof_model_subscribe` 接口、`svp_acl_prof_get*`接口、`svp_acl_prof_model_unsubscribe` 接口；
- `svp_acl_prof_model_subscribe` 接口和 `svp_acl_prof_model_unsubscribe` 接口之间不能调用 `svp_acl_prof_init` 接口、`svp_acl_prof_start` 接口、`svp_acl_prof_stop` 接口、`svp_acl_prof_finalize`。
- 通过方式二实现的 Profiling 采集暂不能很好的支持 Recurrent、RPN、ROI、CPU Loop 的网络，这些网络建议通过 MindStudio 工具采集并解析 Profiling 数据。
- 开启 profiling 进行数据采集时，由于第一帧推理需要配置 profiling 的使能/内存，并且第一帧推理时硬件会额外处理页表等动作，所以第一帧推理采集的数据会偏高。

方式一的接口约束说明

- **调用接口要求：**

- `svp_acl_prof_init` 接口必须在 `svp_acl_init` 接口之后、模型加载之前调用。

如果已经通过 `svp_acl_init` 接口配置了 Profiling 信息，则调用

`svp_acl_prof_init` 接口、`svp_acl_prof_start` 接口、`svp_acl_prof_stop` 接口、`svp_acl_prof_finalize` 时，会返回报错。

如果没有调用 `svp_acl_prof_init` 接口，调用 `svp_acl_prof_start` 接口、`svp_acl_prof_stop` 接口、`svp_acl_prof_finalize` 时，会返回报错。

- `svp_acl_prof_start` 接口在模型执行之前调用，若在模型执行过程中调用 `svp_acl_prof_start` 接口，Profiling 采集到的数据为调用 `svp_acl_prof_start` 接口之后的数据，可能导致数据不完整。

调用 `svp_acl_prof_start` 接口时，可以指定从一个 Device 上采集性能数据，也可以指定从多个 Device 上采集性能数据。

一个用户 APP 进程内，如果连续调用多次 `svp_acl_prof_start` 接口，指定重复的 Profiling 配置，或指定的 Device 重复，会返回报错。

- 在用户 APP 的进程生命周期内，`svp_acl_prof_init` 接口与 `svp_acl_prof_finalize` 接口配对使用，建议只调用一次，如该组合多次调用可以改变保存性能数据的文件的路径。
- `svp_acl_prof_start` 接口与 `svp_acl_prof_stop` 接口需配对使用。

- **接口调用顺序：**

- **建议的接口调用顺序如下**，以“一个用户 APP 进程内采集多个模型推理时的性能数据”为例：

`svp_acl_init` 接口-->`svp_acl_prof_init` 接口-->`svp_acl_prof_start` 接口-->模型 1 加载-->模型 1 执行-->模型 2 加载-->模型 2 执行-->模型卸载--

>`svp_acl_prof_stop` 接口(与 `svp_acl_prof_start` 接口的 `svp_acl_prof_config` 数据保持一致)-->`svp_acl_prof_finalize` 接口-->执行其它任务-->`svp_acl_finalize` 接口

- **错误的接口调用顺序示例如下**，以“一个用户 APP 进程内，如果连续调用多次 `svp_acl_prof_start` 接口，指定的 Device 重复”为例：

`svp_acl_init` 接口-->`svp_acl_prof_init` 接口-->`svp_acl_prof_start` 接口(指定 Device 0)-->`svp_acl_prof_start` 接口(指定 Device 0)-->模型 1 加载-->模型 1 执行-->模型 2 加载-->模型 2 执行-->`svp_acl_prof_stop` 接口--

>`svp_acl_prof_stop` 接口-->`svp_acl_prof_finalize`-->执行其它任务-->模型卸载-->`svp_acl_finalize` 接口

方式二的接口约束说明

- 接口调用要求：
 - `svp_acl_prof_model_subscribe` 接口在模型执行之前调用，若在模型执行过程中调用 `svp_acl_prof_model_subscribe` 接口，Profiling 采集到的数据为调用 `svp_acl_prof_model_subscribe` 接口之后的数据，可能导致数据不完整。
 - `svp_acl_prof_model_subscribe` 接口需与 `svp_acl_prof_model_unsubscribe` 接口配对使用，不能在调用接口前，多次调用 `svp_acl_prof_model_subscribe` 接口重复订阅相同的模型。
 - 不能调用 `svp_acl_prof_model_subscribe` 接口订阅不存在的模型 ID。
 - 不能调用 `svp_acl_prof_model_unsubscribe` 接口取消订阅不存在的模型 ID 或未订阅过的模型 ID。
 - 如果在同一个 Device 上加载了多个模型，只能对多个模型下发同样的订阅配置。
- 接口调用顺序：
 - 建议的接口调用顺序如下：
模型加载-->`svp_acl_prof_model_subscribe` 接口--
>`svp_acl_prof_get_op_desc_size` 接口-->`svp_acl_prof_get_op_num` 接口--
>`svp_acl_prof_get_op_type`/`svp_acl_prof_get_op_name`/`svp_acl_prof_get_op_start`/`svp_acl_prof_get_op_end`/`svp_acl_prof_get_op_duration`/`svp_acl_prof_get_model_id` 接口-->`svp_acl_prof_model_unsubscribe` 接口
 - 错误的接口调用顺序示例如下，以重复定义同一个模型为例：
模型 1 加载-->`svp_acl_prof_model_subscribe` 接口(指定模型 1)--
>`svp_acl_prof_model_subscribe` 接口(指定模型 1)--
>`svp_acl_prof_model_unsubscribe` 接口

9.9.2 svp_acl_prof_init

函数功能：初始化 Profiling，目前用于设置保存性能数据的文件的路径。同步接口。

函数原型：

```
svp_acl_error svp_acl_prof_init(const char *result_path, size_t length)
```

参数说明：

参数名	输入/输出	说明
result_path	输入	指定保存性能数据的文件的路径，此路径为绝对路径。

参数名	输入/输出	说明
length	输入	result_path 的长度, 单位为 Byte, 最大长度不超过 4096 字节。

返回值说明: 返回 0 表示成功, 返回其它值表示失败。

参考资源: 接口调用示例, 参见“[8.6 Profiling 性能数据采集](#)”。

9.9.3 svp_acl_prof_start

函数功能:

- 下发 Profiling 请求, 使能对应数据的采集。同步接口。
- 用户可根据需要, 在模型执行过程中按需调用 svp_acl_prof_start 接口, Profiling 采集到的数据为调用该接口之后的数据, 且只会获取一次模型 Profiling 数据。

函数原型:

```
svp_acl_error svp_acl_prof_start (const svp_acl_prof_config *profiler_config)
```

参数说明:

参数名	输入/输出	说明
profiler_config	输入	指定 Profiling 配置数据。 需提前调用 svp_acl_prof_create_config 接口创建 svp_acl_prof_config 类型的数据。

返回值说明: 返回 0 表示成功, 返回其它值表示失败。

参考资源: 接口调用示例, 参见“[8.6 Profiling 性能数据采集](#)”。

9.9.4 svp_acl_prof_stop

函数功能: 停止 Profiling 数据采集, 同步接口。

函数原型:

```
svp_acl_error svp_acl_prof_stop(const svp_acl_prof_config *profiler_config)
```

参数说明:

参数名	输入/ 输出	说明
profiler_config	输入	指定停止 Profiling 数据采集的配置。 与 svp_acl_prof_start 接口中的 svp_acl_prof_config 类型数据保持一致。

返回值说明：返回 0 表示成功，返回其它值表示失败。

参考资源：接口调用示例，参见[“8.6 Profiling 性能数据采集”](#)。

9.9.5 svp_acl_prof_finalize

函数功能：结束 Profiling。同步接口。

函数原型：

```
svp_acl_error svp_acl_prof_finalize()
```

参数说明：无

返回值说明：返回 0 表示成功，返回其它值表示失败。

参考资源：接口调用示例，参见[“8.6 Profiling 性能数据采集”](#)。

9.9.6 svp_acl_prof_model_subscribe

函数功能：网络场景下，订阅算子的基本信息，包括算子名称、算子类型、算子执行耗时等。同步接口。需要与 [svp_acl_prof_model_unsubscribe](#) 接口配对使用。

函数原型：

```
svp_acl_error svp_acl_prof_model_subscribe(uint32_t model_id, const  
svp_acl_prof_subscribe_config *prof_subscribe_config)
```

参数说明：

参数名	输入/ 输出	说明
model_id	输入	待订阅的网络模型的 ID。 调用 svp_acl_mdload_from_mem 接口加载模型成功后，会返回模型 ID。

参数名	输入/输出	说明
prof_subscribe_config	输入	待订阅的配置信息。 需提前调用 svp_acl_prof_create_subscribe_config 接口创建 svp_acl_prof_subscribe_config 类型的数据。

返回值说明：返回 0 表示成功，返回其它值表示失败。

参考资源：接口调用示例，参见[“8.6 Profiling 性能数据采集”](#)。

9.9.7 svp_acl_prof_model_unsubscribe

函数功能：网络场景下，取消订阅算子的基本信息，包括算子名称、算子类型、算子执行耗时等。同步接口。需要与 [svp_acl_prof_model_subscribe](#) 接口配对使用。

函数原型：

```
svp_acl_error svp_acl_prof_model_unsubscribe(uint32_t model_id)
```

参数说明：

参数名	输入/输出	说明
model_id	输入	已订阅的模型的 ID。

返回值说明：返回 0 表示成功，返回其它值表示失败。

参考资源：接口调用示例，参见[“8.6 Profiling 性能数据采集”](#)。

9.9.8 svp_acl_prof_get_op_desc_size

函数功能：

- 获取单个算子数据结构的大小，单位为 Byte。当前版本中约定每个算子数据结构的大小是一样的。同步接口。
- 建议用户新建一个线程，在新线程内调用该接口，否则可能阻塞主线程中的其它任务调度。

函数原型：

```
svp_acl_error svp_acl_prof_get_op_desc_size(size_t *op_desc_size)
```

参数说明：

参数名	输入/ 输出	说明
op_desc_size	输出	算子数据结构的大小。

返回值说明：返回 0 表示成功，返回其它值表示失败。

参考资源：接口调用示例，参见“[8.6 Profiling 性能数据采集](#)”。

9.9.9 svp_acl_prof_get_op_num

函数功能：

- 获取指定内存中算子的数量。同步接口。
- 建议用户新建一个线程，在新线程内调用该接口，否则可能阻塞主线程中的其它任务调度。

函数原型：

```
svp_acl_error svp_acl_prof_get_op_num(const void *op_info, size_t op_info_len, uint32_t  
*op_number)
```

参数说明：

参数名	输入/ 输出	说明
op_info	输入	指定算子信息的内存地址。 调用 svp_acl_prof_get_op_desc_size 接口获取到单个算子数据结构的大小后，用户需按照“单个算子数据结构的大小*整数系数”得到的数值申请内存，用于存放 Profiling 采集到的算子信息数据，作为本接口的输入。
op_info_len	输入	算子信息的长度。
op_number	输出	算子的数量。

返回值说明：返回 0 表示成功，返回其它值表示失败。

参考资源：接口调用示例，参见“[8.6 Profiling 性能数据采集](#)”。

9.9.10 svp_acl_prof_get_op_type_len

函数功能：

- 获取算子类型的字符串长度，用于内存申请。同步接口。
- 建议用户新建一个线程，在新线程内调用该接口，否则可能阻塞主线程中的其它任务调度。

函数原型：

```
svp_acl_error svp_acl_prof_get_op_type_len(const void *op_info, size_t op_info_len, uint32_t index, size_t *op_type_len)
```

参数说明：

参数名	输入/输出	说明
op_info	输入	包含算子信息的地址。
op_info_len	输入	算子信息的长度。
index	输入	指定获取第几个算子的算子类型名称。 用户调用 svp_acl_prof_get_op_num 接口获取算子数量后，这个 index 的取值范围：[0, (算子数量-1)]
op_type_len	输出	op_type 的长度。

返回值说明：返回 0 表示成功，返回其它值表示失败。

参考资源：接口调用示例，参见“[8.6 Profiling 性能数据采集](#)”。

9.9.11 svp_acl_prof_get_op_type

函数功能：

- 获取指定算子的算子类型名称，同步接口。
- 建议用户新建一个线程，在新线程内调用该接口，否则可能阻塞主线程中的其它任务调度。

函数原型：

```
svp_acl_error svp_acl_prof_get_op_type(const void *op_info, size_t op_info_len, uint32_t index, char *op_type, size_t op_type_len)
```

参数说明：

参数名	输入/输出	说明
op_info	输入	包含算子信息的地址。
op_info_len	输入	算子信息的长度。
index	输入	指定获取第几个算子的算子类型名称。 用户调用 svp_acl_prof_get_op_num 接口获取算子数量后，这个 index 的取值范围：[0, (算子数量-1)]
op_type	输出	算子类型名称。
op_type_len	输入	op_type 的实际内存申请长度。取值范围建议不小于 svp_acl_prof_get_op_type_len ，否则内容会有截断。

返回值说明：返回 0 表示成功，返回其它值表示失败。

参考资源：接口调用示例，参见[“8.6 Profiling 性能数据采集”](#)。

9.9.12 svp_acl_prof_get_op_name_len

函数功能：

- 获取算子名称的字符串长度，用于内存申请。同步接口。
- 建议用户新建一个线程，在新线程内调用该接口，否则可能阻塞主线程中的其它任务调度。

函数原型：

```
svp_acl_error svp_acl_prof_get_op_name_len(const void *op_info, size_t op_info_len, uint32_t index, size_t *op_name_len)
```

参数说明：

参数名	输入/输出	说明
op_info	输入	包含算子信息的地址。
op_info_len	输入	算子信息的长度。

参数名	输入/ 输出	说明
index	输入	指定获取第几个算子的算子名称长度。 用户调用 svp_acl_prof_get_op_num 接口获取算子数量后，这个 index 的取值范围：[0, (算子数量-1)]
op_name_len	输出	op_name 的实际内存申请长度。

返回值说明：返回 0 表示成功，返回其它值表示失败。

参考资源：接口调用示例，参见“[8.6 Profiling 性能数据采集](#)”。

9.9.13 svp_acl_prof_get_op_name

函数功能：

- 获取指定算子的算子名称。同步接口。
- 建议用户新建一个线程，在新线程内调用该接口，否则可能阻塞主线程中的其它任务调度。

函数原型：

```
svp_acl_error svp_acl_prof_get_op_name(const void *op_info, size_t op_info_len, uint32_t index, char *op_name, size_t op_name_len)
```

参数说明：

参数名	输入/ 输出	说明
op_info	输入	包含算子信息的地址。
op_info_len	输入	算子信息的长度。
index	输入	指定获取第几个算子的算子名称。 用户调用 svp_acl_prof_get_op_num 接口获取算子数量后，这个 index 的取值范围：[0, (算子数量-1)]
op_name	输出	算子名称。
op_name_len	输入	op_name 的实际内存申请长度。取值范围建议不小于 svp_acl_prof_get_op_name_len 。

返回值说明：返回 0 表示成功，返回其它值表示失败。

参考资源：接口调用示例，参见[“8.6 Profiling 性能数据采集”](#)。

9.9.14 svp_acl_prof_get_op_start

函数功能：

- 获取算子执行的开始时间，单位为 ns。同步接口。
- 建议用户新建一个线程，在新线程内调用该接口，否则可能阻塞主线程中的其它任务调度。

函数原型：

```
uint64_t svp_acl_prof_get_op_start(const void *op_info, size_t op_info_len, uint32_t index)
```

参数说明：

参数名	输入/输出	说明
op_info	输入	包含算子信息的地址。
op_info_len	输入	算子信息的长度。
index	输入	指定获取第几个算子执行的开始时间。 用户调用 svp_acl_prof_get_op_num 接口获取算子数量后，这个 index 的取值范围：[0, (算子数量-1)]

返回值说明：算子执行的开始时间。

参考资源：接口调用示例，参见[“8.6 Profiling 性能数据采集”](#)。

9.9.15 svp_acl_prof_get_op_end

函数功能：

- 获取算子执行的结束时间，单位为 ns。同步接口。
- 建议用户新建一个线程，在新线程内调用该接口，否则可能阻塞主线程中的其它任务调度。

函数原型：

```
uint64_t svp_acl_prof_get_op_end(const void *op_info, size_t op_info_len, uint32_t index)
```

参数说明：

参数名	输入/ 输出	说明
op_info	输入	包含算子信息的地址。
op_info_len	输入	算子信息的长度。
index	输入	指定获取第几个算子执行的结束时间。 用户调用 svp_acl_prof_get_op_num 接口获取算子数量后，这个 index 的取值范围：[0, (算子数量-1)]

返回值说明：算子执行结束时间。

参考资源：接口调用示例，参见[“8.6 Profiling 性能数据采集”](#)。

9.9.16 svp_acl_prof_get_op_duration

函数功能：

- 获取算子执行的耗时时间，单位为 ns。同步接口。
- 建议用户新建一个线程，在新线程内调用该接口，否则可能阻塞主线程中的其它任务调度。

函数原型：

```
uint64_t svp_acl_prof_get_op_duration(const void *op_info, size_t op_info_len, uint32_t index)
```

参数说明：

参数名	输入/ 输出	说明
op_info	输入	包含算子信息的地址。
op_info_len	输入	算子信息的长度。
index	输入	指定获取第几个算子执行的耗时时间。 用户调用 svp_acl_prof_get_op_num 接口获取算子数量后，这个 index 的取值范围：[0, (算子数量-1)]

返回值说明：算子执行的耗时时间。

参考资源：接口调用示例，参见[“8.6 Profiling 性能数据采集”](#)。

9.9.17 svp_acl_prof_get_model_id

函数功能：

- 获取指定算子所在模型的 ID。同步接口。
- 建议用户新建一个线程，在新线程内调用该接口，否则可能阻塞主线程中的其它任务调度。

函数原型：

```
size_t svp_acl_prof_get_model_id(const void *op_info, size_t op_info_len, uint32_t index)
```

参数说明：

参数名	输入/ 输出	说明
op_info	输入	包含算子信息的地址。
op_info_len	输入	算子信息的长度。
index	输入	指定获取第几个算子所在模型的 ID。 用户调用 svp_acl_prof_get_op_num 接口获取算子数量后，这个 index 的取值范围：[0, (算子数量-1)]

返回值说明：模型的 ID。

参考资源：接口调用示例，参见[“8.6 Profiling 性能数据采集”](#)。

9.10 获取数据大小

9.10.1 svp_acl_data_type_size

函数功能：获取 svp_acl_data_type 数据的大小，单位 bit，同步接口。

函数原型：

```
size_t svp_acl_data_type_size(svp_acl_data_type data_type)
```

参数说明：

参数名	输入/ 输出	说明
data_type	输入	指定要获取大小的 <code>svp_acl_data_type</code> 数据。

返回值说明：返回 `svp_acl_data_type` 数据的大小，单位 bit。

9.11 SoC 扩展接口

9.11.1 `svp_acl_ext_process_aacpu_task`

函数功能：

- 获取模式识别 CPU 任务并执行。同步接口。
- 用户需新建一个线程，在线程函数内调用 `svp_acl_ext_process_aacpu_task` 接口。

约束说明：如果当前进程没有创建任何 stream，会返回错误。

函数原型：

```
svp_acl_error svp_acl_ext_process_aacpu_task(int32_t timeout)
```

参数说明：

参数名	输入/输出	说明
timeout	输入	超时时间，单位为 ms。 取值范围： • -1：表示无限等待 • 大于等于 0：表示等待的时间

返回值说明：返回 0 表示成功，返回其它值表示失败。

9.11.2 `svp_acl_ext_get_mdl_aacpu_task_num`

函数功能：获取模型中模式识别 CPU 任务个数。同步接口。

函数原型：

```
svp_acl_error svp_acl_ext_get_md1_aacpu_task_num(uint32_t model_id, uint32_t *num)
```

参数说明：

参数名	输入/输出	说明
model_id	输入	指定需要执行推理的模型的 ID。 调用 svp_acl_md1_load_from_mem 接口加载模型成功后，会返回模型 ID。
num	输出	获取对应模型所含有 <i>模式识别</i> CPU 任务个数。

返回值说明：返回 0 表示成功，返回其它值表示失败。

9.11.3 svp_acl_ext_get_md1_net_type

函数功能：获取模型中的网络类型

函数原型：

```
svp_acl_error svp_acl_ext_get_md1_net_type(uint32_t model_id, uint32_t *net_type)
```

参数说明：

参数名	输入/输出	说明
model_id	输入	指定需要执行推理的模型的 ID。 调用 svp_acl_md1_load_from_mem 接口加载模型成功后，会返回模型 ID。
net_type	输出	获取对应模型中的网络类型，当前模型中有 RECURRENT 网络会返回 SVP_ACL_NET_TYPE_RECURRENT 类型，其余的模型返回 SVP_ACL_NET_TYPE_CNN 类型。 <pre>#define SVP_ACL_NET_TYPE_CNN 0x1 #define SVP_ACL_NET_TYPE_RECURRENT (0x1 << 1)</pre>

返回值说明：返回 0 表示成功，返回其它值表示失败。

9.12 数据类型及其操作接口

9.12.1 `svp_acl_error`

```
typedef int svp_acl_error;
```

表9-1 返回码列表

返回码	含义	可能原因及解决方法
<code>#define SVP_ACL_SUCCESS 0</code>	执行成功	-
<code>#define SVP_ACL_ERROR_NONE 0</code>	执行成功。	-
<code>#define SVP_ACL_ERROR_INVALID_PARAM 100000</code>	参数校验失败。	请检查接口的入参值是否正确。
<code>#define SVP_ACL_ERROR_UNINITIALIZE 100001</code>	未初始化。	请检查是否已调用 svp_acl_init 接口进行初始化，请确保已调用接口，且在其它 SVP ACL 接口之前调用。
<code>#define SVP_ACL_ERROR_REPEAT_INITIALIZE 100002</code>	重复初始化或重复加载。	请检查是否调用对应的接口重复初始化或重复加载。
<code>#define SVP_ACL_ERROR_INVALID_FILE 100003</code>	无效的文件。	请检查文件是否存在、文件是否能被访问等。
<code>#define SVP_ACL_ERROR_WRITE_FILE 100004</code>	写文件失败。	请检查文件路径是否存在、文件是否有写权限等。
<code>#define SVP_ACL_ERROR_INVALID_FILE_SIZE 100005</code>	无效的文件大小。	请检查文件大小是否符合接口要求。
<code>#define SVP_ACL_ERROR_PARSE_FILE 100006</code>	解析文件失败。	请检查文件内容是否合法。

返回码	含义	可能原因及解决方法
#define SVP_ACL_ERROR_FILE_MISSING_ATTR 100007	文件缺失参数。	请检查文件内容是否完整。
#define SVP_ACL_ERROR_FILE_ATTR_INVALID 100008	文件参数无效。	请检查文件中参数值是否正确。
#define SVP_ACL_ERROR_INVALID_DUMP_CONFIG 100009	无效的 Dump 配置。	请检查 Dump 配置是否正确，详细配置请参见《MindStudio 用户手册》。
#define SVP_ACL_ERROR_INVALID_PROFILING_CONFIG 100010	无效的 Profiling 配置。	请检查 Profiling 配置是否正确。
#define SVP_ACL_ERROR_INVALID_MODEL_ID 100011	无效的模型 ID。	请检查模型 ID 是否正确、模型是否正确加载。
#define SVP_ACL_ERROR_DESERIALIZE_MODEL 100012	反序列化模型失败。	模型可能与当前版本不匹配，请参见《ATC 工具使用指南》重新转换模型。
#define SVP_ACL_ERROR_PARSE_MODEL 100013	解析模型失败。	模型可能与当前版本不匹配，请参见《ATC 工具使用指南》重新转换模型。
#define SVP_ACL_ERROR_READ_MODEL_FAILURE 100014	读取模型失败。	请检查模型文件是否存在、模型文件是否能被访问等。
#define SVP_ACL_ERROR_MODEL_SIZE_INVALID 100015	无效的模型大小。	模型文件无效，请参见《ATC 工具使用指南》重新转换模型。
#define SVP_ACL_ERROR_MODEL_MISSING_ATTR 100016	模型缺少参数。	模型可能与当前版本不匹配，请参见《ATC 工具使用指南》重新转换模型。
#define SVP_ACL_ERROR_MODEL_INPUT_NOT_MATCH 100017	模型的输入不匹配。	请检查模型的输入是否正确。

返回码	含义	可能原因及解决方法
#define SVP_ACL_ERROR_MODEL_ OUTPUT_NOT_MATCH 100018	模型的输出不匹配。	请检查模型的输出是否正确。
#define SVP_ACL_ERROR_MODEL_ NOT_DYNAMIC 100019	非动态模型。	请检查当前模型是否支持动态场景，如不支持，请参见《ATC 工具使用指南》重新转换模型。
#define SVP_ACL_ERROR_OP_TYP E_NOT_MATCH 100020	单算子类型不匹配。	请检查算子类型是否正确。
#define SVP_ACL_ERROR_OP_INP UT_NOT_MATCH 100021	单算子的输入不匹 配。	请检查算子的输入是否正确。
#define SVP_ACL_ERROR_OP_OUT PUT_NOT_MATCH 100022	单算子的输出不匹 配。	请检查算子的输出是否正确。
#define SVP_ACL_ERROR_OP_ATT R_NOT_MATCH 100023	单算子的属性不匹 配。	请检查算子的属性是否正确。
#define SVP_ACL_ERROR_OP_NOT _FOUND 100024	单算子未找到。	请检查算子类型是否支持。
#define SVP_ACL_ERROR_OP_LOA D_FAILED 100025	单算子加载失败。	模型可能与当前版本不匹 配，请参见《ATC 工具使 用指南》重新转换模型。
#define SVP_ACL_ERROR_UNSUPP ORTED_DATA_TYPE 100026	不支持的数据类型。	请检查数据类型是否存在 或当前是否支持。
#define SVP_ACL_ERROR_FORMAT _NOT_MATCH 100027	Format 不匹配。	请检查 Format 是否正 确。
#define SVP_ACL_ERROR_BIN_SEL ECTOR_NOT_REGISTERED 100028	使用二进制选择方式 编译算子接口时，算 子未注册选择器。	请检查是否注册算子选择 器。
#define SVP_ACL_ERROR_KERNEL _NOT_FOUND 100029	编译算子时，算子 Kernel 未注册。	请检查是否注册算子 Kernel。

返回码	含义	可能原因及解决方法
#define SVP_ACL_ERROR_BIN_SELECTOR_ALREADY_REGISTERED 100030	使用二进制选择方式编译算子接口时，算子重复注册。	请检查是否重复注册算子选择器。
#define SVP_ACL_ERROR_KERNEL_ALREADY_REGISTERED 100031	编译算子时，算子 Kernel 重复注册。	请检查是否重复注册算子 Kernel。
#define SVP_ACL_ERROR_INVALID_QUEUE_ID 100032	无效的队列 ID。	请检查队列 ID 是否正确。
#define SVP_ACL_ERROR_REPEAT_SUBSCRIBE 100033	重复订阅。	请检查针对同一个 Stream，是否重复调用 svp_acl_rt_unsubscribe_report 接口。
#define SVP_ACL_ERROR_STREAM_NOT_SUBSCRIBE 100034	Stream 未订阅。	请检查是否已调用 svp_acl_rt_subscribe_report 接口。
#define SVP_ACL_ERROR_THREAD_NOT_SUBSCRIBE 100035	线程未订阅。	请检查是否已调用 svp_acl_rt_subscribe_report 接口。
#define SVP_ACL_ERROR_WAIT_CALLBACK_TIMEOUT 100036	等待 callback 超时。	请检查是否已调用 svp_acl_rt_launch_callback 接口下发 callback 任务； 请检查 svp_acl_rt_process_report 接口中超时时间是否合理； 请检查 callback 任务是否已经处理完成，如果已处理完成，但还调用 svp_acl_rt_process_report 接口，则需优化代码逻辑。
#define SVP_ACL_ERROR_REPEAT	重复去初始化。	请检查是否重复调用

返回码	含义	可能原因及解决方法
<code>_FINALIZE 100037</code>		svp_acl_finalize 接口进行去初始化。
<code>#define SVP_ACL_ERROR_NOT_STATIC_AAPP 100038</code>	静态 AAPP 配置信息不存在。	请传入正确的 index 值。
<code>#define SVP_ACL_ERROR_COMPILE_STUB_MODE 100039</code>	运行应用前配置的动态库路径是编译桩的路径，不是正确的动态库路径。	请检查动态库路径的配置，确保使用运行模式的动态库。
<code>#define SVP_ACL_ERROR_GROUP_NOT_SET 100040</code>	未设置 Group。	请检查是否已调用配置 Group 接口。
<code>#define SVP_ACL_ERROR_GROUP_NOT_CREATE 100041</code>	未创建对应的 Group。	请检查调用接口时设置的 Group ID 是否在支持的范围内，Group ID 的取值范围：[0, (Group 数量-1)]
<code>#define SVP_ACL_ERROR_PROF_ALREADY_RUN 100042</code>	已存在采集 Profiling 数据的任务。	- 请检查在调用 <code>prof</code> 接口配置 Profiling 性能数据采集信息前，是否已调用 svp_acl_init 接口配置 Profiling 信息，如是，请调整代码逻辑，保留一种方式配置 Profiling 信息即可。 - 请检查是否对同一个 Device 重复下发了多次 Profiling 配置。
<code>#define SVP_ACL_ERROR_PROF_NOT_RUN 100043</code>	未进行 Profiling 初始化。	请检查接口调用顺序。
<code>#define SVP_ACL_ERROR_DUMP_ALREADY_RUN 100044</code>	已存在获取 Dump 数据的任务。	请检查在调用 Dump 接口配置 Dump 信息前，是否已调用 svp_acl_init 接口配置 Dump 信息，如是，请调整代码逻辑，保留一

返回码	含义	可能原因及解决方法
		种方式配置 Dump 信息即可。
#define SVP_ACL_ERROR_DUMP_NOT_RUN 100045	未先进行 Dump 初始化。	请检查获取 Dump 数据的接口调用顺序。
#define SVP_ACL_ERROR_PROF_REPEAT_SUBSCRIBE 148046	重复订阅同一个模型。	请检查接口调用顺序。
#define SVP_ACL_ERROR_PROF_API_CONFLICT 148047	采集性能数据的接口调用冲突。	两种方式的 Profiling 性能数据采集接口不能交叉调用。
#define SVP_ACL_ERROR_INVALID_MAX_OPQUEUE_NUM_CONFIG 148048	无效的算子缓存信息老化配置。	请检查算子缓存信息老化配置，参考 svp_acl_init 处的配置说明及示例。
#define SVP_ACL_ERROR_BAD_ALLOC 200000	申请资源失败。	请检查硬件环境上的内存剩余情况或者 Stream、Context 等资源剩余情况。
#define SVP_ACL_ERROR_API_NOT_SUPPORT 200001	接口不支持。	请检查调用的接口当前是否支持。
#define SVP_ACL_ERROR_INVALID_DEVICE 200002	无效的 Device。	请检查 Device 是否存在。
#define SVP_ACL_ERROR_MEMORY_ADDRESS_UNALIGNED 200003	内存地址未对齐。	请检查内存地址是否符合接口要求。
#define SVP_ACL_ERROR_RESOURCE_NOT_MATCH 200004	资源不匹配。	请检查调用接口时，是否传入正确的 Stream、Context 等资源。
#define SVP_ACL_ERROR_INVALID_RESOURCE_HANDLE 200005	无效的资源句柄。	请检查调用接口时，传入的 Stream、Context 等资源是否已被销毁或占用。

返回码	含义	可能原因及解决方法
#define SVP_ACL_ERROR_FEATURE_UNSUPPORTED 200006	特性不支持。	请根据日志报错排查问题，或联系软件工程师。
#define SVP_ACL_ERROR_PROF_MODULES_UNSUPPORTED 200007	下发了不支持的 Profiling 配置。	请检查 Profiling 的配置是否正确。
#define SVP_ACL_ERROR_STORAGE_OVER_LIMIT 300000	超出存储上限。	请检查硬件环境上的存储剩余情况。
#define SVP_ACL_ERROR_INTERNAL_ERROR 500000	未知内部错误。	请根据日志报错排查问题，或联系软件工程师。
#define SVP_ACL_ERROR_FAILURE 500001	系统内部 SVP ACL 的错误。	请根据日志报错排查问题，或联系软件工程师。
#define SVP_ACL_ERROR_GE_FAILURE 500002	系统内部 GE 的错误。	请根据日志报错排查问题，或联系软件工程师。
#define SVP_ACL_ERROR_RT_FAILURE 500003	系统内部 RUNTIME 的错误。	请根据日志报错排查问题，或联系软件工程师。
#define SVP_ACL_ERROR_DRV_FAILURE 500004	系统内部 DRV (Driver) 的错误。	请根据日志报错排查问题，或联系软件工程师。
#define SVP_ACL_ERROR_PROFILING_FAILURE 500005	Profiling 相关错误。	请根据日志报错排查问题，或联系软件工程师。
#define SVP_ACL_ERROR_RT_REPERT_TIMEOUT 507012	获取 task 超时。	请根据日志报错排查问题，或联系软件工程师。

说明

返回码定义规则：

- 规则 1：开发人员的环境异常或者代码逻辑错误，可以通过优化环境或代码逻辑的方式解决问题，此时返回码定义为：1XXXXX。
- 规则 2：资源不足（Stream、内存等）、开发人员编程时使用的接口或参数与当前硬件不匹配，可以通过在编程时合理使用资源的方式解决，此时返回码定义为：2XXXXX。
- 规则 3：业务功能异常，比如队列满、队列空等，此时返回码定义为 3XXXXX。
- 规则 4：软硬件内部异常，包括软件内部错误、Device 执行失败等，用户无法解决问题，需要将问题反馈给软件工程师的，此时返回码定义为：5XXXXX。
- 规则 5：无法识别的错误，当前都映射为 500000。

9.12.2 svp_acl_data_type

```
typedef enum {
    SVP_ACL_DT_UNDEFINED = -1,    //未知数据类型，默认值。
    SVP_ACL_FLOAT = 0,
    SVP_ACL_FLOAT16 = 1,
    SVP_ACL_INT8 = 2,
    SVP_ACL_INT32 = 3,
    SVP_ACL_UINT8 = 4,
    SVP_ACL_INT16 = 6,
    SVP_ACL_UINT16 = 7,
    SVP_ACL_UINT32 = 8,
    SVP_ACL_INT64 = 9,
    SVP_ACL_UINT64 = 10,
    SVP_ACL_DOUBLE = 11,
    SVP_ACL_BOOL = 12,
    SVP_ACL_STRING = 13,
    SVP_ACL_INT10 = 100,
    SVP_ACL_UINT10 = 101,
    SVP_ACL_INT12 = 102,
    SVP_ACL_UINT12 = 103,
    SVP_ACL_INT14 = 104,
    SVP_ACL_UINT14 = 105,
    SVP_ACL_INT24 = 106,
    SVP_ACL_UINT24 = 107,
} svp_acl_data_type;
```

9.12.3 svp_acl_float16

该数据类型仅用于数据传递，不能用于加减乘除等运算。

```
typedef uint16_t svp_acl_float16;
```

9.12.4 svp_acl_format

```
typedef enum {
    SVP_ACL_FORMAT_UNDEFINED = -1,
    SVP_ACL_FORMAT_NCHW = 0,
    SVP_ACL_FORMAT_NHWC = 1,
    SVP_ACL_FORMAT_ND = 2,
    SVP_ACL_FORMAT_NC1HWC0 = 3,
    SVP_ACL_FORMAT_FRACTAL_Z = 4,
```

```
SVP_ACL_FORMAT_NC1HWC0_C04 = 12,
SVP_ACL_FORMAT_FRACTAL_NZ = 29,
} svp_acl_format;
```

- UNDEFINED：未知格式，默认值。
- ND：表示支持任意格式，仅有 Square、Tanh 等这些单输入对自身处理的算子外，其它需要慎用。
- NCHW：NCHW 格式。
- NHWC：NHWC 格式。
- NC1HWC0：自研的 5 维数据格式。其中，C0 与微架构强相关，该值等于 cube 单元的 size，例如 16；C1 是将 C 维度按照 C0 切分： $C1=C/C0$ ，若结果不整除，最后一份数据需要 padding 到 C0。
- FRACTAL_Z：卷积的权重的格式。
- NC1HWC0_C04：自研的 5 维数据格式。其中，C0 固定为 4，C1 是将 C 维度按照 C0 切分： $C1=C/C0$ ，若结果不整除，最后一份数据需要 padding 到 C0。当前版本不支持。
- FRACTAL_NZ：内部格式，用户目前无需使用。

9.12.5 svp_acl_rt_context

```
typedef void *svp_acl_rt_context;
```

9.12.6 svp_acl_rt_stream

```
typedef void *svp_acl_rt_stream;
```

9.12.7 svp_acl_rt_run_mode

```
typedef enum svp_acl_rt_run_mode {
    SVP_ACL_DEVICE, /* 软件栈运行在 Device 的 Control CPU 或板端环境上 */
    SVP_ACL_HOST,   /* 软件栈运行在 Host 侧 */
} svp_acl_rt_run_mode;
```

9.12.8 svp_acl_mdio_dims

```
#define SVP_ACL_MAX_DIM_CNT 128
#define SVP_ACL_MAX_TENSOR_NAME_LEN 128
typedef struct svp_acl_mdio_dims {
    char name[SVP_ACL_MAX_TENSOR_NAME_LEN]; /* tensor name */
    size_t dim_count; /* Shape 中的维度个数，如果为标量，则维度个数为 0 */
    int64_t dims[SVP_ACL_MAX_DIM_CNT]; /* 维度信息 */
} svp_acl_mdio_dims;
```

9.12.9 svp_acl_data_buffer

9.12.9.1 svp_acl_create_data_buffer

函数功能：创建 svp_acl_data_buffer 类型的数据，该数据类型用于描述内存地址、大小等内存信息。同步接口。

函数原型：

```
svp_acl_data_buffer *svp_acl_create_data_buffer(void *data, size_t size, size_t stride)
```

参数说明：

参数名	输入/输出	说明
data	输入	存放数据的内存地址。 该内存需由用户自行管理，调用 svp_acl_rt_malloc 接口/ svp_acl_rt_free 接口申请/释放内存，或调用 svp_acl_rt_malloc_host 接口/ svp_acl_rt_free_host 接口申请/释放内存。
size	输入	内存大小，单位 Byte。 size 大小：(0, 0xFFFFFFFF]。 可参照 svp_acl_mdl_get_input_size_by_index 或 svp_acl_mdl_get_output_size_by_index 中描述的计算方法计算获取。
stride	输入	跨度大小，用于逻辑快速跳转到下一行，要求 16 字节对齐，单位 Byte。 stride 大小：(0, 0xFFFFFFFF)。 可参照 svp_acl_mdl_get_input_default_stride 或 svp_acl_mdl_get_output_default_stride 中的计算方法计算获得。

返回值说明：返回 svp_acl_data_buffer 类型的指针。

9.12.9.2 svp_acl_destroy_data_buffer

函数功能：

- 销毁 `svp_acl_data_buffer` 类型的数据。同步接口。
- 此处仅销毁 `svp_acl_data_buffer` 类型的数据，调用 `svp_acl_create_data_buffer` 接口创建 `svp_acl_data_buffer` 类型数据时传入的 `data` 的内存需由用户自行释放。

函数原型：

```
svp_acl_error svp_acl_destroy_data_buffer(const svp_acl_data_buffer *data_buffer)
```

参数说明：

参数名	输入/输出	说明
data_buffer	输入	待销毁的 <code>svp_acl_data_buffer</code> 类型的指针。

返回值说明：返回 0 表示成功，返回非 0 表示失败。

9.12.9.3 svp_acl_get_data_buffer_addr

函数功能：获取 `svp_acl_data_buffer` 类型中的数据内存地址。同步接口。

函数原型：

```
void *svp_acl_get_data_buffer_addr(const svp_acl_data_buffer *data_buffer)
```

参数说明：

参数名	输入/输出	说明
data_buffer	输入	<code>svp_acl_data_buffer</code> 的地址。

返回值说明：返回 `svp_acl_data_buffer` 类型中的数据内存地址。

9.12.9.4 svp_acl_get_data_buffer_size

函数功能：获取 `svp_acl_data_buffer` 类型中数据的内存大小，单位 Byte。同步接口。

函数原型：

```
size_t svp_acl_get_data_buffer_size(const svp_acl_data_buffer *data_buffer)
```

参数说明：

参数名	输入/输出	说明
data_buffer	输入	svp_acl_data_buffer 的地址。

返回值说明：svp_acl_data_buffer 类型中数据的内存大小。

9.12.9.5 svp_acl_get_data_buffer_stride

函数功能：获取 svp_acl_data_buffer 类型中 stride 的大小，单位 Byte。同步接口。

函数原型：

```
size_t svp_acl_get_data_buffer_stride(const svp_acl_data_buffer *data_buffer)
```

参数说明：

参数名	输入/输出	说明
data_buffer	输入	svp_acl_data_buffer 的地址。

返回值说明：svp_acl_data_buffer 类型中 stride 的大小。

9.12.9.6 svp_acl_update_data_buffer

函数功能：

- 更新 svp_acl_data_buffer 中数据的内存及大小。同步接口。
- 更新 svp_acl_data_buffer 后，之前 svp_acl_data_buffer 中存放数据的内存如果不使用，需及时释放，否则可能会导致内存泄漏。

函数原型：

```
svp_acl_error svp_acl_update_data_buffer(svp_acl_data_buffer *data_buffer, void *data, size_t size, size_t stride)
```

参数说明：

参数名	输入/输出	说明
data_buffer	输入	svp_acl_data_buffer 的地址。
data	输入	存放数据的内存地址。 该内存需由用户自行管理，调用 svp_acl_rt_malloc 接口

参数名	输入/输出	说明
		/ svp_acl_rt_free 接口申请/释放内存, 或调用 svp_acl_rt_malloc_host 接口/ svp_acl_rt_free_host 接口申请/释放内存。
size	输入	内存大小, 单位 Byte。 size 大小: (0, 0xFFFFFFFF]。
stride	输入	跨度大小, 用于逻辑快速跳转到下一行, 需要 16 字节对齐, 单位 Byte。 stride 大小: (0, 0xFFFFFFFF)。 可参照 svp_acl_md1_get_input_default_stride 或 svp_acl_md1_get_output_default_stride 参照中的计算方法计算获得。

返回值说明: 返回 0 表示成功, 返回非 0 表示失败。

9.12.10 svp_acl_md1_dataset

9.12.10.1 svp_acl_md1_create_dataset

函数功能: 创建 `svp_acl_md1_dataset` 类型的数据, 该数据类型用于描述模型推理时的输入数据、输出数据, 模型可能存在多个输入、多个输出, 每个输入/输出的内存地址、内存大小用 [svp_acl_data_buffer](#) 类型的数据来描述。同步接口。

函数原型:

```
svp_acl_md1_dataset *svp_acl_md1_create_dataset()
```

参数说明: 无

返回值说明: 返回 `svp_acl_md1_dataset` 类型的指针。

9.12.10.2 svp_acl_md1_destroy_dataset

函数功能: 销毁 `svp_acl_md1_dataset` 类型的数据。同步接口。

函数原型:

```
svp_acl_error svp_acl_md1_destroy_dataset(const svp_acl_md1_dataset *dataset)
```

参数说明：

参数名	输入/输出	说明
dataset	输入	待销毁的 <code>svp_acl_md1_dataset</code> 类型的指针。

返回值说明：返回 0 表示成功，返回非 0 表示失败。

9.12.10.3 `svp_acl_md1_add_dataset_buffer`

函数功能：向 `svp_acl_md1_dataset` 中增加 `svp_acl_data_buffer`。同步接口。

约束说明：最多添加 256 个 `data_buffer`。

函数原型：

```
svp_acl_error svp_acl_md1_add_dataset_buffer(svp_acl_md1_dataset *dataset,
svp_acl_data_buffer *data_buffer)
```

参数说明：

参数名	输入/输出	说明
dataset	输出	待增加 <code>svp_acl_data_buffer</code> 的 <code>svp_acl_md1_dataset</code> 地址。
data_buffer	输入	待增加的 <code>svp_acl_data_buffer</code> 地址

返回值说明：返回 0 表示成功，返回非 0 表示失败。

9.12.10.4 `svp_acl_md1_get_dataset_num_buffers`

函数功能：获取 `svp_acl_md1_dataset` 中 `svp_acl_data_buffer` 的个数，同步接口。

函数原型：

```
size_t svp_acl_md1_get_dataset_num_buffers(const svp_acl_md1_dataset *dataset)
```

参数说明：

参数名	输入/输出	说明
dataset	输入	<code>svp_acl_md1_dataset</code> 地址。

返回值说明：svp_acl_data_buffer 的个数。

9.12.10.5 `svp_acl_md1_get_dataset_buffer`

函数功能：获取 `svp_acl_md1_dataset` 中的第 `n` 个 `svp_acl_data_buffer`。同步接口。

函数原型：

```
svp_acl_data_buffer *svp_acl_md1_get_dataset_buffer(const svp_acl_md1_dataset *dataset, size_t index)
```

参数说明：

参数名	输入/输出	说明
dataset	输入	svp_acl_md1_dataset 地址。
index	输入	表明获取的是第几个 svp_acl_data_buffer。

返回值说明：

- 获取成功，返回 `svp_acl_data_buffer` 的地址。
- 获取失败返回空地址。

9.12.11 `svp_acl_md1_desc`

9.12.11.1 `svp_acl_md1_create_desc`

函数功能：创建 `svp_acl_md1_desc` 类型的数据，表示模型描述信息。同步接口。

函数原型：

```
svp_acl_md1_desc *svp_acl_md1_create_desc()
```

参数说明：无

返回值说明：返回 `svp_acl_md1_desc` 类型的指针。

9.12.11.2 `svp_acl_md1_destroy_desc`

函数功能：销毁 `svp_acl_md1_desc` 类型的数据。同步接口。

函数原型：

```
svp_acl_error svp_acl_md1_destroy_desc(svp_acl_md1_desc *model_desc)
```

参数说明：

参数名	输入/输出	说明
model_desc	输入	待销毁的 <code>svp_acl_md1_desc</code> 类型的指针。

返回值说明：返回 0 表示成功，返回非 0 表示失败。

9.12.11.3 svp_acl_md1_get_desc

函数功能：根据模型 ID 获取该模型的模型描述信息。同步接口。

函数原型：

```
svp_acl_error svp_acl_md1_get_desc(svp_acl_md1_desc *model_desc, uint32_t model_id)
```

参数说明：

参数名	输入/输出	说明
model_desc	输出	<code>svp_acl_md1_desc</code> 类型数据的指针。
model_id	输入	模型 ID。 <code>svp_acl_md1_load_from_mem</code> 接口加载模型成功后，会返回模型 ID。

返回值说明：返回 0 表示成功，返回非 0 表示失败。

9.12.11.4 svp_acl_md1_get_num_inputs

函数功能：根据模型描述信息获取模型的输入个数。同步接口。

函数原型：

```
size_t svp_acl_md1_get_num_inputs(const svp_acl_md1_desc *model_desc)
```

参数说明：

参数名	输入/输出	说明
model_desc	输入	<code>svp_acl_md1_desc</code> 类型数据的指针。

返回值说明：模型的输入个数。

9.12.11.5 `svp_acl_md1_get_num_outputs`

函数功能：根据模型描述信息获取模型的输出个数。同步接口。

函数原型：

```
size_t svp_acl_md1_get_num_outputs(const svp_acl_md1_desc *modelDesc)
```

参数说明：

参数名	输入/输出	说明
model_desc	输入	<code>svp_acl_md1_desc</code> 类型数据的指针。

返回值说明：模型的输出个数。

9.12.11.6 `svp_acl_md1_get_input_size_by_index`

函数功能：根据模型描述信息获取指定输入的大小，单位为 Byte。同步接口。

函数原型：

```
size_t svp_acl_md1_get_input_size_by_index(const svp_acl_md1_desc *model_desc, size_t index)
```

参数说明：

参数名	输入/输出	说明
model_desc	输入	<code>svp_acl_md1_desc</code> 类型数据的指针。
index	输入	指定获取第几个输入的大小，index 值从 0 开始。

返回值说明：

- 返回指定输入单 batch 大小。单位是 Byte。
- 非 YVU 输入的 `input_size` = `d_0` * `d_1` * ... * `d_n-2` * `default_stride`;
- YUV420SP_U8 输入的 `input_size` = `d_0` * `d_i` * ... * `d_n-2` * `default_stride` / 2;
- YUV422SP_U8 输入的 `input_size` = `d_0` * `d_i` * ... * `d_n-2` * `default_stride` / 3 * 2;

- 其中 d_i 表示第 i 维长度, n 表示指定输入的维度个数, `default_stride` 为默认 stride(详细请参见 [svp_acl_md1_get_input_default_stride](#))。
- 实际使用中的 stride 可能不等于 `default_stride`, 实际输入大小 `actual_input_size` 可以通过 $\text{input_size} / \text{default_stride} * \text{actual_stride}$ 换算得到, 也可将上述的计算公式中的 `default_stride` 替换为 `actual_stride` 计算获得, 其中 `actual_input_size` 表示实际所需输入内存大小, `actual_stride` 表示实际 stride 大小。

9.12.11.7 svp_acl_md1_get_output_size_by_index

函数功能: 根据模型描述信息获取指定输出的大小, 单位为 Byte。同步接口。

函数原型:

```
size_t svp_acl_md1_get_output_size_by_index(const svp_acl_md1_desc *model_desc, size_t index)
```

参数说明:

参数名	输入/输出	说明
<code>model_desc</code>	输入	svp_acl_md1_desc 类型数据的指针。
<code>index</code>	输入	指定获取第几个输出的大小, <code>index</code> 值从 0 开始。

返回值说明:

- 返回指定输出的单 batch 大小。单位是 Byte。
- 返回 $\text{output_size} = d_0 * d_1 * \dots * d_{n-2} * \text{default_stride}$, 其中 d_i 表示第 i 维长度, n 表示指定输出的维度个数, `default_stride` 为默认 stride(详细请参见 [svp_acl_md1_get_output_default_stride](#))。
- 实际使用中的 stride 可能不等于 `default_stride`, 实际输出大小 `actual_output_size` 可以通过 $\text{output_size} / \text{default_stride} * \text{actual_stride}$ 换算得到, 也可将上述的计算公式中的 `default_stride` 替换为 `actual_stride` 计算获得。其中 `actual_output_size` 表示实际所需输出内存大小, `actual_stride` 表示实际 stride 大小。

9.12.11.8 svp_acl_md1_get_input_dims

函数功能: 根据模型描述信息获取模型的输入 tensor 的维度信息。同步接口。

函数原型:

```
svp_acl_error svp_acl_md1_get_input_dims(const svp_acl_md1_desc *model_desc, size_t index,
```

```
svp_acl_mdIo_dims *dims)
```

参数说明：

表9-2 图像输入维度定义规则

参数名	输入/输出	说明
model_desc	输入	svp_acl_mdIo_desc 类型数据的指针。
index	输入	指定获取第几个输入的 Dims， index 值从 0 开始。
dims	输出	获取输入维度信息。

图像格式	Format 参考格式	维度定义规则
YUV420SP_U8	NCHW	n,c,h,w, 其中 c=3
YUV422SP_U8	NCHW	n,c,h,w, 其中 c=3
XRGB_PACKAGE_U8	NCHW	n,c,h,w, 其中 c=1
XRGB_PLANAR_U8	NCHW	n,c,h,w, 其中 c=4
RGB_PACKAGE_U8	NCHW	n,h,w,c, 其中 c=1
RGB_PLANAR_U8	NCHW	n,c,h,w, 其中 c=3
YUV400_U8	NCHW	n,h,w,c, 其中 c=1

返回值说明：返回 0 表示成功，返回非 0 表示失败。

9.12.11.9 `svp_acl_mdIo_get_output_dims`

函数功能：根据模型描述信息获取指定的模型输出 tensor 的维度信息。同步接口。

函数原型：

```
svp_acl_error_t svp_acl_mdIo_get_output_dims(const svp_acl_mdIo_desc *model_desc, size_t index, svp_acl_mdIo_dims *dims)
```

参数说明：

参数名	输入/ 输出	说明
model_desc	输入	svp_acl_md1_desc 类型数据的指针。
index	输入	指定获取第几个输出的 Dims，index 值从 0 开始。
dims	输出	获取输出维度信息。

返回值说明：返回 0 表示成功，返回非 0 表示失败。

9.12.11.10 svp_acl_md1_get_input_name_by_index

函数功能：根据模型描述信息获取模型中指定输入的 bottom 名称。同步接口。

函数原型：

```
const char *svp_acl_md1_get_input_name_by_index(const svp\_acl\_md1\_desc *model_desc, size_t index)
```

参数说明：

参数名	输入/ 输出	说明
model_desc	输入	svp_acl_md1_desc 类型数据的指针。
index	输入	指定获取第几个输入的名称，index 值从 0 开始。

返回值说明：返回指定输入的 bottom 名称，task_buf 和 work_buf 返回的名称分别为“task_buf”和“work_buf”。

9.12.11.11 svp_acl_md1_get_output_name_by_index

函数功能：根据模型描述信息获取模型中指定输出的 top 名称。同步接口。

函数原型：

```
const char *svp_acl_md1_get_output_name_by_index(const svp\_acl\_md1\_desc *model_desc, size_t index)
```

参数说明：

参数名	输入/ 输出	说明
model_desc	输入	svp_acl_md1_desc 类型数据的指针。
index	输入	指定获取第几个输出的 top 名称, index 值从 0 开始。

返回值说明: 返回指定输出的 top 名称。

9.12.11.12 svp_acl_md1_get_input_format

函数功能: 根据模型描述信息获取模型中指定输入的 Format。同步接口。

函数原型:

```
svp_acl_format svp_acl_md1_get_input_format(const svp_acl_md1_desc *model_desc, size_t index)
```

参数说明:

参数名	输入/ 输出	说明
model_desc	输入	svp_acl_md1_desc 类型数据的指针。
index	输入	指定获取第几个输入的 Format, index 值从 0 开始。

返回值说明: 返回指定输入的 Format。

9.12.11.13 svp_acl_md1_get_output_format

函数功能: 根据模型描述信息获取模型中指定输出的 Format。同步接口。

函数原型:

```
svp_acl_format svp_acl_md1_get_output_format(const svp_acl_md1_desc *model_desc, size_t index)
```

参数说明:

参数名	输入/ 输出	说明
model_desc	输入	svp_acl_md1_desc 类型数据的指针。

参数名	输入/ 输出	说明
index	输入	指定获取第几个输出的 Format，index 值从 0 开始。

返回值说明：返回指定输出的 Format。

9.12.11.14 svp_acl_md1_get_input_data_type

函数功能：根据模型描述信息获取模型中指定输入的数据类型。同步接口。

函数原型：

```
svp_acl_data_type svp_acl_md1_get_input_data_type(const svp_acl_md1_desc *model_desc,
size_t index)
```

参数说明：

表9-3 图像输入数据类型定义规则

参数名	输入/输出	说明
model_desc	输入	svp_acl_md1_desc 类型数据的指针。
index	输入	指定获取第几个输入的数据类型，index 值从 0 开始。

图像格式	数据类型定义规则
YUV420SP_U8	SVP_ACL_UINT8
YUV422SP_U8	SVP_ACL_UINT8
XRGB_PACKAGE_U8	SVP_ACL_UINT32
XRGB_PLANAR_U8	SVP_ACL_UINT8
RGB_PACKAGE_U8	SVP_ACL_UINT24
RGB_PLANAR_U8	SVP_ACL_UINT8
YUV400_U8	SVP_ACL_UINT8

返回值说明：返回指定输入的数据类型。

9.12.11.15 `svp_acl_mdl_get_output_data_type`

函数功能：根据模型描述信息获取模型中指定输出的数据类型。同步接口。

函数原型：

```
svp_acl_data_type svp_acl_mdl_get_output_data_type(const svp_acl_mdl_desc *model_desc,
size_t index)
```

参数说明：

参数名	输入/输出	说明
model_desc	输入	svp_acl_mdl_desc 类型数据的指针。
index	输入	指定获取第几个输出的数据类型，index 值从 0 开始。

返回值说明：返回指定输出的数据类型。

9.12.11.16 `svp_acl_mdl_get_input_index_by_name`

函数功能：根据模型中的输入名称获取对应输入的索引编号。同步接口。

函数原型：

```
svp_acl_error svp_acl_mdl_get_input_index_by_name(const svp_acl_mdl_desc *model_desc,
const char *name, size_t *index)
```

参数说明：

参数名	输入/输出	说明
model_desc	输入	svp_acl_mdl_desc 类型数据的指针。
name	输入	指定输入名称。 获取 task_buf， work_buf 索引时，输入的名称分别为“task_buf”，“work_buf”。
index	输出	获取对应输入的索引编号。

返回值说明：返回 0 表示成功，返回非 0 表示失败。

9.12.11.17 `svp_acl_md1_get_output_index_by_name`

函数功能：根据模型中的输出名称获取对应输出的索引编号。同步接口。

函数原型：

```
svp_acl_error svp_acl_md1_get_output_index_by_name(const svp_acl_md1_desc *model_desc,
const char *name, size_t *index)
```

参数说明：

参数名	输入/输出	说明
model_desc	输入	svp_acl_md1_desc 类型数据的指针。
name	输入	指定输出名称。
index	输出	获取对应输出的索引编号。

返回值说明：返回 0 表示成功，返回非 0 表示失败。

9.12.11.18 `svp_acl_md1_get_input_default_stride`

函数功能：根据模型描述信息获取模型中指定输入的默认 stride。同步接口。

函数原型：

```
size_t svp_acl_md1_get_input_default_stride(const svp_acl_md1_desc *model_desc, size_t index)
```

参数说明：

参数名	输入/输出	说明
model_desc	输入	svp_acl_md1_desc 类型数据的指针。
index	输入	获取对应输入的索引编号。

返回值说明：

- 返回指定输入的默认 stride 大小。单位是 Byte。
- 默认 stride = $\text{ceil}(\text{ceil}(\text{last_dim} * \text{svp_acl_data_type_size}(\text{data_type}) / \text{byte_bit_num}) / \text{default_align_byte}) * \text{default_align_byte}$ 。

- 其中 `ceil()`函数表示向上取整；`last_dim` 表示指定输入的最后一维长度；`data_type` 表示指定输入的数据类型；`byte_bit_num` 表示一个字节所占 bit 数，数值为 8；`default_align_byte` 表示生成.om 时[`internal_stride`]项所配置的对齐字节数，数值大于 0 且为 16 的整数倍（倍数需为 2 的非负整数次幂）并且不超过 256。
- 如果实际使用中用户需要根据实际需要计算输入 stride，可将上述计算中 `default_align_byte` 替换成实际所需的对齐字节数 `actual_align_byte`，`actual_align_byte` 不能为 0 并且为 16 的整数倍。

9.12.11.19 `svp_acl_md5_get_output_default_stride`

函数功能：根据模型描述信息获取模型中指定输出的默认 stride。同步接口。

函数原型：

```
size_t svp_acl_md5_get_output_default_stride(const svp_acl_md5_desc *model_desc, size_t index)
```

参数说明：

参数名	输入/ 输出	说明
model_desc	输入	<code>svp_acl_md5_desc</code> 类型数据的指针。
index	输入	获取对应输出的索引编号。

返回值说明：

- 返回指定输出的默认 stride 大小。单位是 Byte。
- 默认 $stride = \text{ceil}(\text{ceil}(last_dim * \text{svp_acl_data_type_size}(data_type) / \text{byte_bit_num}) / \text{default_align_byte}) * \text{default_align_byte}$ 。
- 其中 `ceil()`函数表示向上取整；`last_dim` 表示指定输出的最后一维长度；`data_type` 表示指定输出的数据类型；`byte_bit_num` 表示一个字节所占 bit 数，数值为 8；`default_align_byte` 表示生成.om 时[`internal_stride`]项所配置的对齐字节数，数值为 16 的整数倍（倍数需为 2 的非负整数次幂）且不超过 256。
- 如果实际使用中用户需要根据实际需要计算输出 stride，可将上述计算中 `default_align_byte` 替换成实际所需的对齐字节数 `actual_align_byte`，`actual_align_byte` 不能为 0 并且为 16 的整数倍。

9.12.12 `svp_acl_prof_config`

9.12.12.1 `svp_acl_prof_create_config`

函数功能：

- 创建 `svp_acl_prof_config` 类型的数据，表示创建 Profiling 配置数据。同步接口。
- `svp_acl_prof_config` 类型数据可以只创建一次、多处使用，用户需要保证数据的一致性和准确性。
- 如需销毁 `svp_acl_prof_config` 类型的数据，请参见 [svp_acl_prof_destroy_config](#)。

函数原型：

```
svp_acl_prof_config *svp_acl_prof_create_config(const uint32_t *device_id_list, uint32_t device_num, svp_acl_prof_aacore_metrics aacore_metrics, const svp_acl_prof_aacore_events *aacore_events, uint64_t data_type_config)
```

参数说明：

参数名	输入/输出	说明
<code>device_id_list</code>	输入	Device ID 列表。
<code>device_num</code>	输入	Device 的个数。需由用户保证 <code>device_id_list</code> 中的 Device 个数与 <code>device_num</code> 参数值一致，否则可能会导致后续业务异常。
<code>aacore_metrics</code>	输入	表示模式识别 Core 性能指标采集项。只支持配置为 0。
<code>aacore_events</code>	输入	表示模式识别 Core 事件，目前配置为 NULL。
<code>data_type_config</code>	输入	用户选择如下多个宏进行逻辑或（例如：SVP_ACL_PROF_ACL_API SVP_ACL_PROF_AACORE_METRICS），作为 <code>data_type_config</code> 参数值。每个宏表示某一类性能数据，详细说明如下： • SVP_ACL_PROF_ACL_API：表示采集 ACL 接口的性能数据。

参数名	输入/输出	说明
		- SVP_ACL_PROF_TASK_TIME: 表示采集模式识别Core算子的执行时间。 - SVP_ACL_PROF_AACORE_METRICS: 表示采集模式识别Core性能指标数据, 逻辑或时必须包括该宏, aacore_metrics 入参处配置的性能指标采集项才有效。 - SVP_ACL_PROF_AACPU: 表示采集模式识别CPU任务的开始、结束轨迹数据。

返回值说明:

- 返回 `svp_acl_prof_config` 类型的指针, 表示成功。
- 返回 `nullptr`, 表示失败。

9.12.12.2 svp_acl_prof_destroy_config

函数功能: 销毁通过 `svp_acl_prof_create_config` 接口创建的 `svp_acl_prof_config` 类型的数据。同步接口。

函数原型:

```
svp_acl_error svp_acl_prof_destroy_config(const svp_acl_prof_config *profiler_config)
```

参数说明:

参数名	输入/输出	说明
<code>profiler_config</code>	输入	待销毁的 <code>svp_acl_prof_config</code> 类型的指针。

返回值说明: 返回 0 表示成功, 返回非 0 表示失败。

9.12.13 `svp_acl_prof_subscribe_config`

9.12.13.1 `svp_acl_prof_create_subscribe_config`

函数功能：

- 创建 `svp_acl_prof_subscribe_config` 类型的数据，表示创建订阅配置信息。同步接口。
- 如需销毁 `svp_acl_prof_subscribe_config` 类型的数据，请参见 [svp_acl_prof_destroy_subscribe_config](#)。

函数原型：

```
svp_acl_prof_subscribe_config *svp_acl_prof_create_subscribe_config(int8_t time_info_switch,  
svp_acl_prof_aacore_metrics aacore_metrics, const void *fd)
```

参数说明：

参数名	输入/输出	说明
<code>time_info_switch</code>	输入	是否采集网络模型中算子的性能数据： • 1：采集 • 0：不采集
<code>aacore_metrics</code>	输入	表示模式识别 Core 性能指标采集项。不支持配置。
<code>fd</code>	输入	用户创建的管道写指针。 指针 <code>fd</code> 指向的值范围不能小于 0。

返回值说明：

- 返回 `svp_acl_prof_subscribe_config` 类型的指针，表示成功。
- 返回 `nullptr`，表示失败。

9.12.13.2 `svp_acl_prof_destroy_subscribe_config`

函数功能：销毁通过 [svp_acl_prof_create_subscribe_config](#) 接口创建的 `svp_acl_prof_subscribe_config` 类型的数据。同步接口。

函数原型：

```
svp_acl_error svp_acl_prof_destroy_subscribe_config(const svp_acl_prof_subscribe_config
```

```
*prof_subscribe_config)
```

参数说明：

参数名	输入/输出	说明
prof_subscribe_config	输入	待销毁的 svp_acl_prof_subscribe_config 类型的指针。

返回值说明：返回 0 表示成功，返回非 0 表示失败。

9.12.14 svp_acl_mdل_config_attr

```
typedef enum {
    SVP_ACL_MDل_PRIORITY_INT32 = 0,    //模型执行的优先级，高优先级任务会优先调度，不支持高优先级任务抢占低优先级任务，数字越小优先级越高，取值[0,7]，可选项。默认值为 0。
    SVP_ACL_MDل_LOAD_TYPE_SIZEت,        //模型加载方式，见下文详细的取值说明，必选项。
    SVP_ACL_MDل_PATH_PTR,                //离线模型文件路径的指针，如果选择从文件加载模型，则该选项必选，目前不支持。
    SVP_ACL_MDل_MEM_ADDR_PTR,            //模型在内存中的起始地址，如果选择从内存加载模型，则该选项必选。
    SVP_ACL_MDل_MEM_SIZEت,                //模型在内存中的大小，如果选择从内存加载模型，则该选项必选，与 SVP_ACL_MDل_MEM_ADDR_PTR 选项配合使用。
    SVP_ACL_MDل_WEIGHT_ADDR_PTR,         //Device 上模型权值内存（存放权值数据）的指针，如果需要由用户管理权值内存，则该选项必选。若不配置该选项，则表示由系统管理内存，目前不支持。
    SVP_ACL_MDل_WEIGHT_SIZEت,            //权值内存大小，单位为 Byte，如果需要由用户管理权值内存，则该选项必选，与 SVP_ACL_MDل_WEIGHT_ADDR_PTR 选项配合使用，目前不支持。
    SVP_ACL_MDل_WORKSPACE_ADDR_PTR,      //Device 上模型所需工作内存（存放模型输入/输出等数据）的指针，如果需要由用户管理工作内存，则该选项必选。若不配置该选项，则表示由系统管理内存，目前不支持。
    SVP_ACL_MDل_WORKSPACE_SIZEت,         //模型所需工作内存的大小，单位为 Byte，如果需要由用户管理权值内存，则该选项必选，与 ACL_MDل_WORKSPACE_ADDR_PTR 选项配合使用，目前不支持。
    SVP_ACL_MDل_INPUTQ_NUM_SIZEت,        //模型输入队列大小，带队列加载模型时，该选项必选，与 ACL_MDل_INPUTQ_ADDR_PTR 选项配合使用。当前版本不支持该接口，目前不支持。
    SVP_ACL_MDل_INPUTQ_ADDR_PTR,         //模型输入队列 ID 的指针，带队列加载模型时，该选项必选，一个模型输入对应一个队列 ID。当前版本不支持该接口，目前不支持。
    SVP_ACL_MDل_OUTPUTQ_NUM_SIZEت,       //模型输出队列大小，带队列加载模型时，该选项必选，与 ACL_MDل_OUTPUTQ_ADDR_PTR 选项配置使用。当前版本不支持该接口，目前不支持。
    SVP_ACL_MDل_OUTPUTQ_ADDR_PTR         //模型输出队列 ID 的指针，带队列加载模型时，该选项必选，一个模型输出对应一个队列 ID。当前版本不支持该接口，目前不支持。
} svp_acl_mdل_config_attr;
```

SVP_ACL_MDل_LOAD_TYPE_SIZEت（表示模型加载方式）的取值使用如下宏：

- SVP_ACL_MDل_LOAD_FROM_FILE (目前不支持)

```
#define SVP_ACL_MDل_LOAD_FROM_FILE 1
```

- SVP_ACL_MDL_LOAD_FROM_FILE_WITH_MEM (目前不支持)

```
#define SVP_ACL_MDL_LOAD_FROM_FILE_WITH_MEM 2
```

- SVP_ACL_MDL_LOAD_FROM_MEM

```
#define SVP_ACL_MDL_LOAD_FROM_MEM 3
```

- SVP_ACL_MDL_LOAD_FROM_MEM_WITH_MEM (目前不支持)

```
#define SVP_ACL_MDL_LOAD_FROM_MEM_WITH_MEM 4
```

- SVP_ACL_MDL_LOAD_FROM_FILE_WITH_Q (目前不支持)

```
#define SVP_ACL_MDL_LOAD_FROM_FILE_WITH_Q 5
```

9.12.15 `svp_acl_mdl_config_handle`

9.12.15.1 `svp_acl_mdl_create_config_handle`

函数功能：创建 `svp_acl_mdl_config_handle` 类型的数据，表示一个模型加载的配置对象。同步接口。如需销毁 `svp_acl_mdl_config_handle` 类型的数据，请参见 `svp_acl_mdl_destroy_config_handle`。

函数原型：

```
svp_acl_mdl_config_handle *svp_acl_mdl_create_config_handle()
```

参数说明：无。

返回值说明：

- 返回 `svp_acl_mdl_config_handle` 类型的指针，表示成功。
- 返回 `nullptr`，表示失败。

9.12.15.2 `svp_acl_mdl_destroy_config_handle`

函数功能：销毁通过 [svp_acl_mdl_create_config_handle](#) 接口创建的 `svp_acl_mdl_config_handle` 类型的数据。同步接口。

函数原型：

```
svp_acl_error svp\_acl\_mdl\_destroy\_config\_handle (svp_acl_mdl_config_handle *handle)
```

参数说明：

参数名	输入/输出	说明
handle	输入	待销毁的 svp_acl_mdl_config_handle 类型的指针。

返回值说明：返回 0 表示成功，返回非 0 表示失败。

9.12.16 svp_acl_aapp_input_format

支持的图片详情如下：

```
typedef enum {
    SVP_ACL_YUV420SP_U8 = 0x1,
    SVP_ACL_XRGB8888_U8 = 0x2,
    SVP_ACL_RGB888_U8 = 0x3,
    SVP_ACL_YUV400_U8 = 0x4,
    SVP_ACL_NC1HWC0DI_FP16 = 0x5, /* not support */
    SVP_ACL_NC1HWC0DI_S8 = 0x6, /* not support */
    SVP_ACL_ARGB8888_U8 = 0x7, /* not support */
    SVP_ACL_YUYV_U8 = 0x8, /* not support */
    SVP_ACL_YUV422SP_U8 = 0x9,
    SVP_ACL_AYUV444_U8 = 0xA, /* not support */
    SVP_ACL_RAW10 = 0xB, /* not support */
    SVP_ACL_RAW12 = 0xC, /* not support */
    SVP_ACL_RAW16 = 0xD, /* not support */
    SVP_ACL_RAW24 = 0xE, /* not support */
    SVP_ACL_FEATURE_MAP_U8 = 0x100,
    SVP_ACL_FEATURE_MAP_S8 = 0x101,
    SVP_ACL_FEATURE_MAP_U16 = 0x102,
    SVP_ACL_FEATURE_MAP_S16 = 0x103,
    SVP_ACL_FEATURE_MAP_FP16 = 0x104,
    SVP_ACL_FEATURE_MAP_FP32 = 0x105,
    SVP_ACL_YVU420SP_U8 = 0x106,
    SVP_ACL_YVU422SP_U8 = 0x107,
    SVP_ACL_BGR_PLANAR_U8 = 0x108,
    SVP_ACL_RGB_PLANAR_U8 = 0x109,
    SVP_ACL_BGR_PACKAGE_U8 = 0x10A,
    SVP_ACL_XRGB_PLANAR_U8 = 0x10B,
    SVP_ACL_XBGR_PLANAR_U8 = 0x10C,
    SVP_ACL_RGBX_PLANAR_U8 = 0x10D,
    SVP_ACL_BGRX_PLANAR_U8 = 0x10E,
    SVP_ACL_XBGR_PACKAGE_U8 = 0x10F,
    SVP_ACL_RGBX_PACKAGE_U8 = 0x110,
    SVP_ACL_BGRX_PACKAGE_U8 = 0x111,
    SVP_ACL_AAPP_RESERVED = 0xFFFF
} svp_acl_aapp_input_format;
```

9.12.17 svp_acl_aapp_info

```
typedef struct svp_acl_aapp_info {
```

```
    svp\_acl\_aapp\_input\_format input_format;  
} svp\_acl\_aapp\_info;
```

10 SVP ACL 样例使用指导

10.1 总体说明

当前版本支持的样例以及样例在各形态的编译运行指导请参见[“3.5 如何获取 Sample”](#)。

10.2 基于 Caffe ResNet-50 网络实现图片分类（同步推理）

10.2.1 样例介绍

10.2.1.1 获取样例

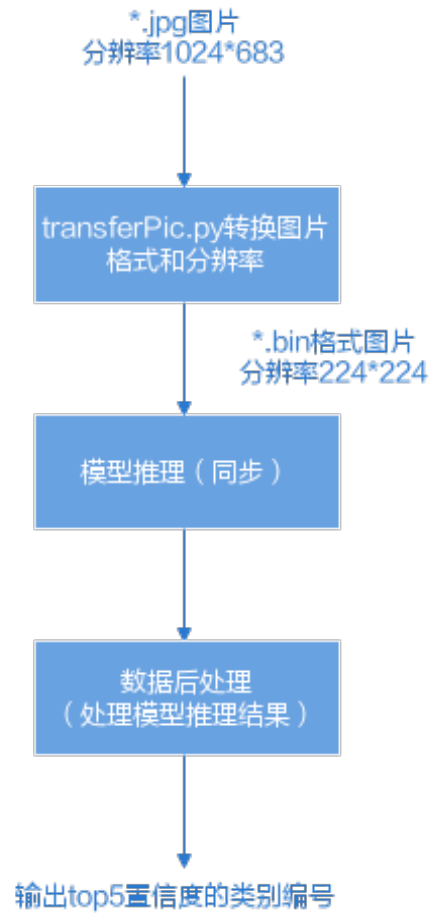
参考[“3.5 如何获取 Sample”](#)，获取样例代码。

10.2.1.2 功能描述

该样例主要是基于 Caffe ResNet-50 网络（单输入、单 Batch）实现图片分类的功能。

将 Caffe ResNet-50 网络的模型文件转换为适配 SoC 的离线模型（*.om 文件），在样例中，加载该 om 文件，对 2 张*.jpg 图片进行同步推理，分别得到推理结果后，再对推理结果进行处理，输出 top5 置信度的类别标识。

图10-1 Sample 示例



10.2.1.3 原理介绍

在该 Sample 中，涉及的关键功能点，如表 10-1 所示。API 接口的详细介绍请参见“9 SVP ACL API 参考”。

📖 说明

Device 表示板端环境上的图像分析引擎，SOC 场景下，当前只有一个 Device。

表10-1 Sample 中的关键功能点

初始化	- 调用 <code>svp_acl_init</code> 接口初始化 SVP ACL 配置。 - 调用 <code>svp_acl_finalize</code> 接口实现 SVP ACL 去初始化。
Device 管理	- 调用 <code>svp_acl_rt_set_device</code> 接口指定用于运算的 Device。 - 调用 <code>svp_acl_rt_get_run_mode</code> 接口获取 <code>svp_acl</code> 软件栈的运行模式，根据运行模式的不同，内部处理流程不同。 - 调用 <code>svp_acl_rt_reset_device</code> 接口复位当前运算的 Device，回收 Device 上的资源。
Context 管理	- 调用 <code>svp_acl_rt_create_context</code> 接口创建 Context。 - 调用 <code>svp_acl_rt_destroy_context</code> 接口销毁 Context。
Stream 管理	- 调用 <code>svp_acl_rt_create_stream</code> 接口创建 Stream。 - 调用 <code>svp_acl_rt_destroy_stream</code> 接口销毁 Stream。
内存管理	- 调用 <code>svp_acl_rt_malloc</code> 接口申请 Device 上的内存。 - 调用 <code>svp_acl_rt_free</code> 接口释放 Device 上的内存。
模型推理	- 调用 <code>svp_acl_mdload_from_mem</code> 接口从存放*.om 文件内存中加载模型。 - 调用 <code>svp_acl_mdexecute</code> 接口执行模型推理，同步接口。 - 调用 <code>svp_acl_mdunload</code> 接口卸载模型。
数据后处理	提供样例代码，在接口 <code>OutputModelResult</code> 中，处理模型推理的结果，直接在终端上显示 top5 置信度的类别编号。 另外，在接口 <code>DumpModelOutputResult</code> 中，将模型推理的结果写入文件（运行可执行文件后，推理结果文件在应用可执行文件的同级目录下）。 <pre>processModel.OutputModelResult(); processModel.DumpModelOutputResult();</pre>

10.2.1.4 目录结构

样例代码结构如下所示。

```
|— caffe_model
|   |— resnet50.caffemodel      //测试权重数据,需要按指导获取测试权重数据, 放到 caffe_model 目录下
|   |— resnet50.prototxt       //测试模型网络,需要按指导获取测试模型网络, 放到 caffe_model 目录下
```


		pixel_mean.txt	//均值文件，数据归一化使用
		data	
		dog1_1024_683.jpg	//测试数据,需要按指导获取测试图片，放到 data 目录下
		dog2_1024_683.jpg	//测试数据,需要按指导获取测试图片，放到 data 目录下
		image_ref_list.txt	//保存测试数据对应文件名的文件
		inc	
		model_process.h	//声明模型处理相关函数的头文件
		sample_process.h	//声明资源初始化/销毁相关函数的头文件
		utils.h	//声明公共函数（例如：文件读取函数）的头文件
		model	//存放转换后模型目录
		script	
		transferPic.py	//将*.jpg 转换为*.bin，同时将图片从 1024*683 的分辨率缩放为 224*224
		src	
		acl.json	//系统初始化的配置文件
		CMakeLists.txt	//编译脚本
		main.cpp	//主函数，图片分类功能的实现文件
		model_process.cpp	//模型处理相关函数的实现文件
		sample_process.cpp	//资源初始化/销毁相关函数的实现文件
		utils.cpp	//公共函数（例如：文件读取函数）的实现文件
		.project	//工程信息文件，包含工程类型、工程描述、运行目标设备类型等
		CMakeLists.txt	//编译脚本，调用 src 目录下的 CMakeLists 文件
		resnet50_config.json	//配置模型生成信息文件，MindStudio 工具转换模型使用
		README.md	

10.2.2 编译及运行应用

步骤 1 模型转换

1. 以运行用户登录开发环境。
2. 参考《ATC 工具使用指南》中的“使用入门”，执行“准备动作”，包括获取工具、设置环境变量。
3. 准备数据。

从 https://gitee.com/ascend/ModelZoo-TensorFlow/tree/master/TensorFlow/contrib/cv/resnet50/ATC_resnet50_caffe_AE

链接中获取 ResNet-50 网络的模型文件 (*.prototxt)、预训练模型文件 (*.caffemodel)，并以运行用户将获取的文件上传至开发环境的“样例目录 /caffe_model”目录下，也可查看 README.md。如果目录不存在，需要自行创建。

请从以下链接获取该样例的输入图片，并以运行用户将获取的文件上传至开发环境的“样例目录/data”目录下。如果目录不存在，需自行创建。

- https://c7xcode.obs.cn-north-4.myhuaweicloud.com/models/aclsample/dog1_1024_683.jpg
- https://c7xcode.obs.cn-north-4.myhuaweicloud.com/models/aclsample/dog2_1024_683.jpg

在“样例目录/data”目录下，创建参考图片列表文件（image_ref_list.txt）并在其中填写图片路径 data/ dog1_1024_683.jpg 及 data/ dog2_1024_683.jpg，各路径换行填写。

4. 将 ResNet-50 网络转换为适配 SoC 的离线模型（*.om 文件）。

切换到“样例目录”目录，执行如下命令：

```
atc --save_original_model="true" --soc_version=${soc_version} --log_level="0" --input_format="NCHW" --weight="./caffe_model/resnet50.caffemodel" --batch_num="256" --compile_mode="0" --image_list="data:/data/image_ref_list.txt" --input_shape="data:1,3,224,224" --online_model_type="0" --framework="0" --insert_op_conf="resnet50_insert_op.cfg" --dump_data="0" --output="./model/resnet50" --model="./caffe_model/resnet50.prototxt"
```

- --soc_version: SoC 的版本，用户根据具体板端环境形态，选择对应的取值。
- --insert_op_conf: 插入图像预处理的配置。配置了图像预处理的 Data 层表示输入数据格式为图像，没有配置的为 Feature Map，可参考《ATC 工具使用指南》配置。
- --output: 生成的 resnet50.om 文件存放在“样例目录/model”目录下，建议使用命令中的默认设置，否则在编译代码前，您还需要修改 sample_process.cpp 中的 omModelPath 参数值。

```
const char* omModelPath = "../model/resnet50.om";
```

关于各参数的详细解释，请参见《ATC 工具使用指南》。

步骤 2 编译代码

1. 以运行用户登录开发环境。
2. 设置环境变量，编译脚本 CMakeLists.txt 通过环境变量所设置的头文件、库文件的路径来编译代码。

如下为设置环境变量的示例，请将“\$HOME/acl”替换为 ACLlib 组件的实际安装路径。

注意：stub 路径中的库仅作为编译使用，在板端环境运行时请使用 SDK 包中的库。

```
export DDK_PATH=$HOME/acl/ascend-toolkit/svp_latest/x86_64-linux
export NNN_HOST_LIB=$HOME/acl/ascend-toolkit/svp_latest/x86_64-linux/acllib/lib64/stub
```

3. 切换到样例目录，创建目录用于存放编译文件，例如，本文中，创建的目录为“build/intermediates/soc”。

```
mkdir -p build/intermediates/soc
```

4. 切换到“build/intermediates/soc”目录，执行 **cmake** 生成编译文件。

“../../src”表示 CMakeLists.txt 文件所在的目录，请根据实际目录层级修改。

```
cd build/intermediates/soc
```

您需要根据所安装的交叉编译器，执行对应的 **cmake** 命令：

若在开发环境上安装的 ACLlib 软件包为 Linux 操作系统 SoC-cs 形态，则执行如下编译命令：

```
cmake ../../src -DCMAKE_CXX_COMPILER=aarch64-mix410-linux-g++ -  
DCMAKE_SKIP_RPATH=TRUE
```

5. 执行 **make** 命令，生成的可执行文件 main 在“样例目录/out”目录下。

```
make
```

步骤 3 准备测试数据。

1. 请参考步骤 1，在“样例目录/data”目录下，存放测试图片 dog1_1024_683.jpg 和 dog2_1024_683.jpg。
2. 以运行用户登录开发环境。
3. 切换到“样例目录/data”目录下，执行 transferPic.py 脚本，将*.jpg 转换为*.bin，同时将图片从 1024*683 的分辨率缩放为 224*224。在“样例目录/data”目录下生成 2 个*.bin 文件。

```
python3.7.5 ../script/transferPic.py
```

说明

如果执行脚本报错“ModuleNotFoundError: No module named 'PIL'”，则表示缺少 Pillow 库，请使用 **pip3.7.5 install Pillow --user** 命令安装 Pillow 库。

步骤 4 运行可执行文件。

1. 将开发环境的样例目录及目录下的文件上传到板端环境。

板端环境的操作系统为 Linux 时，以 root 用户将开发环境上的目录及其文件上传到板端环境的指定目录下，例如“/root/acl_resnet50”。

2. 登录板端环境。

板端环境的操作系统为 Linux 时，以 root 用户登录板端环境。

3. 设置环境变量。

板端环境的操作系统为 Linux 时，必须设置如下环境变量，并根据 libsvp_acl.so 和 libsvp_aacpu.so 文件所在的路径设置 LD_LIBRARY_PATH 环境变量。

```
export LD_LIBRARY_PATH=/home/demo/out/lib
```

4. 切换到可执行文件 main 所在的目录，例如“\$HOME/acl_resnet50/out”，给该目录下的 main 文件加执行权限。

```
chmod +x main
```

5. 切换到可执行文件 main 所在的目录，例如“\$HOME/acl_resnet50/out”，运行可执行文件。

```
./main
```

执行成功后，在屏幕上的关键提示信息示例如下，index 及其 value 会根据板端环境的实际情况有所不同：

```
[INFO] acl init success
[INFO] open device 0 success
[INFO] create context success
[INFO] create stream success
[INFO] load model ../model/resnet50.om success
[INFO] create model description success
[INFO] create model output success
[INFO] start to process file:../data/dog1_1024_683.bin
[INFO] model execute success
[INFO] top 1: index[161] value[0.900391]
[INFO] top 2: index[164] value[0.038666]
[INFO] top 3: index[163] value[0.019287]
[INFO] top 4: index[166] value[0.016357]
[INFO] top 5: index[167] value[0.012161]
[INFO] output data success
[INFO] start to process file:../data/dog2_1024_683.bin
[INFO] model execute success
[INFO] top 1: index[267] value[0.974609]
[INFO] top 2: index[266] value[0.013062]
[INFO] top 3: index[265] value[0.010017]
[INFO] top 4: index[129] value[0.000335]
[INFO] top 5: index[372] value[0.000179]
[INFO] output data success
[INFO] Unload model success, modelId is 1
[INFO] execute sample success
[INFO] end to destroy stream
[INFO] end to destroy context
[INFO] end to reset device is 0
```

----结束

10.3 基于 Caffe ResNet-50 网络实现图片分类（异步推理）

10.3.1 样例介绍

10.3.1.1 获取样例

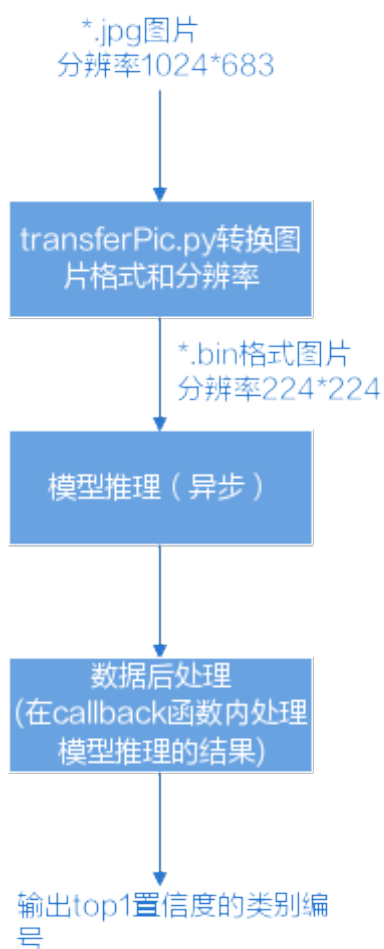
参考“3.5 如何获取 Sample”，获取样例代码。

10.3.1.2 功能描述

该样例主要是基于 Caffe ResNet-50 网络（单输入、单 Batch）实现图片分类的功能。

将 Caffe ResNet-50 网络的模型文件转换为适配 SoC 的离线模型（*.om 文件），在样例中，加载该 om 文件，对 2 张*.jpg 图片进行 n 次异步推理（n 作为运行应用的参数，由用户配置），分别得到 n 次推理结果后，再对推理结果进行处理，输出 top1 置信度的类别标识。

图10-2 Sample 示例



10.3.1.3 原理介绍

在该 Sample 中，涉及的关键功能点，如表 10-2 所示。API 接口的详细介绍请参见“9 SVP ACL API 参考”。

说明

Device 表示板端环境上的图像分析引擎，SOC 场景下，当前只有一个 Device。

表10-2 Sample 中的关键功能点

初始化	- 调用 <code>svp_acl_init</code> 接口初始化 SVP ACL 配置。 - 调用 <code>svp_acl_finalize</code> 接口实现 SVP ACL 去初始化。
Device 管理	- 调用 <code>svp_acl_rt_set_device</code> 接口指定用于运算的 Device。 - 调用 <code>svp_acl_rt_get_run_mode</code> 接口获取软件栈的运行模式，根据运行模式的不同，内部处理流程不同。 - 调用 <code>svp_acl_rt_reset_device</code> 接口复位当前运算的 Device，回收 Device 上的资源。
Context 管理	- 调用 <code>svp_acl_rt_create_context</code> 接口创建 Context。 - 调用 <code>svp_acl_rt_set_current_context</code> 接口设置线程的 Context。 - 调用 <code>svp_acl_rt_destroy_context</code> 接口销毁 Context。
Stream 管理	- 调用 <code>svp_acl_rt_create_stream</code> 接口创建 Stream。 - 调用 <code>svp_acl_rt_destroy_stream</code> 接口销毁 Stream。
内存管理	- 调用 <code>svp_acl_rt_malloc_host</code> 接口申请 Host 上内存。 - 调用 <code>svp_acl_rt_free_host</code> 释放 Host 上的内存。 - 调用 <code>svp_acl_rt_malloc</code> 接口申请 Device 上的内存。 - 调用 <code>svp_acl_rt_free</code> 接口释放 Device 上的内存。
模型推理	- 调用 <code>svp_acl_mdload_from_mem</code> 接口从*.om 文件加载模型。 - 创建新线程（例如 t1），在线程函数内调用 <code>svp_acl_rt_process_report</code> 接口，等待指定时间后，触发回调函数（例如 <code>CallBackFunc</code>，用于处理模型推理结果）。 - 调用 <code>svp_acl_rt_subscribe_report</code> 接口，指定处理 Stream 上回调函数（<code>CallBackFunc</code>）的线程（t1）。 - 调用 <code>svp_acl_mdload_execute_async</code> 接口执行模型推理，异步接口。 - 调用 <code>svp_acl_rt_launch_callback</code> 接口，在 Stream 的任务队列中增加一个需要在 Host/Device 上执行的回调函数（<code>CallBackFunc</code>）。 - 调用 <code>svp_acl_rt_synchronize_stream</code> 接口，阻塞应用程序运行，直到指定 Stream 中的所有任务都完成。 - 调用 <code>svp_acl_rt_unsubscribe_report</code> 接口，取消线程注册，

	Stream 上的回调函数（CallBackFunc）不再由指定线程（t1）处理。 模型推理结束后，调用 <code>svp_acl_md1_unload</code> 接口卸载模型。
数据后处理	提供样例代码，在接口 <code>OutputModelResult</code> 中，处理模型推理的结果，直接在终端上显示 top5 置信度的类别编号。 另外，在接口 <code>DumpModelOutputResult</code> 中，将模型推理的结果写入文件（运行可执行文件后，推理结果文件在应用可执行文件的同级目录下）。 <pre>processModel.OutputModelResult(); processModel.DumpModelOutputResult();</pre>

10.3.1.4 目录结构

样例代码结构如下所示。

└─ caffe_model	
└─ resnet50.caffemodel	//测试权重数据,需要按指导获取测试权重数据，放到 caffe_model 目录下
└─ resnet50.prototxt	//测试模型网络,需要按指导获取测试模型网络，放到 caffe_model 目录下
└─ pixel_mean.txt	//均值文件，数据归一化使用
└─ data	
└─ dog1_1024_683.jpg	//测试数据,需要按指导获取测试图片，放到 data 目录下
└─ dog2_1024_683.jpg	//测试数据,需要按指导获取测试图片，放到 data 目录下
└─ image_ref_list.txt	//保存测试数据对应文件名的文件
└─ inc	
└─ memory_pool.h	//声明内存池处理相关函数的头文件
└─ model_process.h	//声明模型处理相关函数的头文件
└─ sample_process.h	//声明资源初始化/销毁相关函数的头文件
└─ utils.h	//声明公共函数（例如：文件读取函数）的头文件
└─ model	//存放转换后模型目录
└─ script	
└─ transferPic.py	//将*.jpg 转换为*.bin，同时将图片从 1024*683 的分辨率缩放为 224*224
└─ src	
└─ acl.json	//系统初始化的配置文件
└─ CMakeLists.txt	//编译脚本
└─ main.cpp	//主函数，图片分类功能的实现文件
└─ memory_pool.cpp	//内存池处理相关函数的实现文件
└─ model_process.cpp	//模型处理相关函数的实现文件
└─ sample_process.cpp	//资源初始化/销毁相关函数的实现文件
└─ utils.cpp	//公共函数（例如：文件读取函数）的实现文件


```

├── .project      //工程信息文件，包含工程类型、工程描述、运行目标设备类型等
├── CMakeLists.txt //编译脚本，调用 src 目录下的 CMakeLists 文件
├── resnet50_config.json //配置模型生成信息文件，MindStudio 工具转换模型使用
└── README.md

```

10.3.2 编译及运行应用

步骤 1 模型转换。

1. 以运行用户登录开发环境。
2. 参考《ATC 工具使用指南》中的“使用入门”，执行“准备动作”，包括获取工具、设置环境变量。
3. 准备数据。

从 https://gitee.com/ascend/ModelZoo-TensorFlow/tree/master/TensorFlow/contrib/cv/resnet50/ATC_resnet50_caffe_AE 链接中获取 ResNet-50 网络的模型文件 (*.prototxt)、预训练模型文件 (*.caffemodel)，并以运行用户将获取的文件上传至开发环境的“样例目录/caffe_model”目录下，也可查看 README.md。如果目录不存在，需要自行创建。

请从以下链接获取该样例的输入图片，并以运行用户将获取的文件上传至开发环境的“样例目录/data”目录下。如果目录不存在，需自行创建。

- https://c7xcode.obs.cn-north-4.myhuaweicloud.com/models/aclsample/dog1_1024_683.jpg
- https://c7xcode.obs.cn-north-4.myhuaweicloud.com/models/aclsample/dog2_1024_683.jpg

在“样例目录/data”目录下，创建参考图片列表文件 (image_ref_list.txt) 并在其中填写图片路径 data/ dog1_1024_683.jpg 及 data/ dog2_1024_683.jpg，各路径换行填写。

4. 将 ResNet-50 网络转换为适配 SoC 的离线模型 (*.om 文件)。

切换到“样例目录”目录，执行如下命令：

```

atc --save_original_model="true" --soc_version=${soc_version} --log_level="0" --input_format="NCHW" --
weight="./caffe_model/resnet50.caffemodel" --batch_num="256" --compile_mode="0" --
image_list="data:/data/image_ref_list.txt" --input_shape="data:1,3,224,224" --online_model_type="0" --
framework="0" --insert_op_conf="resnet50_insert_op.cfg" --dump_data="0" --output="./model/resnet50" --
model="./caffe_model/resnet50.prototxt"

```

- --soc_version: SoC 的版本，用户根据具体板端环境形态，选择对应的取值。

- `--insert_op_conf`: 插入图像预处理的配置。配置了图像预处理的 Data 层表示输入数据格式为图像，没有配置的为 Feature Map，可参考《ATC 工具使用指南》配置。
- `--output`: 生成的 `resnet50.om` 文件存放在“样例目录/model”目录下，建议使用命令中的默认设置，否则在编译代码前，您还需要修改 `sample_process.cpp` 中的 `omModelPath` 参数值。

```
const char* omModelPath = "../model/resnet50.om";
```

关于各参数的详细解释，请参见《ATC 工具使用指南》。

步骤 2 编译代码。

1. 以运行用户登录开发环境。
2. 设置环境变量，编译脚本 `CMakeLists.txt` 通过环境变量所设置的头文件、库文件的路径来编译代码。

如下为设置环境变量的示例，请将“\$HOME/acl”替换为 ACLlib 组件的实际安装路径。

注意：stub 路径中的库仅作为编译使用，在板端环境运行时请使用 SDK 包中的库。

```
export DDK_PATH=$HOME/acl/ascend-toolkit/svp_latest/x86_64-linux
export NNN_HOST_LIB=$HOME/acl/ascend-toolkit/svp_latest/x86_64-linux/acllib/lib64/stub
```

3. 切换到样例目录，创建目录用于存放编译文件，例如，本文中，创建的目录为“`build/intermediates/soc`”。

```
mkdir -p build/intermediates/soc
```

4. 切换到“`build/intermediates/soc`”目录，执行 **cmake** 生成编译文件。

“`../././src`”表示 `CMakeLists.txt` 文件所在的目录，请根据实际目录层级修改。

```
cd build/intermediates/soc
```

您需要根据所安装的交叉编译器，执行对应的 **cmake** 命令：

若在开发环境上安装的 ACLlib 软件包为 Linux 操作系统 SoC-cs 形态，则执行如下编译命令：

```
cmake ../././src -DCMAKE_CXX_COMPILER=aarch64-mix410-linux-g++ -
DCMAKE_SKIP_RPATH=TRUE
```

5. 执行 **make** 命令，生成的可执行文件 `main` 在“样例目录/out”目录下。

```
make
```

步骤 3 准备测试数据。

1. 请参考步骤 1，在“样例目录/data”目录下，存放测试图片 `dog1_1024_683.jpg` 和 `dog2_1024_683.jpg`

2. 以运行用户登录开发环境。
3. 切换到“样例目录/data”目录下，执行 transferPic.py 脚本，将*.jpg 转换为*.bin，同时将图片从 1024*683 的分辨率缩放为 224*224。在“样例目录/data”目录下生成 2 个*.bin 文件。

```
python3.7.5 ../script/transferPic.py
```

说明

如果执行脚本报错“ModuleNotFoundError: No module named 'PIL'”，则表示缺少 Pillow 库，请使用 **pip3.7.5 install Pillow --user** 命令安装 Pillow 库。

步骤 4 运行可执行文件。

1. 将开发环境的样例目录及目录下的文件上传到板端环境。

板端环境的操作系统为 Linux 时，以 root 用户将开发环境上的目录及其文件上传到板端环境的指定目录下，例如“/root/acl_resnet50_async”。

2. 登录板端环境。

板端环境的操作系统为 Linux 时，以 root 用户登录板端环境。

3. 设置环境变量。

板端环境的操作系统为 Linux 时，必须设置如下环境变量，并根据 libsvp_acl.so 和 libsvp_aacpu.so 文件所在的路径设置 LD_LIBRARY_PATH 环境变量。

```
export LD_LIBRARY_PATH=/home/demo/out/lib
```

4. 切换到可执行文件 main 所在的目录，例如“\$HOME/acl_resnet50_async/out”，给该目录下的 main 文件加执行权限。

```
chmod +x main
```

5. 切换到可执行文件 main 所在的目录，例如“\$HOME/acl_resnet50_async/out”，运行可执行文件。

- 运行可执行文件，不带参数时：

- 执行模型异步推理的次数默认为 10 次；
- callback 间隔默认为，表示 1 次异步推理后，下发一次 callback 任务；
- 内存池中的内存块的个数默认为 10 个。

- 运行可执行文件，带参数时：

- 第一个参数表示执行模型异步推理的次数；
- 第二个参数表示下发 callback 间隔，参数值为 0 时表示不下发 callback 任务，参数值为非 0 值（例如 m）时表示 m 次异步推理后下发一次 callback 任务；

- 第三个参数表示内存池中内存块的个数，内存块个数需大于等于模型异步推理的次数。

```
./main
```

执行成功后，在屏幕上的关键提示信息示例如下，index 及其 value 会根据板端环境的实际情况有所不同：

备注：日志中出现“..... get callback task failed”，是因为执行过程中推理还没有开始执行就起线程进行 callback 处理，此为正常现象。

```
[INFO] ./main param1 param2 param3, param1 is execute model times(default 10), param2 is
callback interval(default 1), param3 is memory pool size(default 10)
[INFO] execute times = 10
[INFO] callback interval = 1
[INFO] memory pool size = 10
[INFO] acl init success
[INFO] open device 0 success
[INFO] create context success
[INFO] create stream success
[INFO] get run mode success
[INFO] load model ../model/resnet50.om success
[INFO] create model description success
[INFO] init memory pool success
[INFO] subscribe report success
[INFO] top 1: index[267] value[0.889648]
[INFO] top 1: index[161] value[0.836914]
[INFO] top 1: index[267] value[0.889648]
[INFO] top 1: index[161] value[0.836914]
[INFO] top 1: index[161] value[0.836914]
[INFO] top 1: index[161] value[0.836914]
[INFO] top 1: index[267] value[0.889648]
[INFO] top 1: index[161] value[0.836914]
[INFO] top 1: index[267] value[0.889648]
[INFO] top 1: index[161] value[0.836914]
[INFO] top 1: index[267] value[0.889648]
[INFO] top 1: index[161] value[0.836914]
[INFO] top 1: index[267] value[0.889648]
[INFO] top 1: index[267] value[0.889648]
.....
[INFO] top 1: index[161] value[0.836914]
[INFO] top 1: index[267] value[0.889648]
[INFO] model execute success
[INFO] unsubscribe report success
[INFO] unload model success, modelId is 1
[INFO] execute sample success
[INFO] end to destroy stream
[INFO] end to destroy context
[INFO] end to reset device is 0
[INFO] end to finalize acl
```

----结束

11 FAQ

11.1 SS928V100 解决方案使用 SVP_NNN 模块注意事项

hnr 与 svp_nnn 互斥：

1. 两者不能同时使用，所依赖的 ko (ot_pqp.ko/ot_svp_nnn.ko) 为互斥，不能同时插入
2. 当前版本默认都没有加载 ko,使用 svp_nnn 的进需要手动更改加载脚本，或者手动 insmod ot_svp_nnn.ko。如果 ot_pqp.ko 已经加载，需要 rmmod ot_pqp.ko，然后加载 ot_svp_nnn.ko

11.2 SVP_NNN 离线模型保护

当前 svp_nnn 中未提供 OM 的完整性与机密性保护机制，需客户对 OM 的完整性与机密性进行保护。